



Safe by Construction: Schema-Bounded Action Generation and Rust Orchestration for Real-Time Immersive Speech Programming in VR

By

Sandeep Rai

Student ID: sxr1156

Supervisors: Dr Daniele Giunchi and Dr Eyal Ofek

A dissertation submitted to the University of Birmingham
for the degree of **MSc in Cyber Security**

Word count: ~24,500 (main body, Chapters 1–9)

School of Computer Science
University of Birmingham

2026

Declaration

I declare that this dissertation is my own original work and that it has been undertaken solely by me. It has not been submitted, in whole or in part, for any other degree or qualification at this or any other institution, and all sources of information and prior work have been properly acknowledged and referenced. In accordance with the University of Birmingham's policy on the use of generative artificial intelligence, I disclose that AI tooling was used, under my own direction, to help implement parts of the software pipeline and to assist with drafting and editing; the research design, the architecture and safety decisions, the threat model, and the interpretation of results are my own, and every quantitative result reported herein was produced by code I directed and was independently inspected and verified by me against the system's logged outputs and automated test results. A fuller AI-assistance statement is given in Appendix G.

Acknowledgements

I thank my supervisors, Dr Daniele Giunchi and Dr Eyal Ofek, for their guidance and for framing the safety and privacy questions that shaped this work; Daniele Giunchi and colleagues at University College London for the original *DreamCodeVR* system on which this dissertation builds; and the School of Computer Science at the University of Birmingham for its support. *(The author may extend this section.)*

Table of Contents

Declaration	
Acknowledgements	
Table of Contents	
List of Figures and Tables	
Abstract	
Chapter 1 — Introduction	6
1.1 Motivation: end-user programming in immersive VR	6
1.2 The original DreamCodeVR	6
1.3 Problem statement	6
1.4 Proposed approach	7
1.5 Research questions	7
1.6 Aim and objectives	7
1.7 Contributions	8
1.8 Dissertation structure	11
Chapter 2 — Background & Literature Review	12
2.1 Speech and natural-language programming	12
2.2 LLM code generation and its risks	12
2.3 Prompt injection and standardised threat taxonomies	13
2.4 Constrained, schema-bounded and grammar-constrained generation	13
2.5 Immersive systems and Ubiq networking	13
2.6 Runtime code execution in Unity (Roslyn / IL2CPP / AOT)	13
2.7 Container and kernel isolation	14
2.8 Rust for secure systems	14
2.9 Privacy in voice and VR, and DPIA obligations	15

2.10 Gap statement and comparison	15
2.11 XR perceptual and embodied attacks: the second safety dimension	16
Chapter 3 — Analysis of the Original DreamCodeVR System (the Mode-A Baseline)	18
3.1 Purpose and scope	18
3.2 The original pipeline, traced end to end	18
3.3 Weakness 1 — Runtime arbitrary code execution on the host/headset	19
3.4 Weakness 2 — Confined to sideloaded 32-bit Quest builds; not Store-publishable or forward-compatible	19
3.5 Weakness 3 — Missing validation gate and global isGenerating serialisation	20
3.6 Weakness 4 — A dead supply-chain dependency (observed)	20
3.7 Implications for the redesign	21
Chapter 4 — Requirements, Threat Model and Design Goals	22
4.1 Functional requirements	22
4.2 Non-functional requirements	22
4.3 Assets, adversaries and trust boundaries	23
4.4 Design goals	24
4.5 Accepted and deferred risks	25
Chapter 5 — Architecture & Implementation	26
5.1 Workspace shape and the four modes	26
5.2 The networking layer	26
5.3 The action-plan IR	27
5.4 The fail-closed validator	27
5.5 Orchestration	27
5.6 External client seams	28
5.7 Mode B: validated C	29
5.8 Mode D: the sandbox	30
5.9 The Unity drop-in	30
5.10 Quality gates	30
5.11 Verification status	30
5.12 Personalization (retrieval-augmented generation), safety-preserving	31
5.13 Live observability and runtime control (the admin panel)	31
Chapter 6 — Evaluation Methodology	33
6.1 Evaluation design	33
6.2 How each claim was evaluated	33
6.3 Metrics	34
6.4 Planned but not-yet-run evaluations	34
6.5 Threats to validity	35
Chapter 7 — Results & Analysis	36
7.1 The test gate	36
7.2 Live Ubiq round-trip (Mode C path) — verified	36
7.3 Mode B (validated C#) — verified, with a defence-in-depth finding	36
7.4 Mode D containment — verified under Docker	37

7.5 Unity 6 execution — Mode C and Mode A verified live (no Quest)	37
7.6 Results-status table (mirroring the evidence table)	38
7.7 Planned quantitative evaluation (not yet conducted)	40
7.8 Analysis	41
7.9 Coverage of the XR threat literature (implemented vs outstanding)	41
7.10 Adversarial red-team of the C# validator and the intent screen (RQ3) — verified	43
7.11 On-device Quest deployment on the university network — verified live	46
7.12 Age-adaptive dual-plane safety coupling — verified (test)	47
7.13 On-device voice age-band gate — verified (test)	48
7.14 Continuous ML attack analyzer — verified (test)	49
7.15 Whole-system verification campaign: seven latent defects, and a durable regression harness — executed	49
Chapter 8 — Discussion, Limitations, Ethics & Privacy	52
8.1 Interpreting the central thesis	52
8.2 Limitations	52
8.3 Ethics, privacy and responsible disclosure	54
Chapter 9 — Conclusion & Future Work	55
9.1 Contributions	55
9.2 Answering the research questions	55
9.3 Future work (prioritised)	56
9.4 Closing statement	57
References and Appendices	58
References	12
Appendices	58

List of Figures and Tables

Figures and tables are numbered within their chapters.

Abstract

Immersive speech programming lets people with no coding background shape behaviour inside virtual reality (VR) simply by speaking. The original *DreamCodeVR* system (Giunchi et al., 2024), built at University College London on the Ubiq social-VR platform, realises this by translating speech into C# that is **compiled and executed at runtime** inside Unity. This is expressive but unsafe — it runs effectively arbitrary, model-generated code on the host or headset with no validation gate — and although it runs on a Meta Quest 1/2 only as a manually **sideloaded 32-bit Mono build**, it **cannot be published to the Quest Store or run on Quest 3**, whose required IL2CPP/AOT ARM64 backend forbids runtime C# compilation (Unity Technologies, 2024a).

This dissertation presents **DreamCodeVR+**, which preserves the original generate-and-compile pipeline as a controlled baseline (Mode A) and replaces the *default* execution path with a **safe-by-construction** alternative. Speech is translated into a small, allow-listed *action plan* — a six-action JSON intermediate representation with hard numeric bounds — that a tiny, audited Unity executor runs **without compiling any code** (Mode C), making it Quest-compatible by construction. A new **Rust** orchestration backend joins the existing Ubiq network as a service peer and enforces a **fail-closed** validator that approves a plan only when it contains zero violations. Two

further research arms enable comparison: a Rust-lexical-plus-Roslyn **validated-C#** path (Mode B) and a hardened **Docker/gVisor sandbox** for untrusted generated C# (Mode D).

The backend has been live-verified end-to-end against the real Ubiq RoomServer, a real OpenAI large-language model, a real .NET Roslyn service, and real Docker containment (the gVisor `runsc` variant is wired and unit-tested but was not exercised — kernel-level isolation is a designed, not a measured, property); the Rust workspace is `unsafe`-free and panic-averse by compiler-enforced lint policy, passing 342 automated tests. The Unity client has since been verified live in **Unity 6**: both the safe action-plan executor (Mode C) and — newly — the original runtime-C# path (Mode A, now *gated by the validator*) drive scene changes from live backend messages, including model-generated C# that **compiles at runtime to build whole structures (e.g. a house) under guardrails that block malicious or over-large code** — demonstrating that the original pipeline can be made safe rather than abandoned. Two further additions complete the system: a safety-preserving **personalization (RAG)** layer that adapts the *aesthetic* of generated output to a user's taste yet remains subordinate to the validator (its retrieved context is treated as untrusted), and a live **web admin panel** that streams every pipeline stage and red-teams the guardrails on demand, making the system's safety behaviour transparent and reproducible. **On-device, live-verified**: on-device deployment has since been **demonstrated live on a physical Meta Quest** on the university network — a command spoken into the headset microphone was captured, transcribed, generated, validated, and compiled at runtime on-device — for the **Quest 1/2 32-bit Mono sideload** (Mode A); the **Quest 3 IL2CPP/Store** build remains future work. An **executed adversarial red-team** hardened the C# lexical validator by closing three novel evasion classes (namespace aliasing, Unicode-escape identifiers, and dynamic late-binding); a reproducible **1,057-vector** campaign then *confirmed* that validator at **258/258** (100%, with all 469 benign inputs allowed) and, on the natural-language surface, hardened the two-layer intent screen from **81.71%** (460 of 563 attack vectors counted across *both* layers — the screen alone caught 202/305, leaving **103 plain-English bypasses** its technical-keyword table missed) to **100%** (563 of 563), with zero over-blocks, by adding a natural-language *paraphrase* layer. These post-hardening figures are **in-sample** (the same hand-authored corpus located the bypasses and validated the fixes); the intent screen itself remains an advisory, partly fail-open pre-filter, and the fail-closed validator is the authoritative gate. An independently-authored holdout corpus is identified as future work. In a separate static code-admission benchmark of five XR attack classes, the fully-hardened system blocks **38 of 40 (95%)** of the attacks while preserving **12 of 12** benign creative commands (0 over-blocked). A whole-system verification campaign — checking each feature by its downstream *effect* rather than merely its response — additionally uncovered and fixed **seven latent defects**, among them two safety controls that silently did nothing, and is captured as a one-command regression harness with a drift-detecting baseline; and a from-scratch **Rust reimplementation of the Ubiq RoomServer** reaches stock-Ubiq protocol parity and remains available on a malformed control message that crashes the genuine UCL Node.js RoomServer. A four-mode differential evaluation of functionality, latency, security and privacy is specified; remaining quantitative results and any human-subjects study are identified as future work pending data collection. The work argues that **bounding the output language is a stronger and more deployable safety lever than sandboxing the output** — while showing the unbounded path can itself be validator-gated when expressiveness is required. A final research thrust adds an **age-adaptive dual-plane safety coupling** — an opt-in, on-device voice *age-band* gate (child / teen / adult, never a precise age and never affect) whose detection of a minor flips the code-safety validator to its hardened profile — the age-to-**code**-plane coupling is built, wired, and tested, while the parallel age-tightening of the **perceptual** plane is specified and unit-tested at the configuration layer, with its wiring into runtime perceptual enforcement identified as future work — turning age inference from a surveillance signal into a protection one — supported by two reproducible, numpy-only ML models (a calibrated age-band gate and a continuous attack analyzer that red-teams that gate), with the platform Meta Quest age API kept authoritative and on-device microphone audio identified as future work.

Chapter 1 — Introduction

1.1 Motivation: end-user programming in immersive VR

Virtual reality promises authoring environments in which people build and modify interactive worlds from inside them, rather than at a desktop through an engine's scripting interface. For the large population of non-programmers, the natural authoring interface is speech: "*make this cube red*," "*spin it slowly*," "*give it gravity*." Lowering the barrier to behaviour authoring is an active goal of end-user development (EUD) research for immersive systems, where tools have historically demanded C# or engine expertise and so excluded the very novices that VR's accessibility might otherwise empower (Yigitbas et al., 2022). Decades of programming-by-voice research further show that speech is a viable but error-prone modality for code entry, and that novices tend to express *intent* rather than syntax (Nowrin and Vertanen, 2023) — which makes an interface that maps spoken intent onto a small, validated behaviour vocabulary an attractive design point. Recent large language models (LLMs) make such mapping practical: they can synthesise programs and structured output from natural language, albeit with a well-documented reliability ceiling and systematic failure modes (Chen et al., 2021). Immersive speech programming therefore sits at a productive intersection of accessibility, voice interaction, and LLM-driven synthesis — but, as the next sections argue, the way that synthesis is *executed* determines whether the result is safe and deployable.

1.2 The original DreamCodeVR

The starting point for this work is *DreamCodeVR* (Giunchi et al., 2024), developed at University College London on top of the Ubiq social-VR platform and its Ubiq-Genie backend framework. In *DreamCodeVR* a user holds a controller trigger to record an utterance and points a ray at a target object; the spoken request is sent to a Node.js server that transcribes it with a faster-whisper speech-to-text (STT) service, prompts an LLM to **generate C#**, and ships that code back to Unity, where the `RoslynCSharp` library (Trivial Interactive, no date) **compiles and runs it at runtime** to mutate the scene. The result is a compelling demonstration of democratising VR behaviour design through speech-driven programming. *DreamCodeVR+* preserves this artefact unchanged and treats it as the baseline against which a safer, deployable redesign is measured.

1.3 Problem statement

The original execution model has four concrete defects that this dissertation treats as central.

(i) Unsafe runtime arbitrary code execution. The system executes model-generated code, with no validation gate between generation and execution, on the user's machine or headset. Running program text derived from untrusted input is the classic code-injection / remote-code-execution pattern, granting that input the privileges of the host process (OWASP, no date a). Because an LLM's output can be steered by a prompt-injected or simply erroneous request (OWASP Gen AI Security Project, 2025a; Greshake et al., 2023), the generated C# can in principle reach arbitrary file, network, and process APIs.

(ii) Meta Quest Store / Quest-3 IL2CPP/AOT incompatibility. The *Quest Store*, and the 64-bit ARM64 target that Quest 3 and current Horizon OS require, use Unity's IL2CPP scripting backend, which compiles C# ahead of time (AOT) and provides **no runtime C# compiler** and no support for dynamic code generation (Unity Technologies, 2024a; 2024b). The runtime-compilation step that `RoslynCSharp` depends on therefore cannot exist under IL2CPP/ARM64 — so the feature is confined to the sideloaded 32-bit **Mono** Quest 1/2 path (where, as §3.4 documents from the project's own `ProjectSettings.asset`, it does run) and cannot ship through the Store or on Quest 3.

(iii) **Unvalidated LLM output.** There is no schema, allow-list, or fail-closed check mediating between the model and execution; whatever the model emits is what runs.

(iv) **A dead supply-chain dependency.** The original server pins the `ubiq` package to a `gitpkg` host URL (`Server/package.json`), which — observed and dated during this project's setup — now returns `402 Payment Required`, so the server cannot be installed as published until the `RoomServer` is vendored manually.

1.4 Proposed approach

DreamCodeVR+ responds not by hardening the execution of model-generated code but by **changing what is generated**. The LLM is asked to emit a schema-bounded *action plan* — a small JSON intermediate representation (IR) whose only legal instructions are six allow-listed behaviours (`set_color`, `set_scale`, `move`, `rotate`, `spawn_primitive`, `set_physics`), each with hard numeric bounds (`MAX_ACTIONS` = 16, per-spawn `MAX_SPAWN_COUNT` = 8, cumulative `MAX_TOTAL_SPAWNED_PER_SESSION` = 64, scale 0.1–5.0, rotation $\leq 360^\circ/\text{s}$, mass 0.1–100, `MAX_PLAN_JSON_BYTES` = 64 KiB) — alongside an optional C# candidate retained purely for baseline comparison. **The action plan is the safety source of truth.** Because the IR is finite and allow-listed, unsafe operations are *unrepresentable* rather than merely filtered, echoing the constrained-generation lineage in which restricting output to a formal schema both improves reliability and guarantees structural validity (Geng et al., 2023; Poesia et al., 2022). A new **Rust** backend joins the existing `Ubiq` network as a service peer, validates the plan fail-closed, and a minimal, audited Unity executor runs it with **no compilation** (Mode C), so the deployable product is Quest-compatible and cannot escape its allow-list. The original generate-and-compile path is preserved as **Mode A**; two further research modes — **Mode B** (validated-C#: a tree-sitter lexical scan plus a .NET Roslyn semantic check) and **Mode D** (sandboxed-C# in a hardened Docker/gVisor container) — enable an honest comparison of safety strategies. Rust was chosen for principled reasons: well-typed safe Rust is provably free of undefined behaviour (Jung et al., 2018), and the workspace forbids `unsafe` and denies `unwrap/expect/panic` in core crates so the validator's safety invariants are compiler-enforced rather than review-dependent.

1.5 Research questions

- **RQ1 — Safety vs expressiveness.** Can a schema-bounded, allow-listed action-plan IR, enforced by a fail-closed validator, prevent the unsafe operations (out-of-bounds values, banned APIs, schema-bypass) that the original runtime-C# pipeline permits, and at what cost to expressiveness?
- **RQ2 — Quest-compatible alternative.** Does the action-plan execution model remove the runtime-compilation dependency that makes the original system incompatible with the Meta Quest IL2CPP/AOT backend, yielding a structurally deployable on-device path? (*On-device operation is demonstrated for the Quest 1/2 Mono sideload of the runtime-compile path; the Quest 3 / Store IL2CPP build of the no-compilation Mode-C path remains future work — see §7.11.*)
- **RQ3 — Gate effectiveness against unsafe/malicious input.** How effectively do the Rust validation layers (the fail-closed action-plan validator; the lexical-plus-semantic C# gate) reject unsafe and adversarial inputs, including evasion attempts?
- **RQ4 — Latency / reliability trade-offs across modes.** Across the four execution modes, how do functionality, latency, security (rejection of malicious/unsafe inputs), privacy, and reliability compare, and what trade-offs do they expose for real-time immersive speech programming?

1.6 Aim and objectives

Aim. To design, implement, and evaluate a safe-by-construction architecture for real-time immersive speech programming in VR that preserves the expressive intent of the original DreamCodeVR while removing its runtime-

arbitrary-code-execution risk and its Meta Quest incompatibility, and to compare it against the original and intermediate safety strategies.

Objectives. 1. **Analyse** the original DreamCodeVR system and characterise its safety, deployability, and supply-chain weaknesses from its actual source (Chapter 3). 2. **Design** an allow-listed action-plan DSL with hard bounds and a fail-closed validator as the default-safe execution model. 3. **Implement** a Rust orchestration backend that joins the existing Ubiq network as a service peer, with per-peer isolation, async timeouts, and privacy-safe telemetry. 4. **Implement** the two research arms: a lexical-plus-semantic validated-C# path and a hardened container sandbox. 5. **Provide** a drop-in Unity executor that runs validated action plans without compilation (*implemented; play-test pending*). 6. **Verify** integration empirically against real Ubiq, real Roslyn, and real Docker (*completed for the backend; the gVisor runsc path is wired and unit-tested but not run*). 7. **Evaluate** the four modes on functionality, latency, security, and privacy (*methodology defined in Chapter 6; quantitative results identified as future work*). 8. **Assess** ethics and privacy obligations, including a Data Protection Impact Assessment (DPIA) for any human-subjects study.

1.7 Contributions

This dissertation makes the following concrete contributions, each grounded in the delivered Rust workspace at the repository root:

1. **An action-plan DSL as an allow-list** (`behaviour-dsl`): a six-variant, internally-tagged, `deny_unknown_fields` JSON IR whose enum is the safety allow-list, with a single authoritative bounds module. (*Verified by automated tests.*)
2. **A fail-closed validator** (`code-policy`): rejects oversized input *before parsing*, accumulates all violations, and approves *iff* the violation set is empty. (*Verified by automated tests.*)
3. **A Rust Ubiq service-peer orchestrator** (`unity-transport` + `command-router`): per-peer sessions with no global lock, async per-call timeouts, and a single spawn-budget chokepoint enforcing the cumulative cap — including two real Ubiq protocol bugs found and fixed by live testing (frame length = `NetworkId(8) + payload`; the `Join args` field must be stringified JSON). (*Verified live.*)
4. **A validated-C# research mode** (`csharp-policy` + `roslyn-client`): defence-in-depth via a tree-sitter lexical scanner (with dotted-name reconstruction to defeat evasion) plus a fail-closed .NET Roslyn semantic check — including the honest finding that the semantic layer is deny-list-only and shallow (it returned a **false negative** for `System.Net.WebClient`), which the Rust lexical layer catches as the backstop. (*Verified live, with the documented semantic-layer limitation.*)
5. **A sandbox mode** (`sandbox` + `services/sandbox-worker`): a hardened Docker runner (`--network none`, `read-only rootfs` + `tmpfs`, `--cap-drop ALL`, `pid/memory/cpu` limits, `no-new-privileges`, `non-root`, `wall-clock timeout` + `docker kill`) where only a structured report crosses back, verified under Docker; a `gVisor (runsc)` runtime is already selectable through the runner (`with_runtime / DCVR_SANDBOX_DOCKER_RUNTIME`, unit-tested) but was not exercised for lack of a `runsc` host, so kernel-level isolation is designed rather than measured. (*Docker verified live; gVisor designed, not measured.*)
6. **Privacy-by-construction logging** (`observability`): a `TimingEvent` JSONL schema carrying only `ids/timestamps/decisions/reason-codes/counts`, with a test asserting no `audio/transcript/secret/image` field can appear. (*Verified by automated tests.*)
7. **A differential four-mode evaluation design** (A/B/C/D) for functionality, latency, security, and privacy, with red-team corpora anchored to the OWASP LLM Top-10, together with an **executed** adversarial red-team: the C# lexical validator was hardened by closing three novel evasion classes, and a reproducible **1,057-vector** campaign confirmed it at **258/258** while hardening the natural-language

intent screen from **81.71%** (460/563 across both layers; the screen alone 202/305, with 103 plain-English bypasses its keyword table missed) to **100%** (563/563), all with zero over-blocks (these post-hardening figures are in-sample; the intent screen is an advisory, partly fail-open pre-filter, the validator being the authoritative gate). (*Executed — see §7.10.*)

8. **An empirically-verified, supply-chain-hardened engineering baseline:** an `unsafe`-free, panic-averse Rust workspace (compiler-enforced lints; `cargo-deny` clean across advisories, bans, licences and sources; 342 tests green) live-verified against real infrastructure, with the original's dead `gitpkg` dependency documented. (*Verified live.*)
9. **A live Unity 6 verification of both execution paths — including the original *made safe*:** the safe action-plan executor (Mode C) and the runtime-C# path (Mode A) were each driven end-to-end from live backend messages in Unity 6 (Editor Play, no Quest required). Mode A compiles validator-approved, model-generated C# at runtime to realise *open-ended* creative commands — building a house, tree, or castle from primitives — while a freeform lexical guardrail blocks malicious APIs and over-large code. This is direct evidence that the original generate-and-compile pipeline can be **retained and gated**, not merely replaced, with the user keeping full creative freedom inside safety bounds. (*Verified live in Unity 6.*)
10. **A safety-preserving personalization (RAG) layer** (`personalization`): a per-user preference profile plus an embedding-retrieval memory of liked creations (OpenAI `text-embedding-3-small`, or a deterministic offline mock) that biases the *aesthetic* of generated output toward the user's taste, learned from thumbs-up / thumbs-down feedback. The retrieved context is injected as clearly-labelled **untrusted data** (a prompt-injection guard) and the fail-closed validator still gates every output, so personalization cannot widen what may be generated — RAG is added *without* weakening the safety argument. (*Verified live.*)
11. **A live observability and runtime-control plane** (`control` + `admin`): a web admin panel (Rust/axum, server-sent events) that streams every pipeline stage live (transcript → prompt → C# → validation → compile, with per-stage timings and a total), exposes runtime-tunable safety knobs (max C#-size/lines, per-mode toggles, model), and a built-in red-team runner that demonstrates the guardrail blocking malicious samples on demand — turning the system's internal safety behaviour into directly inspectable, reproducible evidence for evaluation and demonstration. (*Verified live.*)
12. **A perceptual and embodied safety layer for LLM-generated immersive content** (`code-policy`, `command-router`, and the Unity executor): a user-frame invariant (head-centred personal-space sphere and forward occlusion cone), scene-relational guards (generated-object provenance, no pose-to-behaviour feedback loop, bounded displacement, overlap/disguise checks), and temporal envelopes (a per-peer accepted-plan rate cap of ≤3 plans/s as a WCAG 2.3.1 flash-rate proxy, an anti-vection rotation clamp, per-peer generation rate limiting, and per-recipient routing; a luminance-swing cap and a coherent optic-flow budget are specified as constants but are not yet consumed by enforcement), with a paper-cited perceptual red-team corpus. To the author's knowledge this is the first treatment of XR-specific perceptual and embodied harm in an LLM-driven scene-authoring pipeline. (*Backend defences verified by automated tests; the spatial guards are implemented in the deployable Mode-C Unity executor (ActionPlanExecutor/PerceptualSafety), opt-in via `comfortIntensity`, and compile and run live in Unity 6 (exercised by the GuardrailSelfTest harness); Mode A is retained as the unbounded full-freedom baseline.*)
13. **An age-adaptive dual-plane safety coupling** (`config::age`, `command-router`): an `AgeBand { Child, Teen, Adult, Unknown }` signal (default `Unknown`, which fails safe to *child*) designed to condition **both** safety planes from one age bit. The age-to-**code**-plane coupling is fully wired and enforced — a detected

minor alone flips the C# validator to its hardened `DeployHardened` profile. The parallel age-to-**perceptual**-plane posture (perceptual hardening on, compile-confirmation required, spawn budget 40→20, flash cap 3.0→2.0 Hz, FOV coverage 0.70→0.35, rotation 90→30°/s, luminance swing 1.0→0.4) is computed and unit-tested at the configuration layer but is **not yet consumed by runtime perceptual enforcement**, which presently reads the global runtime configuration; wiring the age-derived posture into that enforcement is future work. The coupling is opt-in behind `DCVR_AGE_GATING`; with it off the pipeline is byte-identical to the legacy path. To the author's knowledge this "age-as-protection, not surveillance" inversion is novel. (*Code-plane coupling verified live by `age_minor_forces_hardened_csharp_gate`; fail-safe and the posture table by `unknown_fails_safe_to_child` and `child_tightens_both_planes_vs_adult`; see §7.12.*)

14. **An on-device voice age-band gate** (`ml/age_gate`): a numpy-only, CPU-only, reproducible classifier that emits a coarse *band* — child (<13) / teen (13–17) / adult (18+) — and never a precise age and never affect, temperature-scaled for calibration ($T \approx 1.93$; Expected Calibration Error 0.051 → 0.012 on the validation split, held-out test ECE 0.007). The band is a secondary safety net only; the platform (Meta Quest) age-group API remains the authoritative source and legal basis. (*Verified by automated tests: 51/51; calibration reproducible via `train.py/evaluate.py`; real-microphone and on-device Quest evaluation are future work — see §7.13.*)
15. **A continuous ML attack analyzer** (`ml/attack_analyzer`): a numpy-only PCA/SVD undercomplete-autoencoder anomaly detector (pooled AUC 1.00 on the controlled corpus — a separation property of that corpus, not a field metric; a hard-negative slice drops the stealthiest family to ~0.70 AUROC — with a ~2% benign false-positive rate at the 99th-percentile threshold) plus an ADWIN2 drift detector (0 false alarms on a stationary stream; injected shift detected at $t = 304$) that also red-teams a numpy *stand-in* of our age gate (`MockAgeGate`) — surfacing an honest 100% spoof rate against that stand-in (the shipped classifier is not itself red-teamed) — over a corpus that starts at a curated 128 vectors and grows a discovered-vector store to 164 entries by mutating against a deliberately naive substring denylist (`MockDenylistSUT`), not the real validator. (*Verified by automated tests: 31/31; see §7.14.*)
16. **A whole-system verification campaign and a durable regression harness.** Every backend feature was verified by its downstream *effect* — not merely that it responded — organised block by block, one per subsystem, and now captured as a 40-check harness, standing up the real .NET Roslyn analyzer, the Mode-D Docker image, and the genuine UCL Ubiq RoomServer. The campaign uncovered and fixed **seven latent defects**, including two *safety controls that silently did nothing* — an admin "Mode A" toggle that still dispatched code to the headset when switched off, and a data-retention sweep (`purge_expired`) that was documented and unit-tested as a GDPR control but had no runtime caller, so profiles were kept forever — and an intent-screen gap that missed 103 plain-English attacks. The whole campaign is captured as a single-command regression tool (`scripts/verify-all.sh`, 40 checks) with a `--baseline` snapshot that later runs diff against to detect drift; drift detection was itself verified by deliberate perturbation. (*Executed; see §7.15.*)
17. **A from-scratch Rust reimplementation of the Ubiq RoomServer** (`crates/roomserver`): a single-binary, `unsafe`-free room relay that reaches stock-Ubiq protocol parity — adding the five previously-missing control messages (`AppendPeerProperties`, `AppendRoomProperties`, `DiscoverRooms`, `SetBlob`, `GetBlob`), with properties that persist into later room state — and validates RFC-4122-v4 room UUIDs as upstream does. In live testing it **stayed available on a malformed `SetBlob` control message that crashes the genuine UCL Node.js RoomServer** (`TypeError: this.server.setBlob is not a function`), where one client can otherwise take down the relay and every peer in the room — a concrete robustness improvement over the upstream artefact. (*Verified live; the upstream crash reproduced and documented, its third-party code left unpatched.*)

1.8 Dissertation structure

Chapter 2 reviews the literature spanning end-user/immersive programming, LLM code generation and its safety failure modes, constrained generation, Unity's AOT constraint, sandboxing, Rust security, and voice/VR privacy. Chapter 3 analyses the original DreamCodeVR as the Mode A baseline and extracts the four weaknesses above. Chapter 4 derives requirements, a threat model, and design goals. Chapter 5 describes the architecture and implementation of the Rust backend, the four modes, and the Unity drop-in. Chapter 6 specifies the four-mode differential evaluation methodology, and Chapter 7 reports what is **verified today** alongside clearly-labelled placeholders for results not yet collected. Chapter 8 discusses findings against the research questions, states limitations, and addresses ethics and privacy. Chapter 9 concludes and sets out future work — principally Unity/Quest integration, gVisor-in-production hardening, real-endpoint evaluation, and wiring Mode A for a fully automated baseline comparison.

Chapter 2 — Background & Literature Review

This chapter situates DreamCodeVR+ within the literatures it draws on and extends. The system replaces an unsafe *speech* → *large language model (LLM)* → *runtime-compiled C#* pipeline with a schema-bounded "action plan" validated by a fail-closed Rust backend, with generated C# retained only behind research-mode gates. Accordingly, the review traverses end-user and speech-driven programming (2.1), LLM code generation and its risks (2.2), prompt injection and the standardised threat taxonomies that frame them (2.3), constrained and schema-bounded generation (2.4), immersive systems and Ubiq networking (2.5), runtime code execution in Unity (2.6), container and kernel isolation (2.7), Rust for secure systems (2.8), and privacy obligations for voice and VR (2.9). Section 2.10 states the gap and the thesis it implies: that bounding the *output language* is a stronger safety lever than sandboxing the output.

2.1 Speech and natural-language programming

The parent problem is end-user behaviour authoring in VR, where construction of interactive scenes has traditionally demanded C# and engine expertise, excluding non-programmers. DreamCodeVR (Giunchi et al., 2024) showed that speech-driven, LLM-mediated code generation can lower that barrier, letting a user simply speak an intent — "*make this cube red*" — and have a target object change. Adjacent end-user-development tools for VR, such as VREUD (Yigitbas et al., 2022), confirm strong demand for novice-facing authoring but achieve safety by restricting users to pre-built block or visual vocabularies rather than free-form speech, and so never generate arbitrary code. Immersive, in-headset authoring tools such as FlowMatic (Zhang and Oney, 2020) similarly broaden novice access to interactive-VR creation, but through visual, direct-manipulation programming rather than free-form speech. DreamCodeVR+ sits between these positions: it keeps DreamCodeVR's speech-first accessibility while removing its unsafe execution model.

The input modality itself motivates a particular design choice. Decades of voice-programming research show speech to be viable but error-prone for verbatim code entry, and that novices tend to describe *intent* rather than syntax (Pane, Ratanamahatana and Myers, 2001). Empirical work on vocal user interfaces for programming (Begel and Graham, 2006) and on user preferences and speaking styles in programming by voice (Nowrin and Vertanen, 2023) reinforces that systems should absorb natural, underspecified phrasing rather than demand dictated syntax. This is precisely why an LLM that maps messy spoken intent onto a small, validated action vocabulary is a better fit than one that dictates C#: the intermediate representation (IR) becomes the natural sink for ambiguous speech.

2.2 LLM code generation and its risks

The engine of the pipeline is an LLM that emits both a structured action plan and an optional C# candidate. Two facts about such models must be established. First, the *capability*: Codex and the HumanEval benchmark (Chen et al., 2021) demonstrated that LLMs can synthesise functioning programs from natural language. Second, the *reliability ceiling*: the same work reported only modest single-sample functional-correctness rates and systematic failure modes, a finding generalised across models and tasks by subsequent surveys (e.g. Huynh and Lin, 2025; Zhang et al., 2023). The empirical conclusion is that raw LLM output cannot be trusted unchecked. This is the quantitative backbone of DreamCodeVR+'s central design move: the system never executes the LLM's C# directly and instead treats the bounded action plan as the source of truth, validating everything downstream.

Executing program text derived from untrusted input is the classic code-injection / remote-code-execution pattern. OWASP classifies dynamic evaluation of attacker-influenced input as *Eval Injection* and *Code Injection* (OWASP, no date a, no date b), the class catalogued canonically as CWE-94 (MITRE, 2026), and the pattern grants the input the privileges of the host process. Running LLM-generated C# through Unity's runtime compiler is functionally equivalent: the "untrusted input" is the model's output plus whatever steered it. That this is not hypothetical is

shown by real incidents in LLM agent frameworks built on `eval` (e.g. the `unsafe-eval` advisory in microsoft/autogen issue #4873, used here as a grey-literature illustration rather than a peer-reviewed source). DreamCodeVR+'s baseline (Mode A), which compiles and runs LLM-authored C# inside Unity, is exactly this hazard, and it is the hazard the rest of the architecture exists to neutralise.

2.3 Prompt injection and standardised threat taxonomies

The threat model assumes the LLM can be *steered* to emit malicious output. The OWASP Top 10 for LLM Applications (OWASP Gen AI Security Project, 2025a) codifies the relevant risks: LLM01 (Prompt Injection) and LLM05 (Improper Output Handling). Greshake et al. (2023) demonstrated *indirect* prompt injection compromising real LLM-integrated applications, establishing that any LLM output flowing into a downstream executor is an attack surface — the model need not be "jailbroken" by the user directly. DreamCodeVR+'s mitigations map onto this taxonomy in a MITRE-style, control-by-control fashion: it treats LLM output as untrusted (LLM05), validates the action plan fail-closed, scans any C# candidate lexically and semantically, and fixes the `request_id` server-side so the model never owns its own identifier. The architecture is thus a direct, implementable response to the LLM05 guidance that LLM output must never be trusted as code.

2.4 Constrained, schema-bounded and grammar-constrained generation

This is the conceptual heart of the project. The action-plan IR (the `behaviour-dsl` crate: a `#[serde(tag="type")]`, `deny_unknown_fields` enum of six actions) is a deliberately tiny, allow-listed DSL whose schema is fixed. The relevant literature shows why this is powerful. Grammar-constrained decoding (Geng et al., 2023) and constrained semantic decoding (Poesia et al., *Synchromesh*, 2022) demonstrate that restricting generation to a formal grammar or schema both improves reliability and *guarantees* structural validity; PICARD (Scholak et al., 2021) showed the same for SQL by rejecting invalid continuations incrementally during decoding. The same principle underlies modern JSON-schema-constrained ("structured output") decoding and its benchmarking (e.g. Geng et al., 2025b; the efficiency follow-up Geng et al., 2025a).

The decisive property for safety is subtle but central to this dissertation: because the IR is finite and allow-listed, unsafe operations are *unrepresentable*, not merely *filtered*. There is no `set_color` argument that opens a socket, because the grammar admits no such instruction. The `code-policy` validator then enforces numeric and budget bounds fail-closed on top of structural validity. This reframes safety from "detect bad code after the fact" (the static-analysis and sandboxing posture of 2.6–2.7) to "make bad code inexpressible in the first place" — the move that differentiates DreamCodeVR+ from its DreamCodeVR predecessor.

2.5 Immersive systems and Ubiq networking

DreamCodeVR and DreamCodeVR+ are built on Ubiq (Friston et al., 2021), UCL's open social-VR networking platform, in which peers exchange messages routed by `NetworkId` through a TCP RoomServer. DreamCodeVR+ preserves the original three channels (audio input, selected object, backend output) and joins the room as an additional *service peer*, so the safety backend is a network participant rather than an in-engine module. This positioning matters for the threat model: trust boundaries are crossed on the wire, and the Ubiq frame format and join handshake had to be implemented exactly (two real protocol defects in the length field and the `Join` `args` encoding were found and fixed against the real RoomServer during this project).

2.6 Runtime code execution in Unity (Roslyn / IL2CPP / AOT)

This is the deployability half of the thesis and its hardest constraint. The original baseline compiles model-generated C# at runtime using a Roslyn-based Unity component. Unity's own documentation states that the

IL2CPP scripting backend uses ahead-of-time (AOT) compilation and does not support runtime or dynamic code generation — `System.Reflection.Emit` is unavailable (Unity Technologies, *IL2CPP scripting restrictions; Introduction to IL2CPP; Overview of .NET in Unity*). IL2CPP is mandatory for the Meta Quest Store and for the 64-bit ARM64 target that Quest 3 and current Horizon OS require, so any architecture that depends on compiling C# at runtime cannot be Store-published or run on current-generation hardware — it is confined to manually sideloaded, 32-bit **Mono** builds on Quest 1/2 (where, as Chapter 3 documents from the project's own `ProjectSettings.asset`, it does run). This is a primary-sourced justification for the action-plan executor: Mode C dispatches vetted UnityEngine operations from a parsed plan with zero compilation, so it builds under IL2CPP/ARM64 and ships through the Store by construction. The Unity-side executor — and, indeed, the runtime-compile path itself, now *gated by the validator* — were both verified live in the Unity 6 editor for this dissertation (Chapter 7), and the runtime-compile path was subsequently run on-device on a Quest 1/2 32-bit Mono sideload (§7.11); what remains argued structurally rather than demonstrated is the Quest 3 IL2CPP/Store build of the no-compilation Mode-C path.

2.7 Container and kernel isolation

For studying genuinely untrusted generated C#, DreamCodeVR+ adds Mode D: a hardened container (`--network none`, read-only rootfs plus tmpfs, `--cap-drop ALL`, `--pids-limit`, memory/CPU caps, `no-new-privileges`, non-root UID, wall-clock timeout with `docker kill`), with a script for the gVisor `runsc` runtime. NIST SP 800-190 (Souppaya et al., 2017) is the authoritative baseline for container threats and countermeasures and justifies both the hardening flags and the project's honesty that a default container is a *soft* boundary. The salience of that honesty is underscored by the runc "Leaky Vessels" container-escape vulnerability, CVE-2024-21626 (MITRE, 2024), which shows that the container runtime's own trusted computing base is itself attackable. gVisor interposes an application kernel (Sentry) between the workload and host, so an attacker must break two disjoint kernels to escape (gVisor project, no date), with the performance cost quantified by Young et al. (2019) and compared across runtimes by Wang et al. (2022). This literature explains why sandboxing is positioned in this work as a *research* mode, not the default deployable path: the default (Mode C) avoids codegen entirely. In this project only the Docker boundary was exercised and should be presented as the lower bound on isolation strength; the gVisor (`runsc`) path is wired and unit-tested in the Rust runner but was not run, so its stronger kernel isolation is a designed rather than a measured property.

2.8 Rust for secure systems

The entire backend is Rust, with the workspace forbidding `unsafe` by lint policy, denying `unwrap / expect / panic` in core crates, holding the API key in `secrecy::SecretString`, and auditing dependencies with `cargo-deny`. The justification is formal, not promotional. RustBelt (Jung et al., 2018) gives the first machine-checked proof that well-typed *safe* Rust is free of undefined behaviour; because DreamCodeVR+ compiler-enforces `forbid(unsafe_code)`, the safe-subset guarantees RustBelt proves actually hold for this codebase — which matters because a memory-corruption bug in the validator would undermine every other layer. Honesty again requires the counterpoint: safety holds only for the safe subset, and real Rust systems carry risk through `unsafe` blocks and their dependency trees, as catalogued by an empirical study of security risks in the Rust ecosystem (Zheng et al., 2024). This is precisely the risk `cargo-deny` is deployed to mitigate, so the counterpoint is reflected in the engineering rather than waved away. The industrial weight of this argument is evidenced by Google (2024), which reports Android's share of memory-safety vulnerabilities falling from 76% to 24% over six years as new development shifted to memory-safe languages such as Rust.

2.9 Privacy in voice and VR, and DPIA obligations

DreamCodeVR+ processes push-to-talk audio and was built "privacy by construction": raw STT/LLM detail goes only to stderr, the wire and logs carry fixed reason codes, and the telemetry `TimingEvent` is asserted by test to contain no audio, transcript, or secret. This design is responsive to a concrete legal baseline. Under the GDPR, voice used for identification is special-category biometric data (Art. 9), high-risk processing of which triggers a mandatory Data Protection Impact Assessment (Art. 35), with privacy-by-design a standing duty (Art. 25) (European Union, 2016, Arts 9, 25, 35). The EDPB's Guidelines 02/2021 address virtual voice assistants specifically (European Data Protection Board, 2021), and academic studies map the broader voice-assistant privacy threat surface (Germanos et al., 2020). VR adds further sensitive signals, such as gaze and motion, that broaden the sensitive-data surface beyond voice alone, and recent work documents user concern toward LLM-powered conversational agents in immersive extended reality (Bozkir and Kasneci, 2025). DreamCodeVR+'s telemetry minimisation and reason-code design are concrete privacy-by-design measures that a DPIA section can map onto these obligations; honestly, however, an actual DPIA has not yet been conducted and remains a precondition for any user study.

2.10 Gap statement and comparison

Two literatures rarely meet at the point this project occupies. On one side sits *structured and grammar-constrained LLM generation* (2.2, 2.4), concerned with making model output valid and reliable. On the other sits *systems sandboxing* (2.6, 2.7), concerned with containing whatever a process does after it runs. DreamCodeVR+ argues that for end-user programming in a safety-sensitive, AOT-constrained immersive setting, the first lever is the stronger and more deployable one: if the output language admits only safe instructions, there is less — ideally nothing — left to sandbox, and no runtime compiler is required at all. Constrained generation moves safety *upstream* of execution and is therefore Store- and Quest-3-deployable in a way that runtime sandboxing of generated C# is not. The three strategies are compared in Table 2.1.

Table 2.1 — Comparison of safety strategies for executing model-derived behaviour.

Axis	Static analysis (lexical + semantic scan of generated code)	Sandboxing (container / kernel isolation of generated code)	Constrained generation (schema-bounded action plan)
Residual attack surface	Moderate–high: bounded by analyser completeness; evasions and false negatives remain (e.g. the shallow Roslyn layer missed <code>System.Net.WebClient</code> , caught only by the lexical backstop)	Low but non-zero: runtime TCB itself is attackable (CVE-2024-21626); plain Docker is a <i>soft</i> boundary (NIST SP 800-190); gVisor reduces it further	Lowest: unsafe operations are <i>unrepresentable</i> in a 6-action allow-list, not merely detected after generation
Quest Store / Quest-3 deployability	Sideload-only: still implies runtime compilation of the analysed C#, which runs only under Mono on sideloaded 32-bit Quest 1/2 — not via the IL2CPP/ARM64 Store path and not on Quest 3	Sideload-only at best: presupposes a runtime compiler/executor, so the same Mono/32-bit confinement applies; not Store- or Quest-3-deployable	Yes by construction: zero compilation, so it builds under IL2CPP/ARM64, ships through the Store, and runs on Quest 3
Latency cost	Low–moderate: scanning adds per-request analysis (lexical fast; semantic adds a service round-trip)	Moderate–high: container spawn, compile-and-run, and isolation overhead per request	Lowest: validation is a bounded JSON parse plus arithmetic bounds checks
Generality	High: scans arbitrary C#	High: runs arbitrary code, then contains it	Low <i>by design</i> : only the six allow-listed behaviours are expressible — the deliberate expressiveness/safety trade-off

The table makes the dissertation's positioning precise. Static analysis and sandboxing are *general but reactive* and, critically, both presuppose the very runtime-compilation step that confines the original system to sideloaded 32-bit Mono Quest 1/2 builds (not Store-distributable or Quest-3-compatible). Constrained generation sacrifices generality — only six behaviours can be expressed — to obtain the strongest residual-attack-surface guarantee, the lowest latency, and the only Store-/Quest-3-deployable execution path. DreamCodeVR+ therefore retains static analysis (Mode B) and sandboxing (Mode D) as research arms for studying generated C#, while making constrained generation (Mode C) the default. The central claim that the remainder of the dissertation defends is methodological: **for safety-sensitive immersive end-user programming, bounding the output language is a stronger and more deployable safety lever than sandboxing the output, and it can be enforced by construction.**

2.11 XR perceptual and embodied attacks: the second safety dimension

The threat surface in §2.2–2.3 concerns the *system*: untrusted code reaching the host's file, network, or process APIs. A distinct and, for immersive speech programming, equally important body of work concerns the *user*: how the immersive medium itself can be weaponised against the person wearing the headset. Krauß et al. (2024) catalogue dark patterns unique to extended reality and identify four amplifying properties — perception, spatiality, physical/virtual barriers, and device sensing — showing that most XR manipulations are familiar two-dimensional tactics re-expressed in three dimensions, but that embodiment, realism, and personalisation unlock new, bodily harm. Tseng et al. (2022) give a severity taxonomy of perceptual manipulations from recoverable (a stumble) to unrecoverable (injury, death), of which the *false-positive mismatch* — a plausible virtual chair, step, or floor with no real counterpart that lures the user to act and fall — is the most directly applicable here. The locomotion channel is concrete: Casey et al. (2021) demonstrate the *human joystick*, sub-perceptually translating the virtual world to walk a user into a real wall, and Ishii et al. (2016) steer a walker's direction by transforming the camera view; Pearlman (2020) frames such immersive harm as physical, not cosmetic. The deception channel spans cursorjacking transposed to VR (Su et al., 2022) and 3D object disguise, where a rendered object hides or misrepresents what is really present (Fujita et al., 2023). Crucially, Lee et al. (2021) not only demonstrate WebVR fraud but provide the defensive blueprint this dissertation adopts — *confinement* of untrusted content to what the user can legitimately see (a field-of-view/visibility constraint, forbidding behind-camera placement, and entity-rate caps) — while Valluripally et al. (2020) contribute an attack-tree-plus-statistical-model-checking method for bounding the probability that a system reaches a user-harm state.

The unifying observation is decisive for DreamCodeVR+: **the harm is compositional, relational, semantic, and temporal, not a property of any single action's magnitude.** A wall of individually in-bounds primitives still occludes; a sequence of sub-threshold moves still drifts the user; a legal colour change repeated rapidly still strobes; a correctly-scaled object placed at the head is still a projectile. The allow-list and numeric bounds of Chapters 4–5 are therefore *necessary but not sufficient*: they delete the catastrophic system-level classes — camera, locomotion, input, network — by making them unrepresentable, yet leave the perceptual/embodied classes expressible within bounds. DreamCodeVR+ answers this with a defence-in-depth ordered by strength: capability omission; a fail-closed validator; a **user-frame invariant** (an un-spoofable, head-centred personal-space sphere and forward occlusion cone that no generated object may enter or occlude); **scene-relational** checks (no pose-to-behaviour feedback loop, generated-object provenance, no false load-bearing affordance, bounded displacement); and **temporal envelopes** (a per-peer accepted-plan rate cap of ≤ 3 plans/s as a WCAG 2.3.1 flash-rate proxy and per-peer generation rate limiting; a luminance-swing cap and a coherent optic-flow budget are specified as constants but are not yet consumed by enforcement). The user-frame invariant and the scene-relational layer are the novel contribution, because an invariant holds however many in-bounds actions are composed, which the per-action numeric validator cannot. These layers are implemented in the deployable Mode-C executor

(`ActionPlanExecutor` / `PerceptualSafety`) and are opt-in via a comfort setting, so free creation is unconstrained while the safety demonstration arms them; Mode A is retained as the original full-freedom baseline, gated only against system access. Honest residuals, returned to in §8, remain: pose-free safety degrades when a bare headset cannot supply a trusted head pose; cumulative/composite effects require stateful temporal accounting split across backend and client; cross-peer attacks await message authentication; and false-affordance detection needs scene-semantic reasoning the action set does not represent.

All sources cited in this chapter appear in the reference list with resolvable URLs or DOIs, and none has been invented. The review establishes only the prior work against which the contribution is positioned; no empirical DreamCodeVR+ results, user studies, or Quest, speech-to-text, or large-language-model outcomes are asserted here.

Chapter 3 — Analysis of the Original DreamCodeVR System (the Mode-A Baseline)

3.1 Purpose and scope

This chapter analyses the original DreamCodeVR system as it actually exists in the inherited source tree, in order to extract the concrete weaknesses that motivate the DreamCodeVR+ redesign. DreamCodeVR is a published research artefact (Giunchi *et al.*, 2024) whose central idea — letting a user shape a virtual scene by *speaking* — is preserved and built upon throughout this dissertation. The analysis below is therefore deliberately fair: the claim defended is **not** that the original "does not work", but the narrower and verifiable proposition that the original is *unsafe by default, deployable to a Meta Quest only as a sideloaded 32-bit Mono build (and therefore neither Store-distributable nor Quest-3-compatible), and — as observed during this project's setup — not installable as published because one of its dependency hosts is dead*. Each weakness is grounded in the original source files (Unity/Assets/CodeGenerationManager.cs, Unity/Assets/Scenes/Scripts/TestRoslyn.cs, Server/samples/apps/code_runtime_generator/app.js, Server/package.json) and forms the Mode-A baseline against which Modes B–D are later compared. The server-side files are reproduced verbatim in the inherited read-only copy (_original-dreamcodevr-daniel/Server/) and every server-side citation is line-exact; the two Unity client scripts *and* the ProjectSettings.asset excerpt of Weakness 2 are cited from the upstream UCL-VR/dreamcodevr repository (accessed 2026) rather than vendored, and the Unity version (2021.3.16f1) and scene path are independently corroborated by the inherited code_runtime_generator/README.md, which does ship in the artefact. (Shipping the two small .cs files and the ProjectSettings.asset excerpt in an appendix would let an examiner check the Unity-side line citations directly; this is noted as an outstanding reproducibility improvement.)

3.2 The original pipeline, traced end to end

The original system is a four-process pipeline carried over UCL's Ubiq social-VR networking layer (Friston *et al.*, 2021). A user wearing the headset holds a controller trigger to record a push-to-talk utterance and points a selection ray at a target object. The Unity client (Unity 2021.3.16f1; scene Unity/Assets/Demos/DynamicCompiler/DynamicCompiler.unity) streams the captured PCM audio to a **Node.js Ubiq-Genie server** over Ubiq NetworkId 98 (NID_AUDIO_INPUT), each body prefixed with a 36-byte ASCII peer UUID and bracketed by __STT_CONTROL__:start / :stop control frames. The server's MessageReader on NID 98 forwards the accumulated audio to a **Python faster-whisper speech-to-text (STT) service** (the original README.md points this at an HTTP endpoint, http://130.136.2.161:50101). The returned transcript is prepended with a > sentinel and passed to a **CodeGenerationService**, which prompts the **OpenAI API** to emit C# source. The server returns that C# to Unity over NetworkId 94 (NID_BACKEND_OUTPUT) as a JSON {type, peer, data} message (app.js:111). On the client, CodeGenerationManager (registered on NID 94, CodeGenerationManager.cs:27,53) receives the message and, with no intervening checks, hands the raw message.data string to TestRoslyn (CodeGenerationManager.cs:87–94), which calls domain.CompileAndLoadMainSource(...) (TestRoslyn.cs:111) — the **RoslynCSharp** asset's runtime compiler — to compile the generated C# into a fresh assembly and execute it against the selected object. Object selection uses the Ubiq-Genie selection channel, NetworkId 93 (NID_SELECTED_OBJECT), which the sibling Ubiq-Genie samples consume (texture_generation/app.js:26, storytelling/app.js:33); the DreamCodeVR code_runtime_generator app itself registers only NID 98 (new MessageReader(this.scene, 98), app.js:19) and declares only outputNetworkId 94, so selection context reaches the model only through the client. NID 93 is therefore a Ubiq-Genie convention DreamCodeVR+ adopts, not a channel the original DreamCodeVR server consumed. The headline property of this design is that *the language model's output is executed as native code on the device*, immediately and unconditionally.

3.3 Weakness 1 — Runtime arbitrary code execution on the host/headset

The most serious weakness is structural: the system performs the equivalent of `eval` on untrusted, model-generated input. `CodeGenerationManager.ProcessMessage` takes the C# arriving on NID 94 and passes it directly to the runtime compiler with no validation gate of any kind (`CodeGenerationManager.cs:83–96`); the compiled assembly then runs with the full privileges of the Unity process. There is no allow-list, no namespace restriction, and no human or programmatic confirmation step between generation and execution. Because the C# is whatever the LLM produced — and the LLM's behaviour is steerable through the user's spoken prompt and any upstream content — this is a textbook instance of the code-injection / arbitrary-code-execution class (OWASP, no date a; MITRE, 2026), and of OWASP's *Improper Output Handling* risk for LLM-integrated applications (OWASP Gen AI Security Project, 2025a). A generated (or prompt-injected) program could legitimately reach `System.IO`, `System.Net`, `System.Reflection`, or `System.Diagnostics` and read files, open sockets, or spawn processes, all inside the user's session. The RoslynCSharp asset does ship an optional security-sandbox/allow-list facility, but the baseline scene does not configure it as a fail-closed gate ahead of execution; safety, where present, is opt-in rather than enforced by construction. This single property is the primary motivation for DreamCodeVR+'s thesis that *bounding what may be generated is a stronger safety lever than auditing what was generated*.

3.4 Weakness 2 — Confined to sideloaded 32-bit Quest builds; not Store-publishable or forward-compatible

The second weakness concerns *how* the headline feature can reach the headline device. Runtime C# compilation depends on generating and loading new managed code while the application runs, which requires a just-in-time-capable scripting backend. On Android/Quest that means Unity's **Mono** backend with a 32-bit **ARMv7** target — and the original project is configured exactly so (`Unity/ProjectSettings/ProjectSettings.asset: scriptingBackend{Android:0} = Mono, AndroidTargetArchitectures: 1 = ARMv7`). Under Mono, `RoslynCSharp's ScriptDomain.CompileAndLoadMainSource` compiles and runs generated C# *on-device*, so the original system **can be sideloaded onto a Quest 1/2 and genuinely works there** — a fact corroborated by the original authors' demonstrations and not disputed here. (An earlier draft of this dissertation overstated the constraint as outright "IL2CPP/AOT non-deployability"; the precise, primary-source-grounded position is the one given here.)

The weakness is that this sideloaded-Mono envelope is the *only* one the design admits, and it is a shrinking one. First, the **Meta Quest Store** requires the **IL2CPP** backend with a 64-bit **ARM64** target; IL2CPP compiles all C# *ahead of time* (AOT) and provides no runtime C# compiler or dynamic code generation (`System.Reflection.Emit` is unsupported under AOT) (Unity Technologies, 2024a, 2024b). The runtime-compile design therefore **cannot be published through the Store** — only manually sideloaded. Second, **Quest 3 and current Horizon OS require 64-bit applications** (32-bit support is deprecated), so the Mono/ARMv7 build that makes runtime compilation possible **will not run on current-generation hardware**. Third, bundling the full Roslyn compiler into a mobile build is a heavy footprint for a wearable. The net constraint is that the original's expressiveness is purchasable only at the cost of confinement to sideloaded, 32-bit, Mono Quest 1/2 builds — unsafe (Weakness 1), undistributable through the Store, and not forward-compatible. This is a deployability constraint independent of the safety one, and it motivates a path that needs *no* runtime compilation at all (Mode C): such a path builds under IL2CPP/ARM64, ships through the Store, and runs on Quest 3. Chapter 7 reports that the runtime-compile path itself was nonetheless reproduced **and made safe** in this project — model-generated C#, validated by the Rust gate, compiled at runtime in the Unity 6 editor — confirming both that the mechanism is real and that DreamCodeVR+ can gate it rather than abandon it.

3.5 Weakness 3 — Missing validation gate and global `isGenerating` serialisation

The third weakness has two facets, both visible in the server's pipeline (`Server/samples/apps/code_runtime_generator/app.js`). First, there is **no validation gate** between the LLM and execution: the transcript is forwarded to code generation (`app.js:96`) and the resulting C# is relayed verbatim to the client (`app.js:111–115`), which compiles it. Nothing in the path inspects the generated code, checks it against a schema, or constrains it to a known-safe operation set — the design trusts the model's output as code. Second, the server enforces a single **global `isGenerating` flag** as its only concurrency control (`app.js:29,72,92,116`). A request is admitted only when `isGenerating == false` (`app.js:72`); the flag is then set `true` for the duration of generation (`app.js:92`) and cleared when the C# returns (`app.js:116`). This is a *global* lock, not a per-peer one: while any one user's utterance is being processed, every other user's transcript that crosses the length threshold is silently discarded rather than queued or handled on an independent session. In a multi-peer social-VR setting this both serialises throughput across all participants and fails *silently*, dropping commands with no feedback to the affected user. Worse, the flag has no error path: it is cleared only inside the `startsWith('>')` success branch (`app.js:107–117`), and the generation service emits that `>` sentinel only on success (`openai_chatgpt_api.py:91–92`, guarded by `if ok`). A single failed generation — an invalid API key, a rate limit, a dropped connection — therefore leaves `isGenerating` permanently `true` and wedges the pipeline for *every* peer until the server is restarted; there is no timeout and no watchdog. Together these facets — no output validation, a single coarse global serialisation point, and a fail-silent lock with no error path — define the safety-and-concurrency gap that the DreamCodeVR+ `code-policy` validator and per-peer `command-router` sessions are designed to close.

Two further defects sit verified in the inherited server source and bear directly on later requirements. First, **transcript persistence**: `app.js:72–86` appends every recognised utterance longer than the ten-character threshold to `data/input.txt` (provided no generation is already in flight), and `openai_chatgpt_api.py:61–63` appends every raw model response to `data/response.txt`, with no retention policy — the original writes user speech and model output to disk by default, which is precisely the "before" that DreamCodeVR+'s reason-codes-never-transcripts telemetry (NF4, §4.2) answers. Second, **absent secret hygiene**: a populated Azure Speech key is committed in `config.json`, and a PEM private key (`key.pem`) ships beside the application — a second, independently checkable supply-chain / secret-hygiene failure alongside the dead `gitpkg` pin analysed in §3.6.

3.6 Weakness 4 — A dead supply-chain dependency (observed)

The fourth weakness was discovered while attempting to stand the original server up for this project, and is reported as an *observed, dated* fact rather than a permanent property of UCL's infrastructure. The server pins its core Ubiq networking dependency to a `gitpkg` host: `"ubiq": "https://gitpkg.now.sh/UCL-VR/ubiq/Node?176b628c..."` (`Server/package.json:15`, mirrored in `Server/package-lock.json:16,1536`). During this project's setup, in June 2026, that host returned **HTTP 402 Payment Required**, so `npm install` could not resolve the dependency and the server could not be installed as published. The practical consequence is that the original artefact is not reproducible out-of-the-box from its manifest. This is consistent with established guidance on software-supply-chain fragility — pinning a runtime dependency to a third-party transformation/CDN host rather than a versioned, archived package introduces exactly this single point of failure. For this dissertation the issue was mitigated by **vendoring the real Ubiq RoomServer directly from the UCL-VR/ubiq GitHub repository** into a local working copy, which restored a runnable server and enabled the live verification reported in later chapters. The honest framing is that the upstream source remains available on GitHub; what is dead is the specific `gitpkg.now.sh` *fetch URL the manifest depends on*, observed on the dates recorded above.

3.7 Implications for the redesign

The four weaknesses are not independent annoyances but a coherent argument for a different execution model. Weaknesses 1 and 3 say the system trusts and immediately runs model-authored code with no gate; weakness 2 says the mechanism that does so cannot ship on the target device anyway; weakness 4 says the published artefact is, as observed, not even installable. DreamCodeVR+ responds by retaining the original generate-and-compile path as a controlled baseline (**Mode A**, preserved but not wired into the new Rust flow) and replacing the *default* path with a schema-bounded action plan that is validated fail-closed and executed without compilation. Table 3.1 maps each weakness to its consequence and the corresponding DreamCodeVR+ response.

Table 3.1 — Original weakness → consequence → DreamCodeVR+ response.

#	Weakness in the original (Mode A)	Consequence	DreamCodeVR+ response
1	Runtime arbitrary code execution: LLM C# compiled and run with no validation gate (<code>CodeGenerationManager.cs:87–94</code> ; <code>TestRoslyn.cs:111</code>)	Model/prompt-injected code can reach file, network, process and reflection APIs on host/headset with session privileges	Default Mode C : speech → allow-listed six-action plan executed by an audited Unity component that compiles nothing ; unsafe operations are <i>unrepresentable</i> . Generated C# only reachable behind a research flag, guarded by lexical + Roslyn-semantic checks (Mode B) and a hardened sandbox (Mode D)
2	Runtime C# compilation incompatible with the Quest Store / Quest 3 IL2CPP/AOT path (<code>System.Reflection.Emit</code> unsupported AOT)	Cannot be Store-published or run on Quest 3; runs only as a sideloaded 32-bit Mono build on Quest 1/2	Mode C removes the runtime-compilation dependency entirely → IL2CPP/ARM64-compatible by construction (Quest 1/2 Mono sideload demonstrated on-device, §7.11; the Quest 3 / Store IL2CPP build is outstanding)
3	No output validation gate + single global <code>isGenerating</code> lock (<code>app.js:72,92,116</code>)	LLM output trusted as code; multi-peer commands serialised and silently dropped during any in-flight request	code-policy fail-closed validator (approve iff zero violations) + command-router per-peer sessions (<code>HashMap<peer, session></code> , no global lock) with per-call async timeouts and a single spawn-budget chokepoint
4	Core <code>ubiq</code> dependency pinned to <code>gitpkg.now.sh</code> (<code>Server/package.json:15</code>), which returned HTTP 402 during this project (observed, dated)	Server not installable as published; artefact not reproducible from its manifest	RoomServer vendored from UCL-VR/ubiq GitHub ; <code>cargo-deny</code> supply-chain gate on the new Rust workspace; recommendation to vendor/version-pin the RoomServer (future work)

In summary, the original DreamCodeVR is a compelling demonstration of speech-driven behaviour authoring whose *execution model* — not its idea — is the problem. It is unsafe by default, not AOT-deployable, and (as observed during this project) not installable as published. These three precise, source-grounded findings frame the requirements and threat model developed in Chapter 4 and the safe-by-construction architecture of Chapter 5.

Chapter 4 — Requirements, Threat Model and Design Goals

This chapter turns the weaknesses of the original DreamCodeVR baseline (Chapter 3) into an explicit specification for DreamCodeVR+. It first states the functional and non-functional requirements, then develops a threat model — assets, adversaries, attack surface and trust boundaries — and finally distils both into a small set of design goals that the architecture of Chapter 5 must satisfy. The governing principle, derived directly from the analysis, is that *the deployable system must never execute server-supplied code and must degrade to a refusal on any error*: safety is sought **by construction** and **fail-closed**, not by detection after the fact.

4.1 Functional requirements

The system must preserve the end-user experience of the original while changing what is generated. A user holds a controller trigger, points a ray at an object, and speaks; the scene changes accordingly. Concretely, the backend must (F1) accept push-to-talk audio and an object selection over the existing Ubiq channels and return a decision; (F2) translate a spoken request into a schema-bounded *action plan* drawn from a fixed, allow-listed instruction set; (F3) execute that plan on the Unity client **without compiling any code**, so that the default path runs on a Meta Quest; (F4) support multiple simultaneous peers, so that one user's request can never stall another's; and (F5) retain the original generate-and-compile path (Mode A) and the two C# research arms (Modes B and D) for comparison, without contaminating the default-safe path. The original Ubiq message identifiers are preserved — NID 98 (audio in) and NID 94 (decision out) unchanged from the original DreamCodeVR server, and NID 93 (selection in) following the Ubiq-Genie selection convention (§3.2) — so that the new backend is a drop-in service peer.

4.2 Non-functional requirements

The non-functional requirements encode the lessons of Chapter 3 and are summarised in Table 4.1. They span Quest-deployability (no runtime code generation on the default path), fail-closed safety (approve a plan only when it carries zero violations), Ubiq/NID compatibility, privacy-by-design (telemetry carries only reason codes, never transcripts or audio — the on-by-default personalization store, disableable/opt-out, being a disclosed exception, §5.12), reliability (per-peer isolation; the backend survives a sandboxed crash), and supply-chain and memory-safety hygiene (an `unsafe`-free, panic-averse workspace).

Table 4.1 — Requirements traceability (requirement → design goal → implementing component → evidence).

ID	Requirement	Goal	Implemented in	Status / evidence
NF1	Quest-deployable default: no runtime C# compilation on Mode C	G1	<code>behaviour-dsl</code> , <code>Unity SafeBehaviourRegistry</code>	Quest 1/2 Mono sideload verified on-device (§7.11); Quest 3 IL2CPP/Store build outstanding
NF2	Fail-closed validation: approve iff zero violations	G2	<code>code-policy::validate_plan</code>	Verified live (real Ubiq → <code>ApproveActionPlan{set_color #FF0000}</code>); 17 tests in <code>code-policy/tests/validation_tests.rs</code>
NF3	Ubiq/NID 98/93/94 compatibility preserved	G6	<code>protocol</code> , <code>unity-transport</code>	Verified live against the real RoomServer
NF4	Privacy-by-design: reason codes in telemetry, never transcripts	G3	<code>command-router</code> , <code>observability</code>	Leak test asserts no audio/transcript/secret/image field can appear in <code>TimingEvent</code> (the personalization store persists commands + C# on by default / opt-out — disclosed, §5.12/§8.2)
NF5	No global lock; concurrent peers isolated	G2	<code>command-router</code> per-peer <code>HashMap</code>	Verified live (per-utterance tasks)
NF6	<code>unsafe</code> -free, panic-averse; clean supply chain	G4, G5	workspace lints, <code>cargo-deny</code>	<code>unsafe_code = forbid</code> ; gate green (342 tests)
NF7	Bounded resource use (plan size, spawn budget)	G2	<code>bounds.rs</code> , <code>code-policy</code> , <code>commit</code>	<code>MAX_PLAN_JSON_BYTES=64 KiB</code> ; cumulative 64-spawn cap

4.3 Assets, adversaries and trust boundaries

Assets. Three assets must be protected. (A1) the **host and headset** — the Quest device and the host machine must never run attacker-controlled native code or reach the file, network or process APIs. (A2) the **backend process** — it must stay available, and one peer must not be able to deny service to others. (A3) the **user and their data** — voice and transcripts are minimised and never logged; their integrity and confidentiality must hold.

Adversaries. Following the OWASP Top 10 for LLM Applications (OWASP Gen AI Security Project, 2025a), the model is treated as untrusted output, not trusted logic. Three adversaries are in scope: (1) a **careless LLM** that, without malice, emits invalid, out-of-bounds or unsafe output (the empirical reliability ceiling of code-generating models (Chen et al., 2021)); (2) an **adversarial prompt or indirect injection** — a user, or content the model ingests, that steers it toward unsafe output, the indirect-injection class demonstrated against real LLM-integrated applications (Greshake et al., 2023) and codified as OWASP LLMo1 (Prompt Injection); and (3) a **malicious generated payload** — generated C# that, if compiled and run, would attempt arbitrary file, network or process access (OWASP code-injection class; CWE-94), the precise hazard of the Mode A baseline.

Attack surface and trust boundaries. The wire surface is the three Ubiq channels. Untrusted bytes arrive on NID 98 and NID 93 from *any* room peer; decisions leave on NID 94. Three trust boundaries follow: **TB1 untrusted → backend** (bytes on NID 98/93); **TB2 LLM output → execution** (all generated text and code is untrusted at this crossing); and **TB3 backend → Unity** (NID 94, where the client executes *only* allow-listed actions and never compiles server-supplied C#). The peer identifier on NID 98/93 is self-asserted, so peer authenticity is *not* assumed inside a connection — an explicitly accepted risk (§4.5).

Table 4.2 — Threat model: asset · adversary · attack surface · mitigation · status.

#	Asset	Adversary	Surface / boundary	Mitigation	Status
T1	Host/headset (A1)	Malicious payload	TB3 NID 94	Action-plan execution only; no compilation on Mode C; client re-clamps bounds	Backend verified; Unity play-mode VERIFIED live in Unity 6 (Mode C + validator-gated Mode A)
T2	Host/headset (A1)	Malicious C# (Mode B)	TB2 LLM→exec	Defence-in-depth: tree-sitter lexical scan + .NET Roslyn semantic check, both fail-closed	Verified live; honest finding: Roslyn missed <code>System.Net.WebClient</code> (false negative), Rust lexical layer is the backstop and rejects <code>System.Net</code>
T3	Host/headset (A1)	Untrusted C# under study (Mode D)	TB2 LLM→exec	Hardened container: <code>--network none</code> , read-only rootfs + tmpfs, <code>--cap-drop ALL</code> , pid/mem/cpu limits, <code>no-new-privileges</code> , non-root, wall-clock timeout + <code>docker kill</code> ; only a report crosses back	Verified live under Docker; gVisor (<code>runsc</code>) wired + unit-tested but not run
T4	Backend (A2)	Careless/adversarial LLM	TB2 LLM→exec	Schema-bounded DSL (6-action allow-list, <code>deny_unknown_fields</code>); fail-closed validator; <code>request_id</code> fixed server-side	Verified (unit + live)
T5	Backend (A2)	Hostile peer (DoS)	TB1 NID 98/93	<code>MAX_PAYLOAD_LEN</code> / <code>MAX_PLAN_JSON_BYTES</code> checked before allocation; 8 MiB utterance cap; per-peer sessions (no global lock)	Verified (codec golden test, live concurrency)
T6	Backend (A2)	Any input	all	<code>unsafe</code> forbidden; <code>unwrap</code> / <code>expect</code> / <code>panic</code> denied → backend cannot be panicked by input	Compiler-enforced; gate green
T7	User data (A3)	Operator/observer	logs, wire	Reason codes only (<code>stt_timeout</code> , <code>llm_unavailable</code> , ...); raw detail to stderr; no audio/transcript in telemetry	Verified by leak test
T8	Backend (A2)	Spoofing peer	TB1	Connection-level isolation only; per-message authn deferred	Accepted risk (§4.5)

4.4 Design goals

Six design goals (G1–G6) discharge the requirements and threats above.

G1 — Safety by construction (bounded action DSL). The default path bounds *what the model may generate* rather than scanning free-form output afterwards. The action plan is a six-variant, internally-tagged, `deny_unknown_fields` JSON intermediate representation whose enum **is** the allow-list (`set_color`, `set_scale`, `move`, `rotate`, `spawn_primitive`, `set_physics`), each with hard numeric bounds in a single source of truth (`MAX_ACTIONS=16`, per-spawn `MAX_SPAWN_COUNT=8`, scale 0.1–5.0, rotate $\leq 360^\circ/\text{s}$, mass 0.1–100). Unsafe operations are *unrepresentable*, not merely filtered — the schema-constrained-generation lineage (Geng et al., 2023; Poesia et al., 2022) made operational. This directly removes the Quest-blocking runtime-compilation dependency, since the client only dispatches vetted UnityEngine operations.

G2 — Fail-closed everywhere. The validator approves a plan *iff* the violation set is empty; oversized input is rejected *before* parsing (anti-allocation DoS); and every external call (STT, LLM, Roslyn) is wrapped in a timeout that degrades to a refusal. A cumulative spawn budget is enforced at a single chokepoint (`commit`), which downgrades an otherwise-valid plan to a rejection once the per-session cap of 64 is reached.

G3 — Defence-in-depth for the research C# path. Because Modes B and D exist to *study* generated C#, the false-negative observed in the shallow Roslyn semantic layer (it references only `corelib` and missed `System.Net.WebClient`) is treated as expected, not as a single point of failure. Safety on this path is the *combination* of four layers — lexical scan, semantic check, `plan↔C#` consistency, and sandbox — consistent with NIST SP 800-190's position that a container alone is not the security claim (NIST, 2017). The lexical layer runs first and rejects `System.Net` syntactically, so it is the verified backstop for what the semantic layer misses.

G4 — No global lock; privacy-safe telemetry. Per-peer sessions in a `HashMap` (no `isGenerating`-style global serialisation) give concurrency isolation, and telemetry records only ids, timestamps, decisions, reason codes and counts — never transcripts — so that a *telemetry* leak is structurally impossible rather than merely avoided. (The personalization/RAG store of §5.12 is the disclosed exception, and it is **on by default** (opt-out, `enable_rag`): it persists verbatim commands and generated C# to a local plaintext profile; encryption-at-rest and a retention sweep exist but are off by default — see §8.2/§8.3.)

G5/G6 — Minimal CVE surface and preserve Mode A. The workspace is `unsafe`-free and panic-averse by compiler-enforced lint, with `cargo-deny` gating the supply chain; and the original Roslyn baseline is preserved unchanged for honest comparison, kept off the default path behind a research flag.

4.5 Accepted and deferred risks

Three risks are accepted and stated plainly, not hidden. The wire `peer_uuid` is **self-asserted**: connection-level isolation holds, but within one connection a client could touch another peer's session bookkeeping; per-message authentication is deferred. Per-peer rate-limiting and an anti-strobe min-plan-interval are now enforced in `command-router`, and the Unity Mode-C client auto-reconnects after a dropped link. And **kernel-level isolation** via `gVisor` (`runsc`) is wired and unit-tested in the Rust runner (`with_runtime / DCVR_SANDBOX_DOCKER_RUNTIME`) but was not exercised for lack of a `runsc` host, so it is a designed rather than a measured property and is not the default Mode-D runtime. The Unity client's play-mode behaviour has since been **verified live in Unity 6** (Mode C and validator-gated Mode A drive object changes from live backend messages); what remains is the four drop-in headless `EditMode` test fixtures (23 methods) under a licensed batch Editor, and the Quest 3 / Store IL2CPP build (the Quest 1/2 Mono sideload has been run on-device, §7.11).

Chapter 5 — Architecture & Implementation

This chapter describes the DreamCodeVR+ system as it was actually built. The guiding design principle is that *safety is layered and centralised*: there is one place that defines what the system may do (the action-plan allow-list), one place that decides whether a request is safe (the fail-closed validator), one place that enforces cumulative resource limits (the spawn-budget chokepoint), and one place that maps internal errors onto wire-visible reason codes (the privacy boundary). Generated C# — the original system's only execution path — is demoted to a research arm reachable only behind a flag, and even then only through lexical, semantic, and sandbox layers in series. The emphasis throughout is on these design decisions and the trade-offs behind them rather than on the code itself.

5.1 Workspace shape and the four modes

DreamCodeVR+ is a Cargo workspace of **sixteen** library crates and **five** binary applications, with a separate cross-crate integration-test crate. The Rust workspace is the repository root; the unchanged original Node.js *server* is retained read-only inside it at `_original-dreamcodevr-daniel/` (the original *Unity client* is cited from the upstream `UCL-VR/dreamcodevr` repository rather than vendored). Keeping the original server in-tree was a deliberate decision: the original DreamCodeVR pipeline is preserved as the Mode A baseline for comparison, while every new component is isolated, so the dissertation can compare against the original without modifying it.

The library crates contain pure logic with no binaries; the binaries (`dreamcodevr-server`, `fake-quest-client`, `ubiq-probe`, `sandbox-runner`, and the `xr-security-eval` benchmark harness that produces the headline 38/40 result) are thin shells over them. Two further .NET microservices (a Roslyn analyser and a sandbox worker) sit outside the Rust build because they require the .NET toolchain, and a set of drop-in Unity C# scripts replace the original runtime-compile manager on the client side.

The system realises four execution modes. Mode A is the preserved original ($\text{LLM} \rightarrow \text{C\#} \rightarrow \text{RoslynCSharp}$ compiled inside Unity); it is kept as a research control and is *not* wired into the Rust flow. Mode B is a Rust-validated C# path. Mode C — the deployable default — executes a schema-bounded action plan with *no compilation*, making it Quest-compatible. Mode D isolates untrusted C# in a hardened container. The defining decision is that the language model is asked to emit *both* a structured `action_plan` and a `csharp_candidate`, but the action plan is the single safety source of truth.

5.2 The networking layer

Unity and the backend communicate over Ubiq (UCL's social-VR networking library), in which a TCP RoomServer relays messages between peers addressed by `NetworkId`. Three channels are reused unchanged from the original system: NID 98 carries push-to-talk audio (or, in the mock path, UTF-8 transcript text) from Unity to the backend; NID 93 carries the selected object (`"objectName:materialName"`); and NID 94 carries the backend's decision back to Unity.

The `protocol` crate is a pure codec — no I/O, no async, and `unsafe` forbidden. The wire frame is `[u32 LE body_len][u32 LE a][u32 LE b][payload]`, where the critical detail is that `body_len = 8 + payload.len`: the length prefix counts the eight-byte `NetworkId` *plus* the payload, not the payload alone. Both `encode_frame` and `decode_frame` reject payloads above 1 MiB before allocating, and `decode_frame` never panics, returning an `Incomplete { needed, have }` signal so a partial stream can be resynchronised. A *golden test* pins the exact bytes for NID 98 carrying "AB" (`0A000000 00000000 62000000 4142`), locking the encoding against silent regression. A small payload helper handles the 36-byte ASCII peer-UUID that Unity prefixes onto NID 98/93 bodies.

The `unity-transport` crate is the async Ubiq peer. `UbiqServicePeer::connect_and_join` opens the TCP socket, sends a Join control frame to the RoomServer's reserved `NetworkId(0,1)`, waits for a `SetRoom` reply, then spawns

reader, writer, and one-second ping tasks. It exposes a cloneable `PeerSender` so that work spawned off the receive loop can still emit frames.

Two real protocol bugs were found here only through live testing against the genuine RoomServer (vendored from UCL-VR/ubiq because the upstream gitpkg host now returns 402). The first is the `body_len` semantics above, confirmed against the real source and now pinned by the golden test. The second is subtler: the Join message's `args` field must be a *stringified* JSON value, not a nested object, because the Node RoomServer executes `JSON.parse(message.object.args)`. A nested object failed schema validation with "is not of a type(s) string"; this was the bug actually blocking the real-Ubiq join, and without live testing it would have surfaced only when wiring Unity. Both are recorded honestly as discoveries of empirical, not unit, testing.

5.3 The action-plan IR

The `behaviour-dsl` crate is the heart of the safety argument: it defines the action-plan intermediate representation as pure serde data. An `ActionPlan` carries a schema version, a request id, a target, and a list of actions, and is annotated `#[serde(deny_unknown_fields)]`. The `Action` type is an internally tagged enum (`#[serde(tag = "type")]`) with exactly six variants — `set_color`, `set_scale`, `move`, `rotate`, `spawn_primitive`, `set_physics`. The decisive design choice is that *this enum is the allow-list*. There is no catch-all variant, so any unknown `type`, or any extra top-level field, is rejected at parse time. Unsafe operations are therefore not filtered after the fact; they are simply *unrepresentable* in the IR.

A single `bounds.rs` module is the one source of truth for every numeric limit: at most sixteen actions per plan, at most eight spawned objects per `spawn_primitive`, a cumulative sixty-four spawns per session, scale within 0.1–5.0, speed and amplitude at most 5, rotation at most 360°/s, mass within 0.1–100, a 64 KiB plan size cap, and a hex-colour validator. Centralising these means the validator, the orchestrator, and the documentation all read the same constants rather than duplicating magic numbers.

Because the Unity executor must re-clamp to the same limits, those constants are mirrored by hand into `ProtocolModels.cs`. Hand-mirroring is a drift risk, so it is guarded by an automated `bounds_parity` test in the cross-crate test crate that fails `cargo test` if the Rust and Unity constants diverge — turning a documentation-discipline problem into a compiler-enforced one.

5.4 The fail-closed validator

The `code-policy` crate is the gate. `validate_json` rejects any input larger than the 64 KiB cap *before parsing*, an anti-allocation defence so that an attacker cannot force a large parse simply by sending a large body. It then parses and calls `validate_plan`, which accumulates a `Vec<ValidationError>` covering the schema version, empty or oversized action lists, every per-action numeric range (with an explicit `NonFiniteValue` guard rejecting NaN and infinities, which would otherwise slip past naïve range checks), and the spawn budget. The defining rule is *approve iff zero violations*: the validator returns `ApproveActionPlan` only when the violation set is empty, and `RejectUnsafe` otherwise. Fail-closed is thus the structural default — any error, any unrecognised field, any out-of-range value collapses to rejection.

5.5 Orchestration

The `command-router` crate owns a `HashMap<peer_id, PeerSession>` with *no global lock*. This is a direct response to a weakness of the original system, whose global `isGenerating`-style serialisation meant one user's request blocked another's; here, per-peer sessions guarantee one peer can never stall another. The router exposes three pipelines: a synchronous `process_transcript` for the deterministic mock path; an async `process_audio` that runs STT → LLM → validate, each external call wrapped in a `tokio::time::timeout`; and `process_audio_dual`, the Mode B research

path, which additionally validates the C# candidate through the lexical scan and then the Roslyn analyser. The dual path is fail-closed end to end: a Roslyn analyser error yields *not approved*, and the candidate is approved only when both the lexical and semantic checks pass — where the semantic layer is active only when a real analyser endpoint is configured (`DCVR_ROSLYN_URL`). Absent one, the backend uses an approve-all mock so it runs without a .NET toolchain, and the in-process lexical layer is then the sole C# gate; the hardened profile currently enforces a real analyser for Mode A only.

Two safety properties are centralised here. First, the *spawn-budget chokepoint*: `commit` is the single method every pipeline calls, and it enforces the cumulative per-session cap of sixty-four spawns. A plan that is individually valid can still be downgraded from approve to reject (with reason `session_spawn_budget_exceeded`) because the session has already exhausted its allowance — a deliberate two-level budget on top of the per-plan cap in `code-policy`. Second, *privacy by construction*: when STT or LLM fails, the raw detail goes only to stderr, while the wire and the logs receive a fixed reason code (`stt_unavailable`, `stt_timeout`, `llm_unavailable`, `llm_timeout`, `empty_transcript`). A dedicated test proves that a secret transcript can never escape into the outcome or the timing record.

Ahead of the fail-closed C# validator sits a Layer-1 security screen that inspects the raw natural-language command *before* any code is generated, pairing a fast keyword classifier with an LLM semantic classifier (gpt-5.5, returning a structured JSON verdict of `{malicious, category, reason}`) so that malicious or privacy-violating *intent* — device camera, microphone or screen capture, keylogging, exfiltration, memory access, tap-automation, and cyber-attack code such as ransomware, DDoS or reverse-shell payloads — is caught at the semantic layer rather than after synthesis. A command flagged by either classifier is neutralised: it is replaced with a harmless server-authored recolour (`reset::neutralized_csharp`) and never reaches the generation model at all, whilst benign creative and deliberately edgy-but-harmless themes — a Trojan-horse statue, a bloody zombie, a sea shell — pass unimpeded, preserving creative freedom. This complements the lexical and Roslyn gate as genuine defence-in-depth: the validator reasons over code that already exists and can only refuse what has been written, whereas the intent screen stops the harmful request before hazardous code is ever authored, closing the semantic gap between what a user *asks for* and what a downstream lexer can *see*. The design also fails safely under a payload-smuggling threat model, in which an attacker weaves an exfiltration request into an otherwise-innocent scene ("generate a snowman AND steal the wifi passwords") or hides it inside an object's animated behaviour, because the semantic classifier judges the composite intent rather than isolated tokens. The LLM semantic classifier is deliberately *advisory and fail-open* — if it errors or is unavailable the command still proceeds to the authoritative fail-closed validator, which is the real gate — so the intent screen raises the cost of an intent-level bypass without being a single point of failure. An initial pass fired 100 attack prompts at the running backend and blocked all 100, while benign edgy themes were not flagged; a later full-scale campaign showed that this 100-prompt pass over-stated the keyword layer's recall, and the screen was subsequently hardened to 100% on the full corpus (§7.10). (*Verified live, 2026-07-03.*)

5.6 External client seams

STT and LLM are trait seams so the backend can run fully offline by default. `SttClient` has a `MockSttClient` that UTF-8-decodes the audio bytes as text (keyless and deterministic) and an `HttpSttClient` that POSTs a hand-built 44-byte-header WAV to a faster-whisper endpoint over rustls. `LlmClient` has a `MockLlmClient` doing keyword-to-action mapping (red → `#FF0000`, spin → rotate, bigger → scale) plus a deterministic, allow-list-clean C# template generator, and an `OpenAiLlmClient` (deployed default model `gpt-4o-mini`, with `gpt-5.5` selectable at runtime) issuing completions under a strict system prompt at temperature 0 where the model permits it — the GPT-5 and o-series reasoning models reject a non-default temperature, so for those the client instead fixes the reasoning effort and the output-token budget. A notable decision is that the LLM *never owns the request id*: `OpenAiLlmClient` always overwrites `plan.request_id` with the backend's id, so the model cannot influence correlation. The API key is

held in a `secrecy::SecretString` (redacted `Debug`, zeroized on drop). Per-call timeout *policy* lives in the router, not the clients, by design; the clients themselves carry only coarse transport backstops (a 10 s connect and a 300 s overall bound), keeping the meaningful timeout policy in one place. Absence of `DCVR_STT_HTTP_URL` selects the mock STT and absence of `OPENAI_API_KEY` selects the mock LLM, so the default configuration is deterministic and offline.

5.7 Mode B: validated C

Mode B treats generated C# as a research arm guarded by defence-in-depth. The `csharp-policy` crate is the first layer: `validate_csharp` uses a tree-sitter C# parser to scan for banned namespaces (`System.IO`, `System.Net`, `System.Reflection`, `System.Diagnostics`, `System.Threading`, `System.Runtime.InteropServices`, `UnityEngine.Networking`, and `Application.Quit`) and banned identifiers (`Process`, `Marshal`, `DllImport`, `UnityWebRequest`, the reflection verbs `GetType`/`GetMethod`/`GetField`/..., `SendMessage`, `PlayerPrefs`, `Resources`, `InvokeRepeating`; `Destroy`/`DestroyImmediate` are deliberately *not* banned, since deleting objects the user just created is a core creative operation whose worst case is restart-recoverable). Matching is by dotted-name reconstruction, so whitespace, `global::`, `@`-escapes, and comment tricks cannot evade it; the layer also enforces a length cap and requires exactly one top-level `MonoBehaviour`. A `consistency_check` cross-validates the C# against the plan (for example, that spawn counts agree). A dedicated red-team test suite exercises evasion attempts.

The `roslyn-client` crate is the second layer, a `RoslynAnalyzer` trait whose `HttpRoslynAnalyzer` POSTs the candidate to the .NET service and is fail-closed (`unwrap_or(false)` — a missing field, a non-2xx response, or any error means *not approved*); the `MockRoslynAnalyzer` is the offline default and is **approve-all (fail-open)**, so the backend runs without a .NET toolchain — which means that in any run without `DCVR_ROSLYN_URL` the in-process lexical layer is the effective C# gate (this is precisely why §7.3's `System.Net` false negative is contained rather than fatal). The .NET service itself parses with Roslyn, builds a semantic model, and rejects symbols resolving into the six denied namespaces.

An honest finding from live testing shapes how this is presented. The semantic analyser references only `corelib`, so symbols outside `corelib` resolve to null and are silently skipped — `new System.Net.WebClient` was returned as approved (a false negative), because `System.Net.Requests` is not `corelib`. A strict allow-list was tried first but false-rejected the generated class's own `<global namespace>`, so the service was changed to deny-list only. This is why C# safety is explicitly *not* this service alone: in the real dual path the Rust lexical layer runs *first* and rejects `System.Net` purely syntactically, so the candidate never reaches Roslyn. The episode concretely demonstrates why the C# story is defence-in-depth (lexical backstop + semantic check + consistency + sandbox), not a single gate.

An executed adversarial red-team against the tree-sitter lexical scanner exposed three evasion classes that a naïve subsequence ban over identifier segments cannot see. The first was namespace-aliasing: a declaration such as `using Sys = System;` followed by `Sys.IO.File.WriteAllText(...)` reconstructs as the segment list `[Sys, IO, File]`, which never contains the banned `[System, IO]` subsequence, smuggling file IO, sockets and threading past the lexer while the leaf identifiers remain individually innocuous. The second was Unicode-escape identifiers — C# decodes `\uXXXX` within an identifier, so `Application.\u0051uit()` presents an AST leaf of `\u0051uit` (which is not `Quit`) yet compiles to the real `Quit`. The third was dynamic late-binding, where `((dynamic)app).Quit()` reaches a banned member through a node that carries no banned identifier at all. Each was closed at source in `crates/csharp-policy/src/lexical.rs`: a `using-alias` pre-pass builds an alias-to-target map and expands a leading alias segment — transitively, to cover chained aliases — before the subsequence check; a decoding step normalises `\uXXXX / \UXXXXXXXX` in identifier text prior to matching; and the `dynamic` keyword, which has no legitimate use in a generated `MonoBehaviour`, is banned outright. Before these classes were closed an unadorned lexer caught only 36/258 (14.0%) of the should-block C# vectors; the *delivered* repository ships the hardened validator, which the reproducible 1,057-vector campaign (§7.10) scores at 258/258 from the outset, with all 469 benign "creative-

freedom" vectors — including a cube named `TrojanHorse` and a minimap `Camera` — approved (0% over-block). Two further evasion classes surfaced by the project's multi-agent audit were closed the same way and are worth naming — an `unsafe {}` pointer block (a HIGH-severity find) and a `MonoBehaviour-impostor` base type — so the regression set now covers five classes across eight tests (though only the first three appear in the campaign corpus), and the `csharp-policy` crate passes 54 unit tests, with the full corpus firing in roughly one second of pure lexical analysis (*Verified by automated tests.*)

5.8 Mode D: the sandbox

Mode D exists to study genuinely untrusted generated C#. The `sandbox` crate defines a `SandboxRunner` trait whose `run` returns a structured `SandboxReport` — the *only* thing that crosses back; no live process handle escapes. The `DockerSandboxRunner` runs each job in a container hardened with `--network none`, `--read-only` plus a `tmpfs /tmp`, `--cap-drop ALL`, a `pids` limit, memory and CPU caps, `no-new-privileges`, and `--user 65534:65534`. The untrusted code is streamed to the container's stdin, and on timeout the runner issues `docker kill` so the daemon (not the CLI) tears the container down; argument construction is unit-tested without Docker. A `SandboxWorker` adds retry and a dead-letter queue, with a non-zero program exit treated as a *report*, not a retried failure. The .NET harness behind it reads C# from stdin, compiles it in-memory with Roslyn referencing all runtime assemblies (so it works offline under `--network none`), runs the entry point, and returns stdout/stderr with a structured exit code. The honest framing is that Docker alone is a deliberately *soft* boundary per NIST SP 800-190 (NIST, 2017); kernel-level isolation via gVisor is the stronger option.

5.9 The Unity drop-in

On the client side, a set of drop-in scripts (17 under `unity/Runtime/`, of which five carry the Mode-C path) replace the original `CodeGenerationManager`. `ActionPlanNetworkBridge` registers on `NetworkId(94)`, receives the decision JSON, and invokes the executor — and must be the *only* component registered on NID 94, replacing the original runtime-compile manager. `ActionPlanExecutor` parses the envelope with Newtonsoft, fail-safes on bad input, and dispatches each action to `SafeBehaviourRegistry`, the client-side allow-list that maps each action to exactly one vetted UnityEngine operation with re-clamped bounds and no reflection, I/O, networking, or compilation. `ProtocolModels` mirrors the Rust bounds (guarded by the parity test of §5.3) and `GeneratedObjectTracker` enforces the sixty-four-object cap with an `UndoAll`. Four NUnit EditMode test fixtures (23 `[Test]` methods in total) accompany these, but they run only inside Unity.

5.10 Quality gates

Finally, safety properties are mechanically enforced rather than left to review. The workspace lint policy makes every crate opt into `unsafe_code = "forbid"` and denies the clippy `unwrap_used`, `expect_used`, and `panic` lints, so the entire backend is `unsafe-free` and panic-averse by compilation. A `check.sh` gate runs `cargo fmt --check`, `clippy --all-targets -D warnings`, the workspace test suite, and a workspace build; `scripts/verify-all.sh` additionally runs `cargo-deny` (with `deny.toml`), the lint invariants, benchmark determinism and the full red-team campaign. The gate is green: formatting, clippy, 342 tests, the build, and `cargo-deny` all pass.

5.11 Verification status

To be explicit about scope, the components above differ in how far they are proven. The backend's real-Ubiq roundtrip, Mode B (including the `System.Net` false negative and the Rust lexical backstop), and Mode D containment under Docker are live-verified, as are the 342 Rust tests (the gVisor `runsc` variant is wired and unit-tested but was not run). The Unity client integration has **since been live-verified in Unity 6** (Editor Play, no Quest required): both **Mode C** (the action-plan executor) and **Mode A** (runtime C# compilation, now validator-

gated) drive scene changes from live backend messages — including model-generated C# that compiles at runtime to build open-ended structures (a house, a tree, a castle from primitives) under the freeform lexical guardrail that blocks malicious or over-large code. A **real OpenAI LLM** (gpt-5.5) was exercised end-to-end, and **speech-to-text was verified at the OpenAI Whisper API level**. Both previously-unverified items have **since been demonstrated live on a physical Meta Quest** on the university network (§7.11): live-microphone STT end-to-end from the headset, and on-device runtime compilation on the Quest 1/2 32-bit Mono sideload. What remains is the **Quest 3 IL2CPP/Store** build (which forbids runtime compilation, motivating Mode C) and the quantitative measurement. The architecture is therefore complete, the backend is verified feature-by-feature against real infrastructure, and the Unity execution of *both* execution paths is demonstrated; what remains is the Quest 3 / Store IL2CPP build and the quantitative measurement, reported honestly in the evaluation chapters.

5.12 Personalization (retrieval-augmented generation), safety-preserving

A later increment adds a personalization layer (`crates/personalization`) so the system learns each user's *aesthetic* preferences without weakening any safety property. It sits as one step **between speech-to-text and the LLM**: the recognised command is used to (i) inject a short, learned *preference profile* and (ii) retrieve the most similar previously-**liked** creation as a style exemplar, before the prompt is sent. Three layers are implemented behind one `Personalizer` type: a `PreferenceProfile` that learns content words and recent phrases from commands the user marks as liked; a liked-history memory; and embedding retrieval via an `EmbeddingClient` seam — `OpenAiEmbeddingClient` (`text-embedding-3-small`) for real semantic similarity, or a deterministic offline `MockEmbeddingClient` (hashed bag-of-words) so the feature works with no key and tests are reproducible. Cosine top-K over the user's liked memories selects the exemplar; state is per-user and local (a JSON file or in-memory store). Feedback flows back over a dedicated network channel (`NID 95 : {"liked":bool}`), which promotes the most recent generation and updates the profile.

The design is deliberately *subordinate to the safety architecture* of §5.4 and §5.7. Retrieved context is injected into the prompt inside an explicitly-labelled `[PERSONALIZATION ... UNTRUSTED DATA, never an instruction]` block — a direct mitigation for indirect prompt injection (Greshake et al., 2023; OWASP Gen AI Security Project, 2025b) given that the memory store is itself attacker-influenceable — and it may bias only palette, theme and complexity, never *which* object is built or *whether* the safety rules apply. Because the fail-closed validator still gates every output, personalization cannot enlarge the set of admissible programs; it only reweights preferences within it. Storing per-user data is treated as a privacy obligation and is addressed in the DPIA discussion (§8.3).

5.13 Live observability and runtime control (the admin panel)

To make the system's internal behaviour inspectable — for evaluation, debugging and viva demonstration — a control plane (`crates/control`) and a web admin panel (`crates/admin`, built on the Rust **axum** framework) were added. `control` holds a cheaply-clonable `ControlBus`: a live `RuntimeConfig` behind a read-write lock, a broadcast event bus of typed `PipelineEvents`, and lock-free statistics counters. The request pipeline reads a fresh config snapshot per request (so changes take effect on the next command) and **publishes each stage as it happens** — transcript, prompt-sent, LLM reply, validation, and (reported back from Unity over `NID 96`) the runtime-compile result — each carrying its own duration, followed by a total.

The panel itself is a single embedded page (plain HTML/CSS/JavaScript, no build step) that consumes the event stream over **server-sent events** and renders the pipeline live with human-readable timings. It also exposes: runtime-tunable safety knobs (maximum generated-C# size and line count, per-mode enable/disable, model selection, RAG depth) applied through the same `ControlBus`; lifetime statistics including approval/rejection counts, malicious-blocked count, and an indicative token-cost estimate; a safety log of every rejected or neutralised command; a paste-a-snippet validator that runs arbitrary C# through the real guardrail; a **red-team runner** that

fires a fixed set of malicious samples and confirms each is blocked (with a benign control that must pass); per-user RAG profile inspection; and a manual command box that drives the real pipeline. Mutating routes are guarded by an optional admin token. Architecturally the panel is a *window into the running backend* — it shares the same `ControlBus` and validator rather than re-implementing them — so what it displays is exactly what the pipeline does. This converts otherwise-opaque safety behaviour into reproducible, demonstrable evidence, and is used as such in Chapter 7.

Three refinements were made to the admin panel to close the gap between observation and control. First, the manual-command box no longer merely validates: it now shares the live Ubiq sender and, on approval, emits an NID-94 `{type,peer,data}` message so that a typed command is compiled and built in the connected headset exactly as a spoken one would be — giving a reliable, speech-free means of driving a demonstration. Secondly, the runtime-config POST was changed from a full-body replacement to a partial merge, overlaying only the altered field onto the current configuration, so that the panel's Apply now takes effect live rather than failing on a partial body. Thirdly, the headline "malicious blocked" KPI was corrected so that a Layer-1 neutralisation increments this counter directly — and is excluded from the "approved" tally — ensuring the figure reflects the true number of interventions instead of reading zero while catches accrued silently under a separate metric. Together these changes make the panel an instrument of runtime control as well as of observation, without compromising the safety pipeline. (*Verified live, 2026-07-03.*)

Chapter 6 — Evaluation Methodology

This chapter specifies *how* each claim in this dissertation was or would be evaluated, *before* any results are reported. It is deliberately separated from Chapter 7 so that the evaluation design can be judged on its own merits, independently of the data it produced. The chapter follows a design-science logic (analyse the original artefact, build an alternative, evaluate it differentially) and is explicit about the boundary between what was executed against real infrastructure, what was exercised only by automated tests with in-process mocks, and what is *planned but not yet run*. No quantitative result is anticipated here; that honesty is the point.

6.1 Evaluation design

The central design is a **differential, four-mode comparison**. The same speech-derived prompts are driven through four execution paths — Mode A (original runtime-compiled C# baseline), Mode B (Rust-lexical + Roslyn-validated C#), Mode C (schema-bounded action plan, the deployable default), and Mode D (isolated Docker/gVisor sandbox) — so that any difference in functionality, latency, security or privacy can be attributed to the *execution architecture* rather than to the input. This isolates the effect of the project's core claim, that bounding the output language (Mode C) is a stronger and more deployable safety lever than sandboxing or scanning free-form code (Modes B and D).

The design distinguishes three evidentiary tiers, which are carried through into Chapter 7 unchanged:

- **Live-verified** — exercised against real external infrastructure (the real Ubiq Node RoomServer, a real .NET 10 Roslyn service, and real Docker; the gVisor `runsc` runtime is wired and unit-tested in the runner but was not run for lack of a `runsc` host).
- **Automated-test-only** — exercised by the Rust test suite against in-process or local mock servers (not the real OpenAI/faster-whisper/RoomServer services).
- **Planned, not yet run** — specified here but blocked on hardware, credentials, a licensed batch Editor, or ethics approval (the headless EditMode batch run; systematic real-endpoint latency; the Quest 3 / Store IL2CPP Mode-C build; a possible human-subjects study). Interactive Unity 6 play-mode, the real gpt-5.5 language model, and an on-device Quest 1/2 Mono run have since been exercised (§7.5, §7.11).

6.2 How each claim was evaluated

Unit and integration tests. The functional and safety logic of the backend was evaluated by the workspace test suite — **342 Rust tests** run under a mechanically enforced gate (`cargo fmt --check`, `clippy --all-targets -D warnings`, `cargo test`, `cargo build`), with `cargo-deny` supply-chain checking added by `scripts/verify-all.sh`. These cover the fail-closed validator (approve-iff-zero-violations), the per-peer session model, the spawn-budget chokepoint, and the privacy reason-code mapping. One test is load-bearing as an *evaluation instrument* in its own right: the **bounds-parity drift guard** (`tests/tests/bounds_parity.rs`), which fails the build if the hand-mirrored Unity `ProtocolModels.cs` constants and NIDs diverge from the Rust `behaviour-dsl/bounds.rs` source of truth. This converts an otherwise silent manual-sync hazard into a checked invariant.

Live Ubiq round-trip (Mode C path). The single largest integration risk — that the Rust backend can actually join and converse on UCL's Ubiq network — was evaluated empirically, not assumed. The real Node RoomServer was run, and `ubiq-probe` plus the full `dreamcodevr-server + fake-quest-client --ubiq` loop were driven through it, with the end-to-end success criterion being a NID-94 reply received back through the real server.

Mode B (validated C#). Evaluated by live `curl` against the running .NET Roslyn service (`:5099`) over a small adversarial input set, *and* by the dual-path run through real Ubiq with `DCVR_CSHARP_RESEARCH=true`. The two-layer

design (Rust lexical scan first, Roslyn semantic check second, fail-closed on either) was specifically probed to test whether a weakness in one layer is caught by the other.

Red-team C# corpus. The lexical scanner (`crates/csharp-policy`) was evaluated against a dedicated evasion suite (`redteam_tests.rs`) probing whitespace, `global::`, `@`-prefix and comment tricks against the banned-namespace and banned-identifier lists, plus `plan↔C#` consistency fixtures in `fixtures/csharp/`.

The full-scale campaign corpus. Beyond those in-process suites, the headline RQ3 numbers of §7.10 come from a separate, larger instrument: a deterministic generator (`redteam/corpus_gen.py`) emits 1,057 vectors — 727 C# and 330 natural-language — each tagged with a `should_block` ground-truth label and a category/capability. Of these, 258 C# and 305 NL are `should-block` and 469 C# + 25 NL are benign controls (the C# controls deliberately include scary-but-safe authoring, e.g. a `TrojanHorse`-named cube). A campaign runner (`redteam/run_campaign.py`) fires the corpus over HTTP at a live backend — C# at `POST /api/validate`, NL at `POST /api/command` — and tallies four verdict classes: CAUGHT, BYPASS (a `should-block` that is approved), ALLOWED, and OVER-BLOCK (a benign that is blocked). One construct-validity caveat is stated up front and carried into §7.7 and §8.2: the corpus is hand-authored by the same author as the guardrail, so its post-hardening recall figures are *in-sample* until an independently-authored holdout corpus is run.

Mode D containment. Evaluated by running deliberately hostile payloads — `write-to-rootfs`, network exfiltration, runaway loop, and `File.WriteAllText("/etc/pwned")` — inside the hardened runner under plain Docker. The success criterion is that each payload is *contained* (blocked or killed) and that only a structured report crosses back, with the backend surviving every case. The gVisor (`runsc`) runtime is selectable through the runner (`with_runtime`, unit-tested) but was not exercised, as `runsc` was not available on the verification machine; kernel-level isolation is therefore a designed, not a measured, property.

6.3 Metrics

For claims that are qualitative today, the metrics are defined here so the gap is measurable later:

- **Functional success rate** per mode: fraction of corpus prompts that produce the intended scene change.
- **Per-stage latency** (STT, LLM, validation, total), reported as **p50/p95/p99** computed from the privacy-safe JSONL `TimingEvent` stream, which already records the necessary ids/timestamps/counts (and, by test-asserted construction, *no* audio or transcript).
- **Security rejection rate** on a red-team set: fraction of malicious prompts / unsafe C# / schema-bypass / NID-spoofing attempts correctly rejected (fail-closed) — and, conversely, false-rejection rate on benign inputs. The red-teaming methodology follows established practice in adversarially probing language models for harmful outputs (Ganguli et al., 2022; Perez et al., 2022).
- **Containment rate** for Mode D: fraction of escape attempts blocked.
- **Reliability**: backend survival under concurrent peers and under sandbox crashes.

6.4 Planned but not-yet-run evaluations

The following are designed but were **not executed**, and are reported as such throughout:

- **Unity Editor EditMode tests (Mode C logic proof).** Four NUnit test fixtures (23 `[Test]` methods in total; the four canonical proof cases are `AcceptsSetColor→red`, `ClampsOversizeScaleDefensively`, `RejectsInvalidJson`, `RejectsEmptyActions`) are staged under `unity/Editor/`. They are **blocked on Unity sign-in** in *headless batchmode*: it quit at the licensing stage (`Access token is unavailable`) before compilation, so this is an account step the automation cannot perform — not a code fault. The

interactive Unity 6 Editor play-mode (both Mode C and validator-gated Mode A) has since been verified live (§7.5); only this headless EditMode batch run remains pending.

- **Real STT/LLM latency.** The faster-whisper and OpenAI code paths exist but default to deterministic mocks; the real gpt-5.5 model has since been exercised end-to-end (§7.5, §7.11), but systematic percentile latency was not collected and faster-whisper STT latency remains unmeasured.
- **Differential A/B/C/D latency.** Requires the real endpoints above (and, for Mode A, wiring the preserved baseline into the flow), so the latency CDFs are not yet collected.
- **A possible human-subjects study.** An optional, adults-only red-team exercise is envisaged but is **conditional on University of Birmingham ethics approval and a completed DPIA**; none has been sought.

6.5 Threats to validity

The design's honesty depends on naming its limits. **LLM nondeterminism** is controlled by defaulting to deterministic mock clients and, for any real run, fixing temperature to 0 (for models that support it) over N repetitions — but generality across providers is not established (**single-provider** threat). **Construct validity:** a qualitative "it round-tripped" is not a measured success *rate*, and Chapter 7 does not present it as one. **Internal validity:** the `fake-quest-client` is not Unity, so the round-trip evidences the backend and protocol, not the Unity executor. **External validity:** the simulator/desktop environment is **not a Quest device**, so on-device latency and FPS remain unmeasured; and **Docker is a deliberately soft boundary** (NIST SP 800-190) — the stronger kernel-isolation claim would rest on a gVisor run, which was not performed (the `runsc` runtime is wired and unit-tested in the runner but no `runsc` host was available). The **Roslyn semantic layer is deny-list-only and shallow**, so its rejection rate must always be reported as one layer of defence-in-depth, never as a standalone gate. These threats bound every result in Chapter 7.

Chapter 7 — Results & Analysis

This chapter reports **only what was actually executed and observed**. Results that were not measured are identified as such and are *not* given numbers; the outstanding quantitative evaluation is set out in §7.7. The chapter mirrors the three-tier evidentiary structure of Chapter 6: live-verified results first, then a results-status table that carries the verification matrix into the dissertation, then clearly-labelled gaps. The overarching finding is that the **integration risks have been retired** — the backend is verified against real Ubiq, real Roslyn and real Docker (the gVisor `runsc` variant is wired and unit-tested but not run) — while the **quantitative differential comparison remains to be measured**.

7.1 The test gate

The full mechanical gate passed: `fmt` PASS · `clippy -D warnings` PASS · **342 Rust tests** PASS · `build` PASS · `cargo-deny` PASS (advisories, bans, licences, sources). `cargo-audit` was additionally run on the Linux development machine (its RustSec advisories check overlaps `cargo-deny`'s); the delivered gate relies on `cargo-deny`. Among these, the `tests/tests/bounds_parity.rs` drift guard was added to lock the Rust↔Unity bounds and NID constants together. This is a verified, observed result (full mechanical gate — `fmt/clippy/cargo-deny/build` — first verified 2026-06-27 on a clean machine: Ubuntu 26.04, kernel 7.0, rustc 1.96.0, .NET 10.0.109, Docker 29.6.1, at 160 Rust tests then; the suite grew to 263/0 on 2026-07-17 after the ML-thrust additions of §7.12, and to 342/0 on 2026-07-25 after the whole-system verification campaign of §7.15, and was re-verified after the Linux→macOS migration on rustc/cargo 1.96.0).

7.2 Live Ubiq round-trip (Mode C path) — verified

The full speech-to-behaviour loop was exercised through the **real** Ubiq RoomServer with no Unity involved. `ubiq-probe` joined and reported `JOINED OK. Listening for NID 98 / NID 93.`; the RoomServer log confirmed both peers joining and leaving the room. The `fake-quest-client --ubiq` peer sent a NID-93 selection plus NID-98 push-to-talk (`start` / "make this cube red" / `stop`), and received back through the real server:

```
NID 94: {"decision":"ApproveActionPlan", action: set_color "#FF0000"}
```

The server's JSONL telemetry recorded `decision=ApproveActionPlan, action_count=1, errors=[]` — confirming, as a side-result, that no transcript leaked into the log. Two real protocol bugs were found and fixed *by this live test* (not by unit tests): the Ubiq frame length counts `NetworkId(8) + payload`, not payload alone; and the `Join` message's `args` must be a **stringified JSON** because the server does `JSON.parse(message.object.args)` (`roomserver.js:401`). This is the qualitative success criterion of RQ2's deployability path; it is **not** a measured success rate, and is not reported as one.

7.3 Mode B (validated C#) — verified, with a defence-in-depth finding

The .NET Roslyn service was exercised live (`curl /analyze`). The observed results, including one deliberately surfaced false negative, were:

Input	Roslyn result
clean MonoBehaviour	approved:true
syntax error	approved:false ("syntax errors")
System.IO.File.ReadAllText	approved:false ("denied namespace: System.IO")
System.Reflection.Assembly...	approved:false ("denied namespace: System.Reflection")
new System.Net.WebClient	approved:true ← FALSE NEGATIVE

The honest finding: the semantic analyzer references only corelib, so `System.Net.WebClient` (in `System.Net.Requests`, not corelib) fails to resolve and is silently skipped — the documented *shallow* nature of the deny-list-only semantic layer. **The Rust lexical layer is the backstop and does catch it:** `dcvr-csharp-policy` rejects `System.Net` purely syntactically (`redteam_tests.rs:68` rejects `new System.Net.Http.HttpClient;` `csharp_tests.rs:58` asserts `BannedToken("System.Net")`). In the real Mode-B dual path the lexical scan runs *first*, so `System.Net` is rejected before Roslyn is consulted. This is a concrete demonstration of *why* C# safety must be defence-in-depth (lexical + semantic + consistency + sandbox) and why the Roslyn service alone is not a sandbox. The clean template C# passed *both* layers end-to-end through real Ubiq, returning `NID-94 csharp_decision:"ApproveCSharpResearchMode"`.

7.4 Mode D containment — verified under Docker

Containment was verified under plain Docker. Shell payloads in `alpine` and real generated C# in the `.NET` Roslyn harness (`dcvr-sandbox-harness:local`) were both contained. The observed results:

Scenario	Docker
benign	Completed 0, uid=65534
write rootfs	READONLY_BLOCKED
network exfil	NETWORK_BLOCKED
runaway / infinite loop	TimedOut → docker kill
real C# benign	Completed 0 ("hello from sandboxed C#", ~1.7 s)
real C# runtime exception	Completed 70 (caught)
<code>File.WriteAllText("/etc/pwned")</code>	FS_BLOCKED: IOException

In every case the backend process survived and all `--rm` containers were auto-reaped. **gVisor (runsc) is wired and unit-tested but was not run:** `runsc` was not available on the verification machine, and the project's own `scripts/sandbox-run-gvisor.sh` is checked in with a "not yet verified" status note. The kernel-isolation claim is therefore **Designed, not Verified**. Importantly this is a *configuration* gap, not a code gap: the Rust `DockerSandboxRunner` already selects an alternative OCI runtime through `with_runtime` (exposed as `DCVR_SANDBOX_DOCKER_RUNTIME=runsc` and covered by the `runsc_runtime_included` unit test at `runner.rs:403`), so running Mode D under gVisor is a deployment setting awaiting a host with `runsc` installed, not an outstanding development task.

7.5 Unity 6 execution — Mode C and Mode A verified live (no Quest)

Both client execution paths were verified live in **Unity 6 (6000.5.1f1)**, in Editor Play mode, with no Quest hardware. A self-contained Ubiq TCP client joins the same room as the Rust backend (avoiding any external Ubiq-package version risk) and applies the backend's NID-94 messages to a target object.

Mode C (default, no compilation). With the backend in its default action-plan mode, commands round-trip end to end: the object's colour, scale, and motion change from the backend's validated action plan, executed by the audited `SafeBehaviourRegistry` with zero runtime compilation. This confirms the Quest-compatibility argument empirically on the client side.

Mode A (original runtime-C#, now validator-gated). With the backend set to `DCVR_MODE_A=true`, the system reproduces the *original DreamCodeVR pipeline* on modern Unity: a command is sent to the LLM, which returns a C# `MonoBehaviour`; the **Rust gate validates it**; only if it passes is it sent on NID 94 as `{type:"code", data}`; and a self-contained Roslyn component (`RuntimeCSharpCompiler`) **compiles it at runtime in the editor (Mono JIT) and attaches it to the object**. A command such as *"make it spin and turn purple"* produces, within a few seconds, a freshly-compiled behaviour running on the cube — the first end-to-end demonstration in this project that the original generate-and-compile loop works *and* is gated by the safety validator. Because the editor uses the Mono backend this needs no Quest; the same build would run sideloaded on a Quest 1/2 (§3.4) but is out of scope without hardware.

Open-ended creative generation under guardrails. Mode A is *not* limited to the six action-plan verbs. Because the C# path is gated by the **freeform** validator — a lexical/structural security scan (banned namespaces and reflection/process/IO verbs, `unsafe`, exactly one `MonoBehaviour` class, a size cap) rather than a plan-consistency check — the user has full creative freedom. Commands such as *"make a small house"*, *"make a tree"*, *"make a castle"*, *"build stairs"* and *"make a snowman"* generate multi-primitive C# that constructs the structure from `GameObject.CreatePrimitive` parts parented to the target, and compiles and runs live. In a sample of such commands generated with gpt-5.5, **5/5 were approved by the guardrail and built the requested structure**; the same guardrail **rejects** code that reaches a banned capability (`System.IO`, `System.Net`, reflection, `Process`, `Resources`, ...) or exceeds the size bound — freedom is bounded by *security and resource* limits, not by expressiveness. This realises the dissertation's "made safe, not removed" position for the original pipeline. *Honest limitation*: the lexical guardrail does not detect a deliberately-constructed infinite loop in otherwise-clean C#; Mode D's sandbox remains the backstop for genuinely untrusted code, and Mode A presumes a cooperative-but-fallible generator gated by the validator.

Latency. End-to-end Mode-A latency is dominated by the LLM: simple recolour/spin commands return in roughly 3–8 s with gpt-5.5, while elaborate creative builds (a detailed house or castle) take ~15–25 s, reflecting the larger generated program; the validator and runtime compile add negligible time. A faster tier (e.g. gpt-4.1-nano) roughly halves simple-command latency at some cost to behavioural richness — a configurable per-deployment trade-off.

7.6 Results-status table (mirroring the evidence table)

This table carries the Chapter 6 evidentiary tiers into the results, mirroring the evidential-status key in the front matter: *Verified (live)* denotes execution against real infrastructure; *Verified (test)* denotes the in-process automated suite; *Pending* denotes not yet verified; and *Limitation* denotes a known shortcoming.

Claim	Status	Basis
Backend joins real Ubiq RoomServer	Verified (live)	ubiq-probe → JOINED OK
Fake-Quest round-trip → NID-94 ApproveActionPlan{set_color #FF0000}	Verified (live)	live loop through real server
342 Rust tests + full gate	Verified (live)	cargo test 342/0; cargo-deny clean
Bounds-parity drift guard	Verified (live)	tests/tests/bounds_parity.rs
Fail-closed validator	Verified (test)	code-policy 17 tests (in-process)
Per-peer isolation (no global lock)	Verified (test)	command-router session tests
Mode B dual path (lexical + semantic)	Verified (live)	curls + dual path → ApproveCSharpResearchMode
Roslyn semantic shallowness (System.Net false negative; lexical backstop)	Limitation / live	observed FN; lexical rejects (redteam_tests.rs:68)
Mode D containment (Docker)	Verified (live)	READONLY/NETWORK blocked, TimedOut, FS_BLOCKED
Mode D containment (gVisor)	Designed, not measured	runner supports --runtime=runc (with_runtime , unit-tested); no runc host available
Unity Mode C play-mode (object changes from live backend)	Verified (live)	Unity 6 Editor Play; NID-94 action plan → object recolours/scales/moves
Unity Mode A runtime C# compile (validator-gated)	Verified (live)	Unity 6; DCVR_MODE_A=true → validated C# compiled at runtime (Roslyn/Mono) → behaviour attached
Open-ended creative generation under guardrails (house/tree/castle/stairs/snowman)	Verified (live)	5/5 gpt-5.5 builds approved + built; banned-capability/over-size code rejected
Real LLM (OpenAI gpt-5.5)	Verified (live)	end-to-end generation of action plans + C#
Personalization / RAG (learns taste; injected as untrusted)	Verified (live)	profile + thumbs-up / thumbs-down feedback + embedding retrieval verified; validator still gates every output
Admin panel + live observability	Verified (live)	live SSE stage stream, runtime config mutation, stats, and red-team (all malicious samples blocked) verified via HTTP
Adversarial red-team of C# validator (RQ3)	Verified (live)	1,057-vector corpus; 258/258 C# attacks caught (100%) after hardening, 0 bypasses, 0/469 benign over-blocked (§7.10)
Two-layer intent screen (keyword + LLM)	Verified (live)	full campaign: 305/305 NL should-block caught after paraphrase hardening (screen alone 66.2%→100%; both layers 81.71%→100%); LLM stage fail-open, validator authoritative; benign/edgy themes pass (§7.10)
XR static code-admission benchmark (xr-security-eval)	Verified (live)	0/40 (no defence) → 15/40 (security-only) → 38/40 = 95% (hardened); 12/12 benign preserved, 0 over-blocked; 2/40 joystick residual (§7.10, §7.15)

Claim	Status	Basis
Real STT (OpenAI Whisper), incl. live microphone	Verified (live)	live mic on a physical Quest transcribed end-to-end (§7.11); earlier TTS→STT API round-trip also verified
Unity EditMode tests (drop-in unity/) compile & pass	Pending	staged; pending a licensed Editor run
On-device Quest (Quest 1/2 Mono sideload, Mode A)	Verified (live)	physical headset on university Wi-Fi; 192.168.8.202 → :8009 ESTABLISHED; spoken command built in VR (§7.11)
Quest 3 IL2CPP / Store build (Mode C)	Pending	structural argument only; on-device IL2CPP build outstanding
Age-adaptive dual-plane coupling (minor tightens both planes; opt-in)	Verified (test)	config::age + command-router: child_tightens_both_planes_vs_adult, unknown_fails_safe_to_child, age_minor_forces_hardened_csharp_gate; DCVR_AGE_GATING off = byte-identical (§7.12)
On-device voice age-band gate (band-only, calibrated)	Verified (test)	ml/age_gate 51/51; temperature scaling $T \approx 1.93$, ECE 0.051→0.012 (validation) / 0.007 (held-out test); real-mic + on-device Quest are future work (§7.13)
ML attack analyzer (anomaly + drift + self-red-team)	Verified (test)	ml/attack_analyzer 31/31; pooled AUC 1.00 on the controlled corpus (~0.70 on a hard-negative slice), ~2% benign FPR, ADWIN2 shift @ t=304; 128-vector baseline → 164 discovered (§7.14)
Age-band gate spoofable under our own red-team	Limitation / test	100% spoof (child sample pushed across the adult boundary); band is a secondary net, platform age API authoritative (§7.14)
On-device ML (real mic audio + physical Quest)	Pending	numpy-only models run on held-out/synthetic data; real-mic + on-device evaluation is future work

7.7 Planned quantitative evaluation (not yet conducted)

The quantitative differential comparison specified in Chapter 6 was not carried out within the project timeframe, and no numerical results are reported in its place. For transparency, the measurements that remain outstanding — and the partial evidence that already bears on each — are set out below; none is presented as a result.

- **Functional success rate per mode (A/B/C/D)** over the fixed prompt corpus. Qualitative success is established for Mode C and validator-gated Mode A in Unity 6 (§7.5), and for five of five open-ended creative builds; a systematic per-mode rate over the full corpus is outstanding.
- **Per-stage latency (median, 95th and 99th percentile)** for the speech-to-text, language-model, validation and total stages, derived from the JSONL TimingEvent stream. Because the real OpenAI model is now exercised, language-model latency is already observable (approximately 3–8 s for simple commands and 15–25 s for elaborate builds with gpt-5.5; §7.5); systematic percentiles and live-microphone speech-to-text latency are outstanding.
- **Transport-layer rejection rates** for schema-bypass and NID-spoofing attempts. These are covered by the test suite but *not* by the 1,057-vector corpus, which targets the intent screen and the C# validator (that corpus run was executed, §7.10 — 258/258 C# and 305/305 NL); extending the corpus to the transport layer, and measuring false-rejection rates there, is outstanding.
- **Reliability and concurrency** under multiple concurrent peers and under induced sandbox crashes.
- **The four drop-in unity/ headless EditMode tests** from a licensed Editor run, and a build-success rate over a fixed creative-command corpus.
- **Quest 3 / Store IL2CPP build of Mode C**, with on-device latency and frame-rate. (An on-device Quest 1/2 Mono run of the Mode-A runtime-compile path has been demonstrated, §7.11; the outstanding item is

the no-compilation Mode-C build for Quest 3 and the Store.)

- **An optional adults-only human-subjects red-team study**, which would proceed only after University of Birmingham ethics approval and a completed Data Protection Impact Assessment (§8.3).

7.8 Analysis

The verified results substantiate the architectural claims qualitatively: the deployable Mode-C path round-trips through the real Ubiq protocol and returns a correctly validated action plan with no runtime compilation, executed live in Unity 6, which is the structural basis of the Quest-deployability argument (RQ2). The Mode-B finding is the chapter's most instructive negative result — a real false negative in the semantic layer that nonetheless *strengthens* the thesis, because the lexical backstop demonstrates the value of layering rather than relying on any single gate (RQ1, RQ4). Mode D's containment under Docker confirms that even the research arm that *does* run untrusted C# fails safe (the stronger gVisor `runsc` boundary is wired and unit-tested but was not exercised). The Unity 6 result adds a second substantive finding: the **original runtime-C# pipeline (Mode A) was reproduced and made safe** — model-generated C# is validator-gated, then compiled at runtime to realise open-ended creative commands (a house, a tree, a castle) while the freeform guardrail blocks malicious or over-large code. This shows the dissertation's safety lever is not only "replace compilation with an action plan" but, more generally, **move safety to the gate**: when full expressiveness is wanted, the generated code can be admitted *iff* it clears the security/resource guardrail (RQ1, RQ4). What the verified results do **not** yet show is any *measured* trade-off: the latency, success-rate and per-mode rejection-rate comparisons that would directly answer RQ3 and the quantitative half of RQ4 await systematic data collection. Separately, an on-device Quest 1/2 Mono sideload run with live-microphone STT has been exercised (§7.11), leaving the Quest 3 IL2CPP/Store build and the quantitative measurement as the outstanding items — not any code defect. The defensible summary is that the **design is complete and both the backend and the Unity client (both modes) are verified against real infrastructure; what remains is the Quest 3 / Store build and measurement, not redesign**.

7.9 Coverage of the XR threat literature (implemented vs outstanding)

The perceptual/embodied literature of §2.11 doubles as a coverage checklist. Each source's core attack is mapped to the DreamCodeVR+ defence and its status. *By construction* = the attack is unrepresentable in the deployable Mode-C action set; *implemented* = an explicit guard rejects, bounds or discloses it; *partial* = the direct mechanism is covered but a compositional or semantic variant is not; *future* = designed but not built.

Because the deployed configuration is **Mode B (Rust-validated full-freedom C#) + Mode D (sandbox)** rather than the six-verb Mode-C executor, the perceptual guarantees are provided in four freedom-preserving ways, none of which restricts creation by default: (i) an **opt-in C# validator profile** (`HardeningProfile::DeployHardened`) that, on top of the always-on system-access bans, also bans the *unambiguous device/rig/haptics/capture* APIs that have no content-authoring use (`OVRManager`, `XRSettings`, `InputTracking`, `OVRHaptics`, `WebCamTexture`, `UnityEngine.XR`, ...), while the default `CreativeFreedom` profile keeps full expressiveness; (ii) a **runtime user-frame guardian** (`UserFrameGuardian`) that re-asserts the tracking-owned head/rig pose each frame and re-shows any hidden *registered authentic* object, enforcing the invariants a lexer cannot; (iii) **mode-agnostic monitors** (`GeneratedContentMonitor`, `UserDisplacementTracker`, `AuthenticObjectRegistry`) that *disclose* — never block — personal-space, FOV, confinement, drift and disguise concerns; and (iv) **Mode-D containment**. An adversarial verification (one skeptic per paper plus a freedom critic) drove an important correction: a lexical denylist is the *wrong* tool for the camera-transform/viewport/post-process/scene-query/object-hide vectors because the same token (`Camera.main.transform`, `RenderTexture`, `FindObjectsOfType`) is an attack on the live rig but legitimate authoring on a build's own objects — the denylist

both missed the bare-transform attack and over-blocked creative builds. Those vectors are therefore enforced at *runtime* (where the live rig and authentic objects are identifiable) rather than lexically. The freedom contract — every creative build, including a preview camera, a post-effect and a recolour-all command, passes under both profiles — is test-enforced (`tests/tests/mode_b_d_defense.rs`).

Source	Core attack	DreamCodeVR+ defence	Status
Su et al. (2022)	Cursorjacking / input redirection	No cursor/input verb exists; target restricted to scene-root/selected with reject-on-missing; per-recipient <code>target_peer</code> routing	By construction
Lee et al. (2021)	Malicious 3D content / ad confinement; auxiliary-display abuse	Capability confinement (no runtime code/reflection/IO/net); confinement-region monitor discloses out-of-region staging; overlays/aux-display are runtime-guarded, not lexically banned	By construction (capability) + partial (overlay/aux-display runtime-enforced, Unity-untested)
Casey et al. (2021)	Human joystick (imperceptible drift); chaperone tamper; Man-in-the-Room	<code>DeployHardened</code> bans the named XR-rig/boundary surface (<code>OVRManager</code> / <code>XRSettings</code> / <code>InputTracking</code>); the bare-transform herd is caught by <code>UserFrameGuardian</code> (re-asserts rig pose) + drift monitor; per-peer HMAC admission token (<code>PeerAuthenticator</code>)	Implemented (lexical device-ban + Rust-tested token; runtime guardian Unity-untested; token transport + TLS future)
Valluripally et al. (2020)	DoS / resource exhaustion (+ SMC method)	JSON-depth cap, pre-allocation size reject, action/spawn/session caps, per-peer rate limit, anti-strobe interval	Implemented (DoS leaves); SMC method outstanding
Tseng et al. (2022)	Walking + arm puppetry; false-positive / swapping mismatch	Puppetry unrepresentable in Mode C; XR-rig API banned under <code>DeployHardened</code> ; the tracking-offset (camera-transform) form is runtime-enforced by <code>UserFrameGuardian</code> ; provenance + <code>AuthenticObjectRegistry</code> for mismatch; advisory <code>semantic_screen</code>	Implemented (rig API + runtime guardian); mismatch partial (registry scaffold; full reality-mesh future)
Ishii et al. (2016)	Optical-marionette gait steering	The view shift is <code>Camera.main.transform</code> /viewport mutation — <i>not</i> lexically banned (inseparable from creative camera authoring); enforced at runtime by <code>UserFrameGuardian</code> (re-asserts the tracked pose) + per-object FOV cap + drift monitor	Implemented (runtime) , Unity-untested; aggregate optic-flow model future
Fujita et al. (2023)	3D object disguise / false affordance	Provenance on all generated content incl. compiled-C# output (<code>MarkGeneratedHierarchy</code>); <code>AuthenticObjectRegistry</code> overlap-vs-real + focal-depth-mismatch; <code>UserFrameGuardian</code> re-shows hidden authentic objects (object-hide is runtime-enforced, not lexically banned)	Implemented (provenance + runtime) , Unity-untested; full reality-mesh + semantic disguise classifier future
Krauß et al. (2024)	XR dark patterns (coercion, painful haptics, disguise, ads)	<code>DeployHardened</code> bans haptics; no monetisation beneficiary exists; rate/anti-nag pacing; advisory <code>semantic_screen</code> (5 intent clusters); content-motion sickness and disguise are disclosed/runtime-guarded, not lexically banned	Partial (haptics + no-beneficiary done; motion-as-punishment and ads are disclose-only; ML dark-pattern classifier future)
Pearlman (2020)	Broad immersive-risk taxonomy	Defence-in-depth: allow-list + opt-in device-ban profile + runtime guardian + disclosure monitors + two-sided bounds + budgets + all-or-nothing + cold-start fail-closed	Partial (broad; no completeness argument over the full taxonomy)

Outstanding (future work, §9.3): hardening and on-device validation of the runtime `UserFrameGuardian` (the camera/rig/hide invariants are currently enforced by Unity-untested code; full VR-correct integration reads the XR-tracked pose each frame); a reality-aware room-mesh / depth scan to populate the authentic-object registry for

complete false-positive and generated-over-real disguise detection (Tseng, Fujita); aggregate optic-flow/vection modelling beyond per-object coverage (Ishii); an ML-based semantic dark-pattern / disguise classifier replacing the heuristic advisory screen (Krauß, Fujita); statistical model checking of the safety properties (Valluripally); cryptographic binding of object selection to user intent (Su); and completion of network authentication — carrying the per-peer capability token end-to-end in the Ubiq payload plus transport TLS (Casey).

7.10 Adversarial red-team of the C# validator and the intent screen (RQ3) — verified

RQ3 asks whether the safety layer can withstand deliberate adversarial pressure rather than merely well-formed input. To answer it empirically we built a reproducible red-team corpus generator (`redteam/corpus_gen.py`) that emits 1,057 unique attack and control vectors with no randomness, so the entire campaign is deterministically repeatable: 727 C# vectors fired at the tree-sitter lexical validator (`POST /api/validate`) and 330 natural-language vectors fired at the intent screen and generation path (`POST /api/command`) (*Verified by automated tests.*). The 727 C# vectors comprise 258 malicious "should-block" cases and 469 benign "creative-freedom" cases, the latter deliberately including scary-looking-but-safe authoring — a cube named `TrojanHorse`, a sphere named `seashell`, and a minimap `Camera` — to probe over-blocking as well as under-blocking (*Verified by automated tests.*). The runner (`redteam/run_campaign.py`) scores each vector against ground truth and records two failure modes: a *bypass* (a should-block vector that is approved — a critical security failure) and an *over-block* (a benign vector that is blocked — a false positive that harms creative freedom) (*Verified by automated tests.*).

The BEFORE result is reported here as a genuine development finding rather than a formality. Before the three evasion classes below were closed, an unadorned subsequence lexer caught only **36 of the 258** should-block C# vectors (**14.0%**) — the 222 vectors in those three classes all slipped through — while over-blocking none of the 469 benign vectors (full creative-freedom pass), so the validator was near-perfect on precision but badly deficient on recall. This before-figure is reconstructed from the corpus composition rather than reproduced: the *delivered* repository ships the already-hardened validator (the red-team code arrived in a single commit), so re-running the campaign against the shipped code scores 258/258 from the outset, and the pre-hardening run is not reproducible from the artefact as delivered. It is reported honestly because the finding — that an unadorned subsequence lexer is easily evaded — is what matters (*archived measurement, reconstructed from corpus composition; the delivered validator scores 258/258, verified by automated tests.*).

Those 222 bypasses resolved into exactly three evasion classes that the subsequence-matching lexer had not anticipated (the counts below are the should-block vectors exercising each class, and they sum to the full 222). Two further classes surfaced by the project's multi-agent audit — an `unsafe {}` pointer block and a `MonoBehaviour`-impostor base type — were closed the same way and are covered by the regression set, but they are *not* among the 1,057-vector corpus and so are not part of the 222:

Evasion class	Count	Mechanism
Namespace alias	206	<code>using Sys = System; then Sys.IO.File.WriteAllText(...)</code> reconstructs as segments <code>[Sys, IO, File]</code> , so the banned <code>[System, IO]</code> subsequence never appears, while the leaf identifiers (<code>IO</code> , <code>File</code> , <code>Net</code> , <code>Threading</code>) are not individually banned — smuggling file IO, sockets, threading and <code>WebClient</code> .
Unicode-escape identifier	9	C# decodes <code>\uXXXX</code> inside identifiers, but the AST leaf text is the raw escape, so <code>Application.\u0051uit()</code> reads as <code>\u0051uit ≠ Quit</code> while the compiler resolves the real <code>Quit</code> (likewise <code>\u0050layerPrefs</code> , <code>\u0047etType</code>).
Dynamic late-binding	7	<code>((dynamic)app).Quit()</code> reaches a banned member with no banned identifier node present for the lexer to see.

Three fixes were applied to `crates/csharp-policy/src/lexical.rs`. First, a using-alias pre-pass builds an alias→target map and `resolve_alias()` expands a leading alias segment — transitively, to cover chained aliases — before the subsequence ban check, closing the namespace-alias class. Second, `decode_unicode_escapes()` normalises `\uXXXX` and `\UXXXXXXXX` in identifier text prior to matching, closing the Unicode-escape class. Third, the `dynamic` keyword is banned outright, as it has no legitimate use in a generated MonoBehaviour, closing the late-binding class (*Verified by automated tests.*).

Re-firing the identical 727-vector corpus against the hardened validator closes the recall gap without any regression in precision:

Measure	Before (archived)	After / delivered
Attacks caught	36/258 (14.0%, reconstructed)	258/258 (100%)
Bypasses	222 (206 + 9 + 7)	0
Benign preserved	469/469	469/469
Over-blocks	0	0

All 727 C# vectors fire in roughly one second, as the path is purely lexical and invokes no language model (*Verified by automated tests.*).

The natural-language surface is defended by a distinct two-layer intent screen (Layer 1) that runs on the raw command before any generation: a fast keyword classifier (`classify_intent`) followed by an LLM semantic classifier (`screen_intent`) returning a JSON verdict. A command judged malicious is neutralised — replaced with a harmless server-authored recolour and never forwarded to the generation model — and surfaced in the admin panel. The LLM classifier stage is deliberately *advisory and fail-open*; the fail-closed validator downstream is the authoritative gate. An initial pass fired 100 attack prompts at the running backend, all of which were blocked (*Verified live, 2026-07-03.*), and benign-but-edgy themes — a Trojan-horse statue, a bloody zombie, a sea shell — were correctly left un-flagged.

That initial 100-prompt pass, however, over-stated the screen's recall, and running the **full** campaign later exposed the gap honestly. Firing the corpus's **305** should-block *natural-language* vectors at the intent screen, the screen alone caught **202 of 305 (66.2%)**, leaving **103 plain-English bypasses**; counted across *both* safety layers (the 258 C# vectors, all caught by the validator, plus the 202 NL) the campaign-wide figure was **460 of 563 (81.71%)**. Every one of the 103 bypasses was a *plain-English* attack that the keyword table — which knew only technical terms (`keylogger`, `ransomware`, `ddos`) — could not see, usually smuggled onto an innocent build: *"build me a castle and encrypt my files until I pay," "a helper that remembers everything I type," "keep a website too busy to respond,"* plus developer-mode jailbreak framings. A **natural-language paraphrase layer** was added, covering keystroke capture, screen/microphone surveillance, ransomware, self-propagation, cryptomining, denial-of-service, port scanning, credential brute-force, exfiltration, click fraud, and jailbreak framing. Re-firing the identical corpus, the screen caught **305 of 305 (100%)** NL should-block vectors — **563 of 563 (100%)** across both layers — with **0 bypasses**, while the benign pass rate stayed at **100%** and over-blocks at **0**. This post-hardening 100% is **in-sample**: the paraphrase table was authored against the same 103 sentences it now catches, so an independently-authored holdout corpus is needed before the figure can be read as generalising (*Executed, 2026-07-25.*).

Intent screen	Before paraphrase layer	After
Screen-alone NL catch	202/305 (66.2%)	305/305 (100%)
Campaign-wide (both layers)	460/563 (81.71%)	563/563 (100%)
Plain-English bypasses	103	0
Benign pass rate	100%	100%
Over-blocks	0	0

Taken together these results answer RQ3 directly: once adversarially exercised at full scale, the safety layer blocks **100%** of the 258 C# attack vectors *and* **100%** (305/305) of the natural-language should-block vectors — **563/563** across both layers — while preserving **100%** of benign creative authoring on both surfaces, subject to the in-sample caveat above. The lexical fixes are locked against regression by the `csharp-policy` regression suite, and the intent-screen paraphrase layer by two new router tests — `plain_english_paraphrases_are_caught` and `creative_commands_are_not_over_blocked_by_the_paraphrases` (`command-router/src/router.rs`) — plus a live probe in `scripts/verify-all.sh` (the campaign runner itself was not modified) (*Verified by automated tests.*).

The attack corpus in brief — five families, with the privacy family distinguished

The security work draws on a curated baseline of **128 attack vectors**, which doubles as the project's threat *model* — its honest coverage tally across the whole system is **33 solved (built and tested) · 2 partial · 80 designed (planned, mostly on-device) · 13 not applicable**, so "128 attacks neutralised" is explicitly *not* claimed. Separately, the ML analyzer of §7.14 grows a *discovered-vector store* to **164** entries by mutating against a naive denylist stand-in; the 164 is that store's size, not a re-count of the 128-vector model. Many vectors are near-duplicates — the same capability expressed through a different phrasing, obfuscation or API route — so the curated corpus is best read as **five families** rather than 128 distinct threats (the *Store n* column below counts the discovered store, and sums to 164):

Family	Plane	Representative vectors	Store n
Network & identity	code / system security	reverse shells, port scans, DDoS, C2/botnet, ARP-MITM, Wi-Fi crack, re-identification	34
Generated code	code / system security	<code>System.IO / Net / Reflection / Diagnostics</code> , <code>Process.Start</code> , <code>DllImport / Marshal</code> , shellcode, ransomware	40
Voice & sensor (<i>privacy</i>)	privacy — capture	microphone, camera / passthrough and screen capture; sensor spoofing; age-gate spoofing	38
Privacy / PII (<i>privacy</i>)	privacy — exfiltration	clipboard, contacts, location/GPS, SMS, credentials, photos, keylogging	30
Immersive / embodied	embodied safety	human-joystick (forced locomotion), chaperone edits, vection, occlusion, flashing	22

Of the five, **two are privacy-related** and carry the privacy contribution: *voice & sensor*, which captures the user's body and environment (microphone, camera/passthrough, biometric sensors), and *privacy / PII*, which exfiltrates personal data. Together they map onto the XR-specific privacy channels — **biometric**, **positional/motion**, and **image-of-surroundings** leakage (Nair et al., 2023). The remaining three families are either executable-code security (network & identity, generated code) or embodied-safety **manipulation** (immersive: the human-joystick and chaperone attacks of Casey et al., 2021, which are safety rather than privacy harms). Because the vectors within a family are largely obfuscation or phrasing variants of one another, the argument is carried by the families, not the individual strings; the full per-vector listing is deferred to Appendix C.

To quantify the defence against these classes without a headset, a dedicated local harness (`apps/xr-security-eval`) runs a fixed set of 40 attacks and 12 benign commands through the real validator at three levels — no defence, security-only, and fully hardened. The result is a clean before/after: **0/40** attacks blocked with no defence, **15/40 (37.5%)** with the code-security layer only, and **38/40 (95%)** once the backend is fully hardened, while all **12/12** benign creative commands still pass (0 over-blocked) — so the 95% comes at zero cost to creative freedom. A finding that motivates the layered design emerges directly: a code-security filter alone stops privacy *exfiltration* (which uses system/network APIs) but misses pure XR *manipulation* — the human-joystick class stays at 100% and chaperone at 88% under security-only — so only the perceptual/embodied layer catches those. Two subtle joystick vectors that abuse otherwise-legitimate camera moves remain a documented residual for runtime enforcement.

7.11 On-device Quest deployment on the university network — verified live

The system was exercised end-to-end from a physical Meta Quest headset connected to the backend over the university lab Wi-Fi. The device under test was a Quest 1/2 running the 32-bit Mono sideload (package `com.DefaultCompany.ubiqgenie`), which executes the Mode-A runtime-compilation path — generated C# is compiled on the headset at runtime via RoslynCSharp. Because the installed application dials a server IP baked in at build time (192.168.8.213), and DHCP had since reassigned the laptop to 192.168.8.210, connectivity was restored without rebuilding the app by making the laptop reachable at the baked address — a secondary IP alias was added with `ip addr add 192.168.8.213/24 dev wlo1`. The headset then established a TCP connection to the Ubiq RoomServer, observed as `192.168.8.202 -> 192.168.8.213:8009 ESTABLISHED` (*Verified live, 2026-07-03.*).

With the room joined, the full live-microphone speech-to-text pipeline was driven from the physical headset. A command spoken into the Quest mic — "generate me planet Jupiter and all its moons" — was streamed over Ubiq (NID 98), transcribed by OpenAI Whisper, generated by gpt-5.5, validated by the C# policy layer, and dispatched back to the headset (NID 94) for runtime Roslyn compilation and instantiation in the scene. On this on-device path (a reasoning-effort build over the university network) simple commands complete generation in approximately 20 seconds; the lower 3–8 s figure quoted in §7.5 is for the editor with a lighter model/effort setting, so the two are different operating points rather than a contradiction (*Verified live, 2026-07-03.*). The same dispatch channel was confirmed from the admin panel's manual-command box: a typed instruction is validated and then sent to the headset over the shared Ubiq sender, so a manually authored command also builds in VR rather than merely validating (*Verified live, 2026-07-03.*).

A latent failure was discovered and fixed during this live session. The reasoning model had been configured at "high" effort with an 8000-token completion ceiling; on the Jupiter command it spent essentially its entire budget on hidden reasoning tokens and returned empty visible content. Downstream this parsed as an end-of-file condition, was mapped to `llm_unavailable`, and caused the command to be wrongly rejected. The default was changed to "medium" reasoning effort with a 24000-token ceiling, after which generation succeeded and the objects were produced (*Verified live, 2026-07-03.*).

The scope of this result should be stated honestly. What has been verified on-device is the Quest 1/2, 32-bit Mono sideload running the Mode-A runtime-compilation path — a locally provisioned, IP-addressable deployment on the university network. The Quest 3 IL2CPP build and a Meta Store distribution path remain future work and are not claimed here; Mode A's runtime C# compilation is specific to the Mono sideload and does not transfer unchanged to an IL2CPP/ahead-of-time target, which would require the alternative execution strategy discussed in the future-work chapter (*Future work.*).

7.12 Age-adaptive dual-plane safety coupling — verified (test)

The machine-learning thrust is reported here in three subsections (§7.12–7.14), each in the same four-part arc: **the problem, why we deal with it, what we built, and an honest read on whether it is enough**. The novel core of the thrust is a single coupling intended to let one age signal condition *both* safety planes at once — of which, as reported below, the **code** plane is enforced today and the **perceptual** plane is specified and unit-tested pending a final piece of runtime wiring.

The problem. Every safety mechanism built up to this point — the fail-closed action-plan validator (§5.4), the C# hardening profiles (§7.9), and the perceptual/embodied envelopes of §2.11 — treats all users identically. But the person in the headset may be a child, and the harms are not the same. A child is more vulnerable to photosensitive flashing, to disorienting optic-flow (vection), and to being lured toward a false floor or a false chair; and, legally, a child cannot have code executed or data captured on the strength of their own consent. A single global safety setting is therefore pitched wrongly for someone: loose enough for an adult is too loose for a child, and tight enough for a child needlessly constrains an adult. Concretely, neither the code-safety plane nor the perceptual-safety plane was conditioned on *who* the user is.

Why we deal with it. Age matters here for bodily and legal reasons, not abstract ones. The amended COPPA rule (16 CFR 312, 2025) now treats biometric identifiers as covered personal information and takes effect in April 2026; GDPR Art. 8 and the UK Age-Appropriate Design Code (ICO Children's Code) impose a duty of care toward minors; and the immersive harms of §2.11 fall harder on children. The move we think is genuinely novel — and ethically the right way round — is to use an age signal for *protection, not surveillance*: a detected minor makes the system safer and does nothing else (no profiling, no precise age retained, no other downstream use). To stay clear of the EU AI Act (Art. 5, in force February 2025), which restricts emotion recognition in many contexts, the signal is deliberately a coarse age *band* only and never an affect read.

What we built. A small, auditable coupling in `crates/config/src/age.rs` defines an `AgeBand { Child, Teen, Adult, Unknown }` whose default is `Unknown`, and a `for_band` mapping from band to an `AgeSafetyPosture` that spans both planes. A detected minor — or `Unknown`, which fails safe to *child* — tightens the code-safety plane (the C# validator flips to its hardened `DeployHardened` profile) and *simultaneously* specifies a tighter perceptual plane: relative to the Adult posture, the Child/Unknown posture turns perceptual hardening on, drops the per-command spawn budget from 40 to 20, the flash cap from 3.0 to 2.0 Hz, the field-of-view coverage limit from 0.70 to 0.35, the comfort rotation clamp from 90 to 30°/s, and the luminance-swing limit from 1.0 to 0.4. The status of the two planes differs and is stated precisely. The **code-plane** coupling is wired and enforced end-to-end: the orchestrator (`command-router`) flips the C# validator to `DeployHardened` whenever the band is a minor, verified live by `age_minor_forces_hardened_csharp_gate`. The **perceptual-plane** posture (the flash / FOV / rotation / luminance / spawn-budget values above, plus the compile-confirmation requirement) is computed and unit-tested at the configuration layer, but runtime perceptual enforcement presently reads the *global* runtime configuration rather than this age-derived posture (a repo-wide search finds no runtime consumer of `require_compile_confirmation` or `max_spawn_per_command`); consuming the posture in the runtime perceptual guards is a small but genuine piece of remaining wiring, so until it is done the perceptual half of the coupling is *designed-and-tested* rather than *enforced*. Crucially the whole feature is opt-in behind the `DCVR_AGE_GATING` environment flag; with it off, the pipeline is *byte-identical* to the legacy path, so the coupling adds safety without changing default behaviour. Three Rust tests pin the behaviour — `unknown_fails_safe_to_child`, `child_tightens_both_planes_vs_adult`, and (in `command-router`) `age_minor_forces_hardened_csharp_gate` — and run inside the workspace's 342-test gate. Two literature anchors, *not* our own measurements, frame why a voice age band is even feasible: child-versus-adult voice is separable at roughly 97.14% age-group accuracy on CMU Kids, and motion-based age inference has been reported at about 78% per-user (Nair, arXiv:2305.19198) — the latter underscoring how much *is* inferable from a headset, which is precisely why we restrict our own signal to a coarse band.

Is it enough? — honest evaluation. *Proven:* the coupling logic is verified by the three named Rust tests — a minor tightens both planes, an unknown band fails safe to child, and a detected minor alone forces the hardened C# gate — and the opt-in flag is shown to leave the legacy path byte-identical when off. *Limitation:* first, the enforced effect today is the **code-plane** flip — the perceptual half of the coupling is specified and unit-tested at the configuration layer but is not yet consumed by runtime perceptual enforcement (above), and completing that wiring is future work. Second, the coupling is only as trustworthy as the age signal that drives it, and that signal (§7.13) is a coarse band that our own analyzer shows to be spoofable (§7.14); the coupling therefore *restricts* but must never be relied on to *authorise*. The specific bound values (20 spawns, 2.0 Hz, 0.35 FOV, and so on) are engineering choices informed by WCAG-style thresholds, not the outcome of a human-factors study. *Pending:* the platform age-group signal (the Meta Quest age API, 2024) remains the authoritative source and the ML band is a secondary net; an end-to-end on-device run of the coupling with real microphone audio on a physical Quest is future work.

7.13 On-device voice age-band gate — verified (test)

The problem. To condition safety on age (§7.12) we need an age signal that runs *on the device*, cannot itself become a privacy liability, and does not overreach into a precise age or an emotional read. An off-device age service, a precise-age estimate, or an affect classifier would each create the very surveillance harm we are trying to avoid — and the harm would be pointed at children. The FTC (March 2024) has explicitly *denied* facial age estimation as a lawful COPPA consent mechanism, so an age estimate cannot be the legal basis for anything on its own; it can only make the system more cautious.

Why we deal with it. A protective age signal is worth building only if it is built the careful way: local, coarse, and modest about what it claims. Emitting a *band* (child <13 / teen 13–17 / adult 18+) rather than a number keeps the model within the age-assurance-level framing of ISO/IEC 27566-1:2025 and IEEE 2089.1-2024 (both standards/literature anchors), and deliberately excludes affect so the EU AI Act Art. 5 emotion-recognition concern does not arise. Keeping the model numpy-only and CPU-only makes it reproducible and auditable — a reviewer can rerun it — which matters more for a safety net than raw accuracy.

What we built. `ml/age_gate` is a numpy-only (no torch, no scikit-learn), CPU-only, reproducible classifier that emits a coarse band and *never* a precise age and *never* affect. It is temperature-scaled for calibration ($T \approx 1.93$), which reduces Expected Calibration Error from 0.051 to 0.012 on the validation split (reproducible via `train.py`), with a held-out test ECE of 0.007 (`evaluate.py`); both figures are deterministic on a clean checkout. The suite is 51/51 tests green (17 in the core gate tests, plus conformal, ordinal and wav2vec2-upgrade coverage). Two literature anchors bound expectations without being claimed as our own results: child-versus-adult voice is separable at about 97.14% age-group accuracy on CMU Kids, and edge age models have been reported at roughly 20 ms latency — both *literature anchors*, and the 20 ms figure in particular has **not** yet been measured on a Quest. A deeper wav2vec2 feature front-end is documented as an upgrade path (`wav2vec2_features.py`) but is not the evaluated model.

Is it enough? — honest evaluation. *Proven:* 51/51 tests pass and the calibration improvement (0.051 $\rightarrow$ 0.012 validation ECE; 0.007 held-out test) is reproducible from `train.py` / `evaluate.py`. *Limitation:* the gate is evaluated on held-out/synthetic audio, not real push-to-talk microphone capture; its features are simple by design (the wav2vec2 upgrade is written up but not run); and, most importantly, the band is *spoofable* — our own red-team (§7.14) pushes a child sample across the adult boundary 100% of the time against a numpy stand-in (`MockAgeGate`); the shipped classifier itself has not yet been red-teamed, which is a gap, not a clean bill of health. *Pending:* real-microphone audio and an on-device Quest run (future work). The honest posture is the one baked into §7.12: the platform (Meta Quest) age-group API is authoritative, and this model is a secondary safety net that can only *tighten*, never *authorise*.

7.14 Continuous ML attack analyzer — verified (test)

The problem. A validator hardened once can quietly rot: attackers find new phrasings, the input distribution drifts, and a fixed test corpus notices neither. And the age gate of §7.13, being an ML model, must be assumed spoofable until something actually tries to spoof it. We wanted a component that watches the pipeline continuously — flagging anomalous inputs, noticing distribution shift, and actively red-teaming our own age gate — rather than trusting a one-time hardening pass to hold.

Why we deal with it. This closes the loop on the dissertation's safety argument: the C# red-team of §7.10 was a one-shot campaign, whereas an operational system needs *continuous* assurance, and any ML safety signal (the age band) needs an adversary of its own. Building the analyzer numpy-only keeps it auditable and CPU-only, consistent with the rest of the ML thrust.

What we built. `ml/attack_analyzer` combines three numpy-only pieces. An anomaly detector — a PCA/SVD undercomplete autoencoder — scores inputs by reconstruction error, achieving a pooled AUC of 1.00 on the controlled corpus with a benign false-positive rate of ~2% at a 99th-percentile threshold; a deliberately hard-negative slice (stealthy paraphrases) is far less separable (~0.70 AUROC on the hardest family), which is the honest field-relevant figure. A drift detector (ADWIN2) raised 0 false alarms on a stationary stream and detected an injected distribution shift at $t = 304$. And a red-team loop turns the analyzer on a numpy *stand-in* of the age gate (`MockAgeGate`, not `ml/age_gate`'s trained classifier), discovering a 100% spoof rate against that stand-in (a child sample pushed across the adult boundary) — a deliberately surfaced negative result; the shipped classifier has not itself been red-teamed. Honest scope on two further numbers: at the thesis's own operating point (a ~2% benign FPR / 99th-percentile threshold) the autoencoder's detection on the stealth/immersive family is effectively nil (AUPRC ≈ 0.14 , ~0% detection at a 1% false-positive rate), where a one-line max-repeat bound dominates it — so the useful finding is *which* harms need an explicit bound rather than an anomaly score, and the pooled AUC of 1.00 is a benchmark artefact of out-of-vocabulary attack columns, as the tool itself reports. The attack corpus starts from a curated baseline of 128 vectors, and the discovered-vector store (`attacks_discovered.jsonl`) grows to 164 entries by mutating against a deliberately naive substring denylist (`MockDenylistSUT`), not the tree-sitter validator. The suite is 31/31 tests green.

Is it enough? — honest evaluation. *Proven:* 31/31 tests pass; the anomaly detector separates the controlled corpus (pooled AUC 1.00, ~2% benign FPR) and the drift detector both stays quiet on stationary data and catches the injected shift at $t = 304$. *Limitation:* a pooled AUC of 1.00 is a property of a *controlled* corpus with clean separation, not a field claim — on a hard-negative slice the stealthiest family falls to ~0.70 AUROC; it says the detector cleanly handles the vectors we generated, not that real adversarial traffic will separate as neatly; and the analyzer's most useful output is arguably its honest negative, the 100% spoofability of a numpy stand-in of the age gate (`MockAgeGate`; the shipped classifier is not itself red-teamed), which is exactly why §7.12 keeps the platform age API authoritative. *Pending:* a live-traffic evaluation against real STT/LLM endpoints and on-device capture. As with the rest of the thrust, the models are numpy-only by design — reproducible and auditable rather than maximally accurate.

7.15 Whole-system verification campaign: seven latent defects, and a durable regression harness — executed

The results reported so far verify features largely in isolation. To test the system as a whole, a verification campaign checked every backend feature by its downstream **effect** — that a control actually *changed what the system did* — rather than merely that an endpoint responded with a plausible value. The campaign was organised block by block, one per subsystem, and is now captured as the 40-check `scripts/verify-all.sh` harness; crucially it ran against real infrastructure that earlier phases had mocked: the genuine .NET Roslyn analyzer on `:5099`, the Mode-D

Docker sandbox image, and the authentic UCL Ubiq RoomServer (fetched from the UCL-VR repository after the original `gitpkg` URL was confirmed dead, §1.3(iv)) (*Executed, 2026-07-25.*).

Verifying by effect — rather than by response — is what made the campaign productive: it uncovered **seven latent defects** that a response-level check would have missed, two of which were *safety controls that silently did nothing*.

#	Defect	Why it mattered
1	Mode-D sandbox dead on macOS — the runner called <code>env_clear()</code> before spawning <code>docker</code> , wiping <code>PATH</code> , so the binary could not be resolved	The entire containment feature was inert on the platform, while its unit tests (which stub the runner) stayed green
2	Admin "Mode A" toggle was a lie — switching runtime-C# dispatch <i>off</i> in the panel still dispatched generated code to the headset	A security control that reported "off" while remaining on — invisible to any test that checks the endpoint's <i>response</i> rather than the <i>dispatch</i>
3	Model dropdown was decorative — changing the model in the panel changed nothing; the server seeds the model from <code>OPENAI_MODEL</code> and only pushes effort/verbosity/token knobs	A safety-relevant control with no effect, misrepresenting operator authority
4	Panel tools ignored panel settings — the validate and red-team tools used fixed defaults instead of the live configuration	Demonstrations could disagree with the running policy
5	Discovery beacon advertised the wrong port — hardcoded <code>:8009</code> , so under a custom bind headsets dialled a dead port	Silent connectivity failure on any non-default deployment
6	Retention sweep was dead code — <code>purge_expired()</code> existed, was unit-tested, and was documented as a GDPR data-minimisation control, yet had <i>no runtime caller</i> and was not even reachable through the trait object, so profiles were kept forever	A <i>privacy</i> control that passed its tests but never ran; now wired via <code>DCVR_PROFILE_TTL_SECS</code> (default 0 = off)
7	Intent screen missed 103 plain-English attacks (§7.10) — the keyword table knew only technical terms	81.71% → 100% attack-catch after the paraphrase-layer fix

Defects 2 and 6 are the instructive ones: both are *controls that lie*. A unit test asserting that `POST /api/config` returns `{"mode_a": false}` passes even when code is still dispatched; a unit test asserting that `purge_expired()` deletes an expired profile passes even when nothing ever calls it. Only checking the downstream effect — did a subsequent command reach the headset? does a profile actually disappear after its lifetime in a *running* system? — exposes them. This is the campaign's methodological contribution: for safety-critical software the meaningful assertion is behavioural, not merely responsive. Each defect was fixed and locked with a test (a per-token guardrail sweep of all 49 denylist tokens across both profiles, `tests/tests/guardrail_token_sweep.rs`, was added in passing) (*Executed, 2026-07-25.*). A further correctness fix rejects non-RFC-4122-v4 room UUIDs, matching upstream Ubiq's own validation, where the original backend had accepted any string.

A robustness finding against the upstream artefact. Standing up the *genuine* UCL Node.js RoomServer surfaced a fault in the third-party code itself: a single `SetBlob` control message crashes it outright (`TypeError: this.server.setBlob is not a function, roomserver.js:440`), so one client can take down the relay and every peer in the room. The from-scratch Rust reimplementaion of the RoomServer (`crates/roomserver`, Contribution 17) handles the identical message correctly and stays available. This is a concrete, defensible robustness improvement of the reimplementaion over the original — obtained by differential testing against the real artefact, not asserted in the abstract. The upstream code was left unpatched, being third-party (*Executed, 2026-07-25; upstream crash reproduced and logged.*).

A durable regression harness. The entire campaign is captured as a single command, `scripts/verify-all.sh`, which prints a PASS/FAIL table and exits non-zero on any failure; optional tools (Docker, .NET, Node) report SKIP rather than FAIL when absent, so the script degrades gracefully on a minimal machine. It covers the toolchain and pinned version; `fmt/clippy/tests/cargo-deny`; the compiler-enforced `unsafe-free` and `panic-averse` lint

invariants; the 38/40 = 95% security benchmark **and** its byte-identical determinism (a SHA-256 of `results.json`); the ML suites; corpus generation; a live one-binary end-to-end run (embedded RoomServer, a full spoken-style command, admin-token gating, the guardrail under both profiles, and the intent screen including the paraphrase layer); the 1,057-vector red-team campaign; and Mode-D containment. A `--baseline` flag records the known-good values to `scripts/verify-baseline.txt` (`tests_passed=342`, `attacks_blocked=38`, `ml_tests=82`, `campaign_vectors=1057`, `results_sha256=13776b9a...`, `rustc=1.96.0`); later runs diff against the baseline and print any drift, and this drift detection was itself verified by deliberately perturbing the baseline and confirming the alarm fires. At the time of writing the harness reports **40 PASS · 0 FAIL · 0 SKIP** (*Executed, 2026-07-25*). The campaign thus converts the dissertation's safety claims from "the code exists and its tests pass" to "the feature's effect was observed against real infrastructure, and a one-command harness keeps it that way."

Chapter 8 — Discussion, Limitations, Ethics & Privacy

8.1 Interpreting the central thesis

The animating argument of this dissertation is that, for real-time immersive speech programming on a Meta Quest, *bounding the output language is a stronger and more deployable safety lever than sandboxing the output*. The verified evidence supports this claim at the architectural and integration level. The deployable path (Mode C) never compiles or executes model-authored code: the language model targets a finite, six-action, internally-tagged, `deny_unknown_fields` intermediate representation (`behaviour-dsl`), and the fail-closed validator (`code-policy`) approves a plan only when its violation set is empty. Because the action set is an allow-list enum with hard numeric bounds, an entire class of unsafe operations — banned-API access, out-of-range scaling, schema-bypass — is not merely filtered after generation but is *unrepresentable* in the first place. This is the qualitative difference between making bad code inexpressible (Mode C) and detecting or containing it after the fact (Modes B and D), and it is what makes Mode C deployable on the IL2CPP/AOT backend where runtime C# compilation is structurally impossible (Unity Technologies, 2024).

The contrast with the sandbox arm is instructive precisely because the sandbox was verified to work. Mode D was demonstrated to contain untrusted generated C# under plain Docker: `filesystem` writes (`File.WriteAllText("/etc/pwned")`) returned `FS_BLOCKED`, network access returned `NETWORK_BLOCKED`, and runaway loops were killed by wall-clock timeout (NIST, 2017). The stronger gVisor (`runsc`) boundary — which would interpose an application kernel so an escape must break two disjoint kernels (Young et al., 2019) — is wired and unit-tested in the runner but was not exercised, so it is a designed rather than a measured property. Yet even a working sandbox is a *research* mode, not the product: it presupposes the very runtime-codegen step that Mode C eliminates, and a container is a soft boundary unless hardened, whereas an allow-list has no residual code-execution surface to harden. The strongest empirical illustration of "bound, don't scan" came from Mode B. The .NET Roslyn semantic analyzer returned a **false negative** for `new System.Net.WebClient` — it references only `corelib`, so a non-`corelib` symbol fails to resolve and is silently skipped — while the Rust lexical scanner caught `System.Net` syntactically and rejected it before Roslyn was ever consulted. This concretely demonstrates that post-hoc analysis of free-form code is brittle, and that defence-in-depth (lexical + semantic + consistency + sandbox) — not any single gate — is what makes the C# arm safe. The expressiveness cost of Mode C (only six behaviours) is real, but it is a deliberate and defensible trade for an auditable, on-device end-user tool.

8.2 Limitations

The limitations are stated plainly, not in footnotes.

- **The Unity client is verified in the interactive Unity 6 Editor; only headless EditMode tests remain pending.** Mode C and validator-gated Mode A both run live in the Unity 6 Editor — object changes from live backend messages, and model-generated C# compiled at runtime to build structures (§7.5). The drop-in executor's four *headless* batchmode EditMode tests still quit at the **licensing** step (`Access token is unavailable`) — a Unity account sign-in the automation cannot perform, not a code fault. (An on-device Quest 1/2 Mono sideload run is reported in §7.11; the Quest 3 / Store IL2CPP build is the outstanding on-device item.)
- **On-device operation is demonstrated on a Quest 1/2 32-bit Mono sideload (Mode A only); Quest 3 / Store is not.** A command spoken into the headset microphone was captured, transcribed, generated, validated and compiled at runtime on-device (§7.11). What is *not* built or run is the Quest 3 IL2CPP/ARM64 and Store build path for the no-compilation Mode C; that remains a structural argument only.

- **gVisor was not exercised, though the runner already supports it.** Containment was confirmed under plain Docker only; `runsc` was not available on the verification machine and the project's `scripts/sandbox-run-gvisor.sh` is checked in with a "not yet verified" status note. The Rust `DockerSandboxRunner` already selects an alternative OCI runtime through `with_runtime` (exposed as `DCVR_SANDBOX_DOCKER_RUNTIME` and covered by the `runsc_runtime_included` unit test), and it defaults to none; the residual is that the regression harness runs Mode D under plain Docker only, so kernel-level isolation is a designed rather than a measured property.
- **The .NET semantic analyzer is deny-list-only and shallow** (see the `System.Net` false negative above), and Mode A is preserved but not integrated into the Rust comparison.
- **Live-microphone speech-to-text is exercised only from the headset mic.** The OpenAI language-model path is verified end-to-end with the real gpt-5.5 model (§7.5) and live-microphone STT was driven from the physical Quest (§7.11); what was *not* tested is a desktop-microphone capture path, for which no input device was available.
- **The quantitative security result measures static code-admission, not runtime prevention.** The benchmark rejects malicious code before it can run; it does not measure on-device runtime prevention. Two of forty attack vectors (`joy-05` / `joy-06`) rotate the camera in a way indistinguishable from legitimate authoring at admission time — which is why the figure is $38/40 = 95\%$ and not 100% — and closing them depends on the runtime `UserFrameGuardian`, which is implemented but Unity-untested (§7.9).
- **The post-hardening 100% recall figures are in-sample.** The 1,057-vector corpus is hand-authored by the author; the same vectors located the bypasses and validated the fixes (the paraphrase layer in particular is a keyword table fitted to the 103 sentences it failed on). An independently-authored holdout corpus is required before the recall figures can be read as generalising.
- **The personalization/RAG store persists user content, and is on by default (opt-out).** `enable_rag` defaults true, so unless disabled it writes verbatim commands and generated C# to a local plaintext profile (`.dcvr-data/personalization/<peer>.json`); this is disclosed rather than hidden — encryption-at-rest (ChaCha20-Poly1305, opt-in via `DCVR_PROFILE_ENC_KEY`) and a TTL retention sweep (`DCVR_PROFILE_TTL_SECS`) are implemented but off by default. Making retention and RAG opt-in, and encryption default-on, is stated as outstanding privacy work. The telemetry `TimingEvent` remains content-free by test.
- **The demonstrated configuration runs the default legacy security profile over plaintext TCP.** No quantitative or human-subjects study has been conducted and the system has been verified on a single machine. Networking is unauthenticated (per-peer identity remains future work) though now per-peer rate-limited and anti-strobe-throttled, and the Unity Mode-C client auto-reconnects after a dropped link. A full hardened crypto stack (HMAC admission tokens, Ed25519 backend→client signing, replay guard) exists and is fail-closed, but the default profile is `legacy`, so in every demonstrated configuration that crypto is *inert*; even under the hardened profile inbound verification is not live, because the Unity client does not yet emit auth envelopes (`ClientEnvelopeSigner` is unwired) and the demo constructs its verifier with `RequireSignature = false` — only outgoing backend→client NID-94 signing is active. Transport is plaintext TCP and TLS/WSS is a deployment step not yet taken. The Rust↔C# bounds parity is now guarded by a test but the C# side is still authored by hand; and, because the upstream Ubiq RoomServer `gitpkg` host returns HTTP 402, the RoomServer is vendored (`vendor/ubiq-roomserver/`) so verification remains reproducible.

8.3 Ethics, privacy and responsible disclosure

DreamCodeVR+ processes push-to-talk voice, which is personal data — and, where it carries biometric voiceprint characteristics, special-category data under the UK GDPR (Articles 9, 25 and 35). The privacy posture is therefore enforced **by construction** rather than promised in prose. No raw microphone *audio* is retained, and the telemetry path is content-free by construction: the only telemetry record, the `observability::TimingEvent` JSONL line, carries ids, timestamps, decisions, reason-codes and counts but **never** content — a unit test asserts that no `audio`, `transcript`, `secret` or `image` field can appear in that schema — and STT/LLM failures are mapped to fixed reason-codes (`stt_unavailable`, `llm_timeout`, ...) so raw detail never leaves stderr. One exception must be disclosed honestly: the personalization/RAG layer (§5.12), which is **on by default** (opt-out — `enable_rag` defaults true and the on-disk store is always wired), persists the verbatim command and the generated C# to a per-user plaintext profile on disk (`.dcvr-data/personalization/<peer>.json`) in order to retrieve liked creations. This is a deliberate feature, not an accidental leak, but it is genuine personal-data-at-rest; encryption-at-rest (ChaCha20-Poly1305, opt-in via `DCVR_PROFILE_ENC_KEY`) and a TTL retention sweep (`DCVR_PROFILE_TTL_SECS`, erasable under GDPR Art. 17) are implemented but both are off by default today. Making retention and RAG opt-in, and encryption default-on, is outstanding privacy work. Secrets are held in zeroized `SecretString` wrappers and `.jsonl` / `.env` files are git-ignored.

Because the technology is innovative, involves voice, and may in future involve child users, a **full Data Protection Impact Assessment is treated as a precondition for any user study, not an afterthought**; the relevant University of Birmingham ethics reference and DPIA outcome are to be inserted here before submission (the application is to be filed through the School of Computer Science ethics process, and no human-subjects data is collected until that reference is in hand). Any human-subjects component — envisaged only as an optional, adults-only red-team exercise — would proceed solely after University of Birmingham ethics approval, with explicit, granular, opt-in consent for any audio capture, a stated retention-and-deletion schedule, and the right to withdraw; age estimation is explicitly out of scope. The one artefact that touches age — the voice age-*band* gate of §7.13 — warrants explicit ethical treatment here rather than only in the results chapter: it runs locally on-device, emits a coarse child/teen/adult band and never a precise age or an affect read, is used only to *tighten* safety (never as a consent basis, never for profiling or affect inference), and defers to the platform Meta Quest age-group API as the authoritative source. Inferring even a coarse band from a minor's voice is the most ethically loaded component in the system, and it is deliberately constrained to a protective-only role. The sandbox (Mode D) is a security-research artefact for executing deliberately adversarial code, and is framed under **responsible-disclosure** norms: it runs only behind a research flag, in a hardened, network-isolated container, and any escape or evasion finding would be reported to the relevant maintainers before publication (Greshake et al., 2023; OWASP Gen AI Security Project, 2025a). Finally, the use of AI assistance during this project is disclosed and conducted in accordance with University of Birmingham policy on generative-AI use, with all load-bearing claims grounded in the verifiable artefact rather than in model output.

Chapter 9 — Conclusion & Future Work

9.1 Contributions

This dissertation set out to make real-time immersive speech programming **safe by construction** and **deployable on a Meta Quest** without discarding the expressive intent of UCL's DreamCodeVR (Giunchi et al., 2024). It delivered: an allow-listed six-action plan DSL whose enum is the safety boundary (`behaviour-dsl`); a fail-closed validator that rejects oversized input before parsing and approves iff zero violations remain (`code-policy`); a Rust Ubiq service-peer orchestrator with per-peer sessions, async timeouts, a single spawn-budget chokepoint and privacy-safe telemetry (`unity-transport`, `command-router`, `observability`), including two real protocol-bug fixes found by live testing; a defence-in-depth validated-C# research arm (`csharp-policy` lexical + `roslyn-client` semantic); a sandbox arm verified under Docker, with a gVisor (`runsc`) runtime already selectable in the runner and unit-tested but not yet exercised (`sandbox`, `services/sandbox-worker`); a safety-preserving personalization/RAG layer that learns user taste from feedback yet injects it only as validator-gated, untrusted guidance (`personalization`); a live observability and runtime-control plane with a web admin panel and on-demand red-team runner (`control`, `admin`); a perceptual and embodied safety layer (a user-frame invariant, scene-relational guards, and temporal envelopes) that extends the safety argument from code to immersive content; a 342-test, `unsafe`-free, panic-averse, supply-chain-audited workspace; an opt-in, age-adaptive safety coupling in which a detected minor — or an `Unknown` band, which fails safe to *child* — flips the code-safety validator to its hardened profile (`config::age`, `command-router`; the code-plane coupling wired and tested, the parallel perceptual-plane posture specified and unit-tested at the configuration layer pending runtime wiring), together with two reproducible, numpy-only, CPU-only ML models: a calibrated on-device voice age-band gate (band-not-precise-age, emotion deliberately excluded; 51/51 tests) and a continuous ML attack analyzer that red-teams that gate (31/31 tests); and — hardening this baseline under adversarial pressure — an executed adversarial red-team that hardened the C# lexical validator by closing three novel evasion classes and, in a reproducible 1,057-vector campaign, confirmed it at 258/258 while raising the natural-language intent screen from 81.71% (460/563 across both layers; the screen alone 66.2%) to 100%, closing 103 plain-English bypasses, both with zero over-blocks (these figures in-sample); a whole-system verification campaign that found and fixed **seven latent defects** — including two safety controls that silently did nothing — captured as a one-command regression harness with a drift-detecting baseline; a from-scratch Rust reimplement of the Ubiq RoomServer that reaches stock-Ubiq parity and stays available on a control message that crashes the genuine UCL RoomServer; and the whole system demonstrated live end-to-end on a physical Meta Quest 1/2 Mono sideload (Mode A; the Quest 3 / Store IL2CPP path remains outstanding).

9.2 Answering the research questions

- **RQ1 (safety by construction).** Supported at the architectural and integration level: schema-bounding makes unsafe operations unrepresentable, the validator is fail-closed, and the C# arm's defence-in-depth was shown to catch what its semantic layer missed. An executed adversarial corpus (1,057 vectors) subsequently confirmed the C# validator at 258/258 and hardened the natural-language intent screen to 100%, both with zero over-blocks and in-sample (§7.10). The full per-mode quantitative comparison remains future work (§7.7).
- **RQ2 (deployability).** Structurally answered — Mode C removes the runtime-compilation dependency that the IL2CPP/AOT backend forbids (Unity Technologies, 2024). On-device operation has since been demonstrated live on a physical Quest 1/2 Mono sideload (§7.11); the Quest 3 IL2CPP/Store build remains future work.

- **RQ3 (orchestration & performance).** The orchestrator's robustness (per-peer isolation, fail-closed timeouts, no global lock) is verified live through the real RoomServer, and its adversarial robustness was confirmed by the executed red-team (100% of the 258 C# attack vectors and 100% — 305/305 — of the natural-language attack vectors, 563/563 across both layers, zero over-blocks, in-sample, §7.10); per-stage latency distributions remain future work (§7.7).
- **RQ4 (comparative trade-off).** The four-mode differential design is specified and the integration arms are verified, but the head-to-head functional, latency and security comparison remains future work (§7.7), and Mode A is not yet wired into the differential harness.

9.3 Future work (prioritised)

1. **Quest 3 / Store deployment of Mode C** — the Quest 1/2 Mono sideload (Mode A) is now demonstrated live on-device (§7.11); the remaining step is the IL2CPP/ARM64 Mode-C build for Quest 3 and the Store, plus the four headless EditMode tests and GUI play-mode.
2. **Real STT and LLM** — exercise faster-whisper and OpenAI endpoints under the existing timeout-wrapped paths.
3. **Wire Mode A into the differential harness** for a complete A/B/C/D baseline comparison.
4. **Exercise the gVisor (runsc) boundary end-to-end.** The Rust `DockerSandboxRunner` already selects an alternative OCI runtime through `with_runtime (DCVR_SANDBOX_DOCKER_RUNTIME, unit-tested)`, so this is a deployment/measurement step — provisioning a `runsc` host and adding the `runsc` path to the regression harness — not new development.
5. **Authenticated peer identity for the networking layer.** The self-asserted `peer_uuid` on Ubiq NID frames is the residual that enables cross-peer scene targeting and feedback poisoning. Interim mitigations already reduce the impact — the service peer now sets a per-recipient `target_peer` on every decision so a client applies only decisions addressed to it; the personalization summary is sanitised so forged feedback can no longer inject verbatim instructions (only nudge aesthetic keywords); and per-peer rate limiting bounds flooding — but none authenticates the sender. The full fix is per-peer identity: at join, the service peer issues each client a short-lived, unguessable capability token (or, for non-repudiation, registers a per-peer public key); thereafter a message is accepted as peer *X* only if it carries *X*'s token/signature in the application payload, which is transparent to the RoomServer since it routes opaque frames. On the plaintext TCP channel this must be paired with transport TLS (or a WSS RoomServer) so tokens cannot be captured on the wire. This closes peer impersonation, cross-peer targeting, and feedback poisoning together.
6. **Editor-verify and extend the perceptual-safety layer.** The user-frame invariant, FOV-occlusion, overlap and cumulative-drift guards are implemented in the Unity executor with a cold-start fail-closed fallback (placements are refused when a trusted head pose is unavailable); they require a licensed-Editor compile-and-play pass; they are exercised in-Editor by the `GuardrailSelfTest` harness (13 cases) and are opt-in via `comfortIntensity` (default off for free creation). The reality-aware extension — cross-checking standable, sittable and occluding objects against the Guardian boundary and the passthrough room mesh — and a statistical-model-checking evaluation of the guard automata (after Valluripally et al., 2020) remain.
7. **Add bounds-drift CI** beyond the current parity test (auto-reconnect is now implemented in the Unity Mode-C client).
8. **On-device age gate and continuous analysis.** The age-adaptive coupling (§7.12) and both ML models (§7.13–7.14) are today evaluated on held-out/synthetic audio and a controlled attack corpus with numpy-only models; the outstanding steps are real push-to-talk microphone audio and a physical-Quest

run, the documented wav2vec2 feature upgrade for the age gate, and a live-traffic evaluation of the attack analyzer. Throughout, the platform Meta Quest age-group signal remains the authoritative age source; the ML age band is a secondary safety net and never the legal basis for consent.

9.4 Closing statement

The honest balance is that the **design is complete and the backend is verified feature-by-feature against a one-command harness** (`scripts/verify-all.sh`: 342 tests, a 38/40 benchmark pinned by SHA-256, and a drift-detecting baseline — a verification campaign that itself surfaced seven latent defects at the then-current head, which is exactly why the claim is "verified feature-by-feature" and not "proven"), while what remains is **integration and measurement, not redesign**: the headless EditMode batch run, systematic real-endpoint latency, and running the differential evaluation end-to-end. The broader claim the work defends is methodological — that for end-user programming in safety-sensitive immersive settings, constraining the output language is a stronger and more deployable safety lever than sandboxing the output, and that it can be enforced by construction rather than by vigilance.

References and Appendices

References

The reference list below is ordered alphabetically by the first author or organisation and follows the Harvard (author–date) convention; for corporate, standards, and grey-literature sources the issuing organisation is treated as the author. Every source was located and confirmed to carry a real, resolvable URL or DOI, and every source cited in the running text appears here.

Begel, A. and Graham, S.L. (2006) 'An assessment of a speech-based programming environment', in *Proceedings of the IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC 2006)*. Brighton, UK: IEEE Computer Society, pp. 116–120. doi:10.1109/VLHCC.2006.9. Available at: <https://doi.org/10.1109/VLHCC.2006.9> (Accessed: 29 June 2026).

Bozkir, E. and Kasneci, E. (2025) *Exploring user acceptance and concerns toward LLM-powered conversational agents in immersive extended reality*. arXiv:2512.15343. Available at: <https://arxiv.org/abs/2512.15343> (Accessed: 29 June 2026).

Burkhardt, F., Wagner, J., Wierstorf, H., Eyben, F. and Schuller, B. (2023) *Speech-based age and gender prediction with transformers*. arXiv preprint arXiv:2306.16962. Available at: <https://arxiv.org/abs/2306.16962> (Accessed: 17 July 2026).

Casey, P., Baggili, I. and Yarramreddy, A. (2021) 'Immersive virtual reality attacks and the human joystick', *IEEE Transactions on Dependable and Secure Computing*, 18(2), pp. 550–562. doi:10.1109/TDSC.2019.2907942. Available at: <https://doi.org/10.1109/TDSC.2019.2907942> (Accessed: 29 June 2026).

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P. de O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. (2021) *Evaluating large language models trained on code*. arXiv preprint arXiv:2107.03374. Available at: <https://arxiv.org/abs/2107.03374> (Accessed: 27 June 2026).

Eskenazi, M., Mostow, J. and Graff, D. (1997) *The CMU Kids Corpus LDC97S63*. Philadelphia, PA: Linguistic Data Consortium. Available at: <https://catalog.ldc.upenn.edu/LDC97S63> (Accessed: 17 July 2026).

European Data Protection Board (2021) *Guidelines 02/2021 on virtual voice assistants. Version 2.0*. Adopted on 7 July 2021. Available at: https://www.edpb.europa.eu/our-work-tools/our-documents/guidelines/guidelines-022021-virtual-voice-assistants_en (Accessed: 28 June 2026).

European Union (2016) *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)*, Arts 9, 25 and 35. *Official Journal of the European Union* L 119, 4 May 2016, pp. 1–88. Available at: <https://eur-lex.europa.eu/eli/reg/2016/679/oj/eng> (Accessed: 28 June 2026).

European Union (2024) *Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)*, *Official Journal of the European Union*, L 2024/1689, 12 July. Available at: <https://eur-lex.europa.eu/eli/reg/2024/1689/oj> (Accessed: 17 July 2026).

Federal Trade Commission (2024) *FTC denies application for new parental consent mechanism under COPPA*. Press release, 29 March. Washington, DC: Federal Trade Commission. Available at: <https://www.ftc.gov/news-events/news/press-releases/2024/03/ftc-denies-application-new-parental-consent-mechanism-under-coppa> (Accessed: 17 July 2026).

Federal Trade Commission (2025) *Children's Online Privacy Protection Rule*. Final rule, 16 CFR Part 312. Federal Register, 90(76), 22 April, pp. 16918–16983. Available at: <https://www.federalregister.gov/documents/2025/04/22/2025-05904/childrens-online-privacy-protection-rule> (Accessed: 17 July 2026).

Friston, S.J., Congdon, B.J., Swapp, D., Izzouzi, L., Brandstätter, K., Archer, D., Olkkonen, O., Thiel, F.J. and Steed, A. (2021) 'Ubiq: a system to build flexible social virtual reality experiences', in *Proceedings of the 27th ACM Symposium on Virtual Reality Software and Technology (VRST '21)*. New York: ACM, pp. 1–11. doi:10.1145/3489849.3489871. Available at: <https://dl.acm.org/doi/10.1145/3489849.3489871> (Accessed: 29 June 2026).

Ganguli, D., Lovitt, L., Kernion, J. et al. (2022) *Red teaming language models to reduce harms: methods, scaling behaviors, and lessons learned*. arXiv:2209.07858. Available at: <https://arxiv.org/abs/2209.07858> (Accessed: 29 June 2026).

Fujita, M., Kurasaki, S. and Kanaoka, A. (2023) 'Securing cross reality: unraveling the risks of 3D object disguise on head mount display', in *Proceedings of the 13th International Conference on the Internet of Things (IoT '23)*. New York: ACM, pp. 281–286. doi:10.1145/3627050.3631570. Available at: <https://doi.org/10.1145/3627050.3631570> (Accessed: 29 June 2026).

Geng, S., Josifoski, M., Peyrard, M. and West, R. (2023) 'Grammar-constrained decoding for structured NLP tasks without finetuning', in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*. Singapore: Association for Computational Linguistics, pp. 10932–10952. Available at: <https://aclanthology.org/2023.emnlp-main.674/> (Accessed: 27 June 2026).

Geng, S., Döner, K., Wendler, C., Josifoski, M. and West, R. (2025a) *Flexible and efficient grammar-constrained decoding*. arXiv:2502.05111. Available at: <https://arxiv.org/abs/2502.05111> (Accessed: 27 June 2026).

Geng, S., Cooper, H., Moskal, M., Jenkins, S., Berman, J., Ranchin, N., West, R., Horvitz, E. and Nori, H. (2025b) *JSONSchemaBench: a rigorous benchmark of structured outputs for language models*. arXiv:2501.10868. Available at: <https://arxiv.org/abs/2501.10868> (Accessed: 29 June 2026).

Germanos, G., Kavallieros, D., Kolokotronis, N. and Georgiou, N. (2020) 'Privacy issues in voice assistant ecosystems', in *2020 IEEE World Congress on Services (SERVICES)*. Beijing: IEEE, pp. 205–212. doi:10.1109/SERVICES48979.2020.00050. Available at: <https://arxiv.org/abs/2109.01606> (Accessed: 29 June 2026).

Giunchi, D., Numan, N., Gatti, E. and Steed, A. (2024) 'DreamCodeVR: towards democratizing behavior design in virtual reality with speech-driven programming', in *2024 IEEE Conference on Virtual Reality and 3D User Interfaces (IEEE VR 2024)*. Orlando, FL: IEEE, pp. 579–589. doi:10.1109/VR58804.2024.00078. Available at: <https://discovery.ucl.ac.uk/id/eprint/10190408/> (Accessed: 27 June 2026).

Google (2024) 'Eliminating memory safety vulnerabilities at the source', *Google Online Security Blog*, 25 September. Available at: <https://security.googleblog.com/2024/09/eliminating-memory-safety-vulnerabilities-Android.html> (Accessed: 28 June 2026).

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T. and Fritz, M. (2023) 'Not what you've signed up for: compromising real-world LLM-integrated applications with indirect prompt injection', in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*. New York: ACM, pp. 79–90. doi:10.1145/3605764.3623985. Available at: <https://dl.acm.org/doi/abs/10.1145/3605764.3623985> (Accessed: 27 June 2026).

gVisor project (no date) *gVisor architecture guide*. Available at: https://gvisor.dev/docs/architecture_guide/intro/ (Accessed: 27 June 2026).

Huynh, N. and Lin, B. (2025) *Large language models for code generation: a comprehensive survey of challenges, techniques, evaluation, and applications*. arXiv:2503.01245. Available at: <https://arxiv.org/abs/2503.01245> (Accessed: 29 June 2026).

IEEE (2024) *IEEE Std 2089.1-2024: IEEE Standard for Online Age Verification*. Piscataway, NJ: Institute of Electrical and Electronics Engineers. doi:10.1109/IEEESTD.2024.10542699. Available at: <https://doi.org/10.1109/IEEESTD.2024.10542699> (Accessed: 17 July 2026).

Information Commissioner's Office (2020) *Age appropriate design: a code of practice for online services*. Wilmslow: Information Commissioner's Office. Available at: <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/childrens-information/childrens-code-guidance-and-resources/age-appropriate-design-a-code-of-practice-for-online-services/> (Accessed: 17 July 2026).

Ishii, A., Suzuki, I., Sakamoto, S., Kanai, K., Takazawa, K., Doi, H. and Ochiai, Y. (2016) 'Optical Marionette: graphical manipulation of human's walking direction', in *Proceedings of the 29th Annual ACM Symposium on User Interface Software and Technology (UIST '16)*. Tokyo: ACM, pp. 705–716. doi:10.1145/2984511.2984545. Available at: <https://doi.org/10.1145/2984511.2984545> (Accessed: 29 June 2026).

ISO/IEC (2025) *ISO/IEC 27566-1:2025 Information security, cybersecurity and privacy protection — Age assurance systems — Part 1: Framework*. Geneva: International Organization for Standardization/International Electrotechnical Commission. Available at: <https://www.iso.org/standard/88143.html> (Accessed: 17 July 2026).

Jung, R., Jourdan, J.-H., Krebbers, R. and Dreyer, D. (2018) 'RustBelt: securing the foundations of the Rust programming language', *Proceedings of the ACM on Programming Languages*, 2(POPL), Article 66. doi:10.1145/3158154. Available at: <https://dl.acm.org/doi/10.1145/3158154> (Accessed: 27 June 2026).

Krauß, V., Saeghe, P., Boden, A., Khamis, M., McGill, M., Gugenheimer, J. and Nebeling, M. (2024) 'What makes XR dark? Examining emerging dark patterns in augmented and virtual reality through expert co-design', *ACM Transactions on Computer-Human Interaction*, 31(3). doi:10.1145/3660340. Available at: <https://doi.org/10.1145/3660340> (Accessed: 29 June 2026).

Lee, H., Lee, J., Kim, D., Jana, S., Shin, I. and Son, S. (2021) 'AdCube: WebVR ad fraud and practical confinement of third-party ads', in *Proceedings of the 30th USENIX Security Symposium (USENIX Security '21)*. USENIX Association, pp. 2543–2556. Available at: <https://www.usenix.org/conference/usenixsecurity21/presentation/lee-hyunjoo> (Accessed: 29 June 2026).

Meta (2024) *Ensuring age-appropriate experiences on Meta Quest*. Meta Newsroom, 2 April. Available at: <https://about.fb.com/news/2024/04/ensuring-age-appropriate-experiences-on-meta-quest/> (Accessed: 17 July 2026).

MITRE (2024) *CVE-2024-21626: runc internal file-descriptor leak enabling container escape via a host-filesystem working directory ('Leaky Vessels')*. National Vulnerability Database (NVD). Available at: <https://nvd.nist.gov/vuln/detail/CVE-2024-21626> (Accessed: 29 June 2026).

MITRE (2026) *CWE-94: improper control of generation of code ('code injection')*. The MITRE Corporation. Available at: <https://cwe.mitre.org/data/definitions/94.html> (Accessed: 28 June 2026).

Nair, V., Rack, C., Guo, W., Wang, R., Li, S., Huang, B., Cull, A., O'Brien, J.F., Latoschik, M., Rosenberg, L. and Song, D. (2023) *Inferring private personal attributes of virtual reality users from head and hand motion data*. arXiv preprint arXiv:2305.19198. Available at: <https://arxiv.org/abs/2305.19198> (Accessed: 17 July 2026).

National Institute of Standards and Technology (NIST) (2017) *Application container security guide* (SP 800-190), by Souppaya, M., Morello, J. and Scarfone, K. Gaithersburg, MD: NIST. Available at: <https://nvlpubs.nist.gov/nistpubs/specialpublications/nist.sp.800-190.pdf> (Accessed: 27 June 2026).

Nowrin, S. and Vertanen, K. (2023) 'Programming by voice: exploring user preferences and speaking styles', in *Proceedings of the 5th ACM Conference on Conversational User Interfaces (CUI '23)*. Eindhoven: ACM. doi:10.1145/3571884.3597130. Available at: <https://doi.org/10.1145/3571884.3597130> (Accessed: 29 June 2026).

Open Worldwide Application Security Project (OWASP) (no date a) *Code injection*. Available at: https://owasp.org/www-community/attacks/Code_Injection (Accessed: 27 June 2026).

Open Worldwide Application Security Project (OWASP) (no date b) *Direct dynamic code evaluation ('eval injection')*. Available at: https://owasp.org/www-community/attacks/Direct_Dynamic_Code_Evaluation_Eval%20Injection (Accessed: 27 June 2026).

OWASP Gen AI Security Project (2025a) *OWASP top 10 for LLM applications 2025*. Available at: <https://genai.owasp.org/llm-top-10/> (Accessed: 27 June 2026).

OWASP Gen AI Security Project (2025b) *LLM01:2025 prompt injection*. Available at: <https://genai.owasp.org/llmrisk/llm01-prompt-injection/> (Accessed: 27 June 2026).

Pane, J.F., Ratanamahatana, C.A. and Myers, B.A. (2001) 'Studying the language and structure in non-programmers' solutions to programming problems', *International Journal of Human-Computer Studies*, 54(2), pp. 237–264. doi:10.1006/ijhc.2000.0410. Available at: <https://doi.org/10.1006/ijhc.2000.0410> (Accessed: 28 June 2026).

Pearlman, K. (2020) *Virtual reality brings real risks: are we ready?* USENIX Enigma 2020, San Francisco, CA. Available at: <https://www.usenix.org/conference/enigma2020/presentation/pearlman> (Accessed: 29 June 2026).

Perez, E., Huang, S., Song, F., Cai, T., Ring, R., Aslanides, J., Glaese, A., McAleese, N. and Irving, G. (2022) 'Red teaming language models with language models', in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP 2022)*. Abu Dhabi: Association for Computational Linguistics, pp. 3419–3448. doi:10.18653/v1/2022.emnlp-main.225. Available at: <https://aclanthology.org/2022.emnlp-main.225/> (Accessed: 29 June 2026).

Poesia, G., Polozov, A., Le, V., Tiwari, A., Soares, G., Meek, C. and Gulwani, S. (2022) 'Synchromesh: reliable code generation from pre-trained language models', in *Proceedings of the 10th International Conference on Learning Representations (ICLR 2022)*. Available at: <https://arxiv.org/abs/2201.11227> (Accessed: 27 June 2026).

Scholak, T., Schucher, N. and Bahdanau, D. (2021) 'PICARD: parsing incrementally for constrained auto-regressive decoding from language models', in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP 2021)*. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, pp. 9895–9901. doi:10.18653/v1/2021.emnlp-main.779. Available at: <https://aclanthology.org/2021.emnlp-main.779/> (Accessed: 28 June 2026).

Su, Z., Shezan, F.H., Tian, Y., Evans, D. and Heo, S. (2022) 'Perception hacking for 2D cursorjacking in virtual reality', in *CHI 2022 Workshop on Novel Challenges of Safety, Security and Privacy in Extended Reality (SSPXR)*. New Orleans, LA. (Workshop paper; no archival DOI.)

Trivial Interactive (no date) *Roslyn C# – runtime compiler*. Unity Asset Store. Available at: <https://assetstore.unity.com/packages/tools/integration/roslyn-c-runtime-compiler-142753> (Accessed: 28 June 2026).

Unity Technologies (2024a) *Scripting restrictions*. Unity Manual. Available at: <https://docs.unity3d.com/6000.0/Documentation/Manual/scripting-restrictions.html> (Accessed: 27 June 2026).

Unity Technologies (2024b) *Introduction to IL2CPP*. Unity Manual. Available at: <https://docs.unity3d.com/6000.4/Documentation/Manual/il2cpp-introduction.html> (Accessed: 27 June 2026).

Tseng, W.-J., Bonnal, E., McGill, M., Khamis, M., Lecolinet, E., Huron, S. and Gugenheimer, J. (2022) 'The dark side of perceptual manipulations in virtual reality', in *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems (CHI '22)*. New Orleans, LA: ACM. doi:10.1145/3491102.3517728. Available at: <https://doi.org/10.1145/3491102.3517728> (Accessed: 29 June 2026).

Valluripally, S., Gulhane, A., Mitra, R., Hoque, K.A. and Callyam, P. (2020) 'Attack trees for security and privacy in social virtual reality learning environments', in *2020 IEEE 17th Annual Consumer Communications & Networking Conference (CCNC)*. Las Vegas, NV: IEEE. doi:10.1109/CCNC46108.2020.9045724. Available at: <https://arxiv.org/abs/1911.03563> (Accessed: 29 June 2026).

Wang, X., Du, J. and Liu, H. (2022) 'Performance and isolation analysis of runC, gVisor and Kata Containers runtimes', *Cluster Computing*, 25(2), pp. 1497–1513. doi:10.1007/s10586-021-03517-8. Available at: <https://doi.org/10.1007/s10586-021-03517-8> (Accessed: 29 June 2026).

Yigitbas, E., Klauke, J., Gottschalk, S. and Engels, G. (2022) 'End-user development for interactive web-based virtual reality scenes', *Journal of Computer Languages*, 71, 101140. Available at: <https://www.sciencedirect.com/science/article/pii/S2590118422000843> (Accessed: 27 June 2026).

Young, E.G., Zhu, P., Caraza-Harter, T., Arpaci-Dusseau, A.C. and Arpaci-Dusseau, R.H. (2019) 'The true cost of containing: a gVisor case study', in *Proceedings of the 11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud '19)*. Renton, WA: USENIX Association. Available at: <https://www.usenix.org/conference/hotcloud19/presentation/young> (Accessed: 27 June 2026).

Zhang, L. and Oney, S. (2020) 'FlowMatic: an immersive authoring tool for creating interactive scenes in virtual reality', in *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology (UIST '20)*. New York: ACM, pp. 342–353. doi:10.1145/3379337.3415824. Available at: <https://doi.org/10.1145/3379337.3415824> (Accessed: 28 June 2026).

Zhang, Q., Fang, C., Xie, Y., Zhang, Y., Yang, Y., Sun, W., Yu, S. and Chen, Z. (2023) *A survey on large language models for software engineering*. arXiv:2312.15223. Available at: <https://arxiv.org/abs/2312.15223> (Accessed: 29 June 2026).

Zheng, X., Wan, Z., Zhang, Y., Chang, R. and Lo, D. (2024) 'A closer look at the security risks in the Rust ecosystem', *ACM Transactions on Software Engineering and Methodology*, 33(2), Article 34, pp. 1–30. doi:10.1145/3624738. Available at: <https://doi.org/10.1145/3624738> (Accessed: 29 June 2026).

Grey-literature illustration (cited only as a real-world incident)

- *microsoft/autogen* (2024) *Code injection risk due to unsafe usage of eval — issue #4873*. GitHub. Available at: <https://github.com/microsoft/autogen/issues/4873> (Accessed: 27 June 2026). (Cited only as a concrete, real-world illustration of the eval-injection class in an LLM agent framework; flagged in text as a grey-literature source, not a peer-reviewed claim.)

Integrity note. Every entry in this reference list was located and verified at its canonical source — the publisher, the ACL Anthology, the relevant standards body or institution, or a DOI resolver — and no author, title, venue, DOI, or date has been invented. Every source cited in the running text appears here, and no citation gaps remain.

Appendices

The appendices provide the reproducibility, transparency, and audit material that the main body references but does not embed in full. Each appendix is sourced verbatim or by faithful extraction from the verified project artefacts; the provenance is stated in every entry. Material that depends on results not yet collected (for example, quantitative red-team bypass rates, an executed Unity play-test, or a completed DPIA) is included only as a clearly-labelled template or as material identified as outstanding, consistent with the honest-status policy applied throughout this dissertation. All paths are given relative to the repository root unless stated otherwise.

Appendix A — Action-Plan v1 JSON Schema

Purpose. Present a JSON Schema for the action-plan intermediate representation — derived from the Rust data model that is the system's safety source of truth — so that the IR can be independently inspected and re-implemented. (The schema is a *description* of the IR; enforcement is the Rust validator of Appendix B, not a shipped schema file.)

Source (verified). The canonical, authoritative definition of the IR is the Rust data model itself — `crates/behaviour-dsl/src/actions.rs` (the six-variant, internally-tagged `Action` enum and the `deny_unknown_fields` `ActionPlan` envelope) and the bounds module `crates/behaviour-dsl/src/bounds.rs`. A standalone JSON Schema file is *not* shipped in the repository; the schema reproduced in this appendix is derived from, and pinned to, that Rust data model (enforcement is the Rust validator, Appendix B, not a separate schema document).

Contents. - A1. The complete JSON Schema document (Draft schema, top-level `ActionPlan` object: `schema_version`, `request_id`, `target`, `actions`). - A2. A short annotated key, noting which schema constructs implement which safety property — in particular: the closed `actions` array, the internally-tagged action objects, and the absence of any catch-all/additional action type (mirroring the Rust enum's six allow-listed variants `set_color`, `set_scale`, `move`, `rotate`, `spawn_primitive`, `set_physics`). - A3. **Honest scope note (carried from the engineering record):** the Rust struct enforces `#[serde(deny_unknown_fields)]` at the envelope and action-tag level; this appendix states explicitly that `deny_unknown_fields` is enforced at parse time in Rust and notes the documented limitation that unknown keys *inside* an action variant are not rejected by the schema alone (decision record D9), so the JSON Schema is a description of the IR, not the sole enforcement point — enforcement is the Rust validator (Appendix B).

Appendix B — Full validator rule set and bounds table

Purpose. Document, exhaustively, the fail-closed validation logic and every numeric/structural bound the validator enforces, so a reader can reconstruct exactly what makes a plan "approved" versus "rejected."

Source (verified). `crates/code-policy/` (the `validate_json/validate_plan` logic; approve-iff-zero-violations; 17 tests in `code-policy/tests/validation_tests.rs`) and `crates/behaviour-dsl/src/bounds.rs` (single source of truth for all limits). The Rust↔Unity bounds-parity guard is `tests/tests/bounds_parity.rs`.

Contents. - B1. **Validation pipeline (prose + flow):** size cap checked *before* parsing (anti-allocation DoS), then parse, then `validate_plan`, accumulating a `Vec<ValidationError>`; decision is `ApproveActionPlan` **iff** the violation set is empty, else `RejectUnsafe`. The fail-closed default (any error/timeout ⇒ reject) is stated. - B2. **Bounds table (verbatim from `bounds.rs`):**

Constant	Value	Meaning
MAX_ACTIONS	16	maximum actions per plan
MAX_SPAWN_COUNT	8	maximum primitives per single <code>spawn_primitive</code>
MAX_TOTAL_SPAWNED_PER_SESSION	64	cumulative spawn budget per session
scale range	0.1 – 5.0	<code>set_scale</code> bounds
speed / amplitude	≤ 5	<code>move</code> bounds
rotate	$\leq 360^\circ/\text{s}$	<code>rotate</code> bound
mass	0.1 – 100	<code>set_physics</code> bound
MAX_PLAN_JSON_BYTES	64 KiB	size cap enforced before parse
hex colour	<code>is_valid_hex_color</code>	<code>set_color</code> format check

- **B3. Per-action rule listing:** the specific checks per variant, including the `NonFiniteValue` guard rejecting NaN/Inf.
- **B4. The two-level spawn budget:** the per-plan cap (in `code-policy`) and the cumulative session cap enforced at the single `command-router::commit` chokepoint, including the documented `Approve→Reject downgrade (session_spawn_budget_exceeded)`.
- **B5. Bounds-drift guard:** a note that `tests/tests/bounds_parity.rs` fails `cargo test` if the Rust bounds and the hand-mirrored `Unity ProtocolModels.cs` constants diverge, together with the honest caveat that the Unity mirror is maintained by hand.

Appendix C — Red-team C# corpus and results

Purpose. Present the adversarial C# corpus used to probe the Mode-B C# safety layers and record the outcomes, separating what has been verified from what remains to be measured.

Source (verified). `fixtures/csharp/` (`valid_set_color.cs`, `banned_network_exfil.cs`, `inconsistent_spawns.cs` — all confirmed present) and `crates/csharp-policy/tests/` (`csharp_tests.rs`, `redteam_tests.rs` — the evasion suite). The lexical scanner (`crates/csharp-policy`, tree-sitter-based, dotted-name reconstruction defeating `whitespace/global::/@/comment` tricks) and the .NET semantic analyzer (`services/roslyn-analyzer`) are the systems under test.

Contents. - **C1. Corpus inventory:** each fixture and red-team case, the evasion technique it represents (whitespace insertion, `global::` prefixing, `@-verbatim` identifiers, comment interleaving), and the banned namespace/identifier it targets. - **C2. Verified results (real, dated 2026-06-27):** the live defence-in-depth finding — the .NET semantic analyzer returned a **false negative** on `new System.Net.WebClient` (because it references only `corelib`, so the non-`corelib` `System.Net.Requests` symbol fails to resolve and is skipped), and the **Rust lexical backstop catches it** (`redteam_tests.rs` rejects `new System.Net.Http.HttpClient`; `csharp_tests.rs` asserts `BannedToken("System.Net")`). Because the lexical scan runs first in the dual path, `System.Net` is rejected before Roslyn is consulted. This appendix presents that as the worked example of *why* C# safety is layered (lexical + semantic + consistency + sandbox), not a single gate. - **C3. Test-suite status:** the red-team suite and the privacy leak-test pass within the green workspace gate (342 Rust tests total). - **C4. Quantitative bypass-rate results (executed, 2026-07-03).** The red-team campaign was run, not merely specified. `redteam/corpus_gen.py` emits a reproducible 1057-vector corpus (727 C# fired at `POST /api/validate`, the tree-sitter lexical validator, plus 330 natural-language fired at `POST /api/command`); the generator is deterministic, so the campaign is repeatable. The 727 C# vectors comprise 258 malicious "should-block" cases and 469 benign "creative-freedom" cases (including scary-looking-but-safe names — a cube named "TrojanHorse", a sphere "seashell", a minimap Camera). `redteam/run_campaign.py` scores each vector against ground truth, counting

a should-block that is approved as a BYPASS and a benign that is blocked as an OVER-BLOCK. Before the evasion classes below were closed an unadorned lexer caught only **36 of 258 (14.0%)** should-block C# vectors — the 222 vectors in the three classes all bypassed; this before-figure is reconstructed from the corpus composition rather than reproduced, since the delivered repository ships the already-hardened validator that the reproducible campaign scores at 258/258 from the outset (with all 469 benign vectors approved, 0 over-blocks). The 222 bypasses reduced to exactly three evasion classes the lexer had missed — namespace-alias reconstruction (206 vectors), Unicode-escape identifiers (9), and `dynamic` late-binding dispatch (7), which sum to 222. Two further classes surfaced by the multi-agent audit were closed the same way (an `unsafe {}` pointer block and a `MonoBehaviour-impostor` base type) but are *not* part of the 1,057-vector corpus. After the fixes in `crates/csharp-policy/src/lexical.rs` — an alias-resolution pre-pass, `\uXXXX/\UXXXXXXXX` identifier normalisation, and an outright ban on `dynamic` — the 727-vector corpus scores 258/258 attacks caught (100%), 0 bypasses, and 469/469 benign still approved (0 over-blocks). The whole C# corpus fires in ~1 s (pure lexical, no LLM). The five classes are locked by eight regression tests; the `csharp-policy` crate passes 54 unit tests — (Verified by automated tests.).

Appendix D — Reproduction commands

Purpose. Give the exact, dated commands needed to reproduce every live verification, so that the empirical claims in the Results chapter are independently checkable.

Source (verified). The one-command regression harness `scripts/verify-all.sh` and its recorded baseline `scripts/verify-baseline.txt` (`tests_passed=342`, `attacks_blocked=38`, `ml_tests=82`, `campaign_vectors=1057`, `results_sha256=...`, `rustc=1.96.0`), together with `scripts/check.sh`, `scripts/sandbox-run-docker.sh` and `scripts/sandbox-run-gvisor.sh`; the environment of record is the one under which the gate was first run (Appendix D1).

Contents. - D1. **Environment of record (2026-06-27):** Ubuntu 26.04 LTS (kernel 7.0); `rustc/cargo 1.96.0`; `Node v24.18.0 (nvm)`; `.NET 10.0.109`; `Docker 29.6.1`; `cargo-deny 0.19.9` (delivered gate); `cargo-audit` additionally available on this machine; **`runsc/gVisor` was not installed** — the `runsc` sandbox path is scripted (`scripts/sandbox-run-gvisor.sh`) but was not exercised; `Unity Hub + Editor 2021.3.16f1` installed (requires the `libxml2.so.2 → libxml2.so.16` shim on 26.04). - D2. **Workspace gate:** `cargo fmt --all -- --check · cargo clippy --workspace --all-targets -- -D warnings · cargo test --workspace (342 tests) · cargo build --workspace · cargo deny check` (the delivered gate; `cargo audit` was additionally available on the Linux machine of record, D1). - D3. **Real Ubiq roundtrip (Mode C path):** vendoring and starting the real Node RoomServer (`:8009`), then the full loop (`DCVR_UBIQ_ADDR=... dreamcodevr-server + fake-quest-client --ubiq ...`) yielding NID 94 `ApproveActionPlan{set_color #FF0000}`, plus `ubiq-probe`. **Note the dead `gitpkg.now.sh` dependency** (402 Payment Required) that necessitated the one-time GitHub vendoring. - D4. **Mode B (validated C#):** starting `services/roslyn-analyzer (:5099)`, the live `curl /analyze` matrix, and the dual path under `DCVR_CSHARP_RESEARCH=true DCVR_ROSLYN_URL=...`. - D5. **Mode D (sandbox) under Docker:** `docker build -t dcvr-sandbox-harness:local services/sandbox-worker`; `sandbox-runner` (shell containment) and `sandbox-runner --csharp` (real C# harness). The `gVisor` path is a configuration step, not a code change: setting `DCVR_SANDBOX_DOCKER_RUNTIME=runsc` (equivalently `--runtime=runsc`) selects the wired, unit-tested `runsc` runtime on any host that has it installed; this was not run here, as no `runsc` host was available. - D6. **Environment variables** (all optional; unset $\Rightarrow$ offline mocks): `DCVR_LISTEN_ADDR`, `DCVR_UBIQ_ADDR`, `DCVR_ROOM_GUID`, `DCVR_MODE`, `DCVR_STT_HTTP_URL`, `DCVR_STT_TIMEOUT_MS`, `OPENAI_API_KEY`, `OPENAI_MODEL`, `OPENAI_BASE_URL`, `DCVR_LLM_TIMEOUT_MS`, `DCVR_CSHARP_RESEARCH`, `DCVR_ROSLYN_URL`. - D7. **Honest reproduction caveats:** the real OpenAI LLM is exercised when a key is present in `.env` (mocks are the default); live-microphone STT and on-device Quest are not reproducible without hardware; `Unity play-mode` is reproducible in the interactive Unity 6 Editor (see Appendix E). - D8. **Red-team campaign (executed 2026-07-03):** Regenerate the corpus and re-fire it against the running backend with two commands. First build the reproducible 1057-vector corpus:

```
python3 redteam/corpus_gen.py redteam/corpus.json
```

Then score each layer against ground truth:

```
python3 redteam/run_campaign.py --layer csharp
python3 redteam/run_campaign.py --layer nl
```

`--layer csharp` fires the 727 C# vectors at `POST /api/validate` (the tree-sitter lexical validator); `--layer nl` fires the 330 natural-language vectors at `POST /api/command` (the Layer-1 intent screen plus generation). The harness writes per-vector verdicts to `redteam/results_*.json` and reports counts of BYPASS (should-block approved) and OVER-BLOCK (benign blocked) scored against the corpus ground truth. Because `corpus_gen.py` is deterministic, the corpus and the scores are reproducible run-to-run — (Executed 2026-07-03.).

Appendix E — Unity integration and play-test checklist

Purpose. Provide the step-by-step procedure to wire and play-test the Mode-C Unity client, and to record that this integration is **verified in the interactive Unity 6 Editor** (Mode C and validator-gated Mode A); only headless batchmode EditMode tests remain blocked on a licensing sign-in.

Source (verified). The drop-in scripts under `unity/Runtime/` (and `unity/Runtime/Security/`) and the EditMode tests under `unity/Editor/`; the backend launch scripts (`scripts/run-backend.sh`, `scripts/run-roomserver.sh`) that a live play-test connects to. The headless-batch licensing block noted below was observed directly, not from a stored log.

Contents. - E1. **Integration steps:** add the `unity/Runtime` scripts; register `ActionPlanNetworkBridge` on `UbiqNetworkId(94)` and **remove the original `CodeGenerationManager` from NID 94** (do not register both); wire `SelectRay/executor` references; point the Unity Ubiq config at the backend's room. - E2. **Headless EditMode procedure (Mode-C logic proof, no VR):** the documented `libxml2.so.2 → libxml2.so.16` shim, `LD_LIBRARY_PATH`, and the `-batchmode -nographics -runTests -testPlatform EditMode` invocation, with the four EditMode test fixtures to be exercised (23 [Test] methods in all; the canonical four are `AcceptsSetColor → red`, `ClampsOversizeScaleDefensively`, `RejectsInvalidJson`, `RejectsEmptyActions`). - E3. **Full VR play-test checklist:** GUI + headset run of the speak→point→object-changes loop. - E4. **Honest status (verbatim posture):** the scripts are *staged/drop-in* and **not yet play-tested**; headless batchmode currently fails at **licensing** (`Access token is unavailable`) before any compile/test, because the Editor is installed but **not signed in**. This is an account step the automation cannot perform; it is **not** a fault in the DreamCodeVR+ scripts. All play-test outcomes remain pending a licensed Editor and a Quest device.

Appendix F — gVisor / Docker sandbox configuration

Purpose. Document the exact container-hardening configuration for Mode D under plain Docker (the verified boundary), and the wired-but-not-yet-exercised gVisor (`runsc`) runtime, so the isolation claims can be audited against NIST SP 800-190 guidance.

Source (verified). `crates/sandbox/src/runner.rs` (`DockerSandboxRunner`, including the hermetically unit-tested `build_args`); `services/sandbox-worker/` (`Dockerfile`, two-stage `dotnet/sdk:10.0 → dotnet/runtime:10.0`, `USER 65534`, `stdin-fed harness`; image `dcvr-sandbox-harness:local`); `scripts/sandbox-run-docker.sh` and `scripts/sandbox-run-gvisor.sh` (the latter checked in with a "not yet verified" status note); the Docker containment results were observed by running `sandbox-run-docker.sh` directly.

Contents. - F1. **Hardening flag set:** `--network none`, `--read-only + tmpfs /tmp`, `--cap-drop ALL`, `--pids-limit`, `--memory/ --cpus`, `no-new-privileges`, `--user 65534:65534`, `--rm`, wall-clock timeout enforced by the

runner with `docker kill` on timeout; report-only crossing (only a structured `SandboxReport` returns). - F2. **Harness image:** the two-stage Dockerfile, in-memory Roslyn compile-and-execute referencing all runtime assemblies (so it works offline under `--network none`), and the exit-code convention (64 empty / 65 compile-error / 66 no-entry / 70 runtime-exception). - F3. **Verified containment results (real, Docker):** the Docker table (benign `Completed 0 uid=65534; READONLY_BLOCKED; NETWORK_BLOCKED; TimedOut → docker kill`) and the real-C# table (`File.WriteAllText("/etc/pwned") → FS_BLOCKED: IOException; runtime exception caught; infinite loop TimedOut`). The gVisor (`runsc`) runtime is wired into the runner and unit-tested but was **not** exercised (no `runsc` host was available), so no gVisor containment results are claimed. - F4. **Honest boundary statement (per NIST SP 800-190):** plain Docker is a deliberately *soft* boundary; gVisor interposes an application kernel for stronger isolation; the Rust runner selects it through `with_runtime / DCVR_SANDBOX_DOCKER_RUNTIME=runsc`, unit-tested at `runner.rs:403`, defaulting to none. Mode D is verified under **Docker**; the gVisor (`runsc`) boundary is wired and unit-tested but was not run for lack of a `runsc` host, so kernel-level isolation is a designed, not a measured, property.

Appendix G — AI-assistance declaration template (per university policy)

Purpose. Provide a completed-on-submission declaration of how generative-AI tools were used in preparing the dissertation and the underlying artefact, in line with University of Birmingham academic-integrity requirements for declaring AI assistance.

Source. The University of Birmingham policy on the use of generative AI tools in assessment, to be cited by the author from the current programme handbook (with the exact title and reference confirmed at submission). The declaration wording is completed and signed by the author.

Contents. - G1. **Declaration statement (completed by the author):** the University-mandated wording confirming that the work is the author's own and that AI assistance is declared as set out below. - G2. **Itemised AI-use table (template):** for each use — tool/model and version; date(s); the task (e.g. drafting prose, summarising literature located by the author, code scaffolding, editorial polishing); the scope and extent; and how the output was checked and edited by the author. with per-item entries completed by the author. - G3. **Explicit exclusions:** a statement that **no results, measurements, citations, or user-study data were generated by AI**, consistent with the integrity policy enforced throughout this dissertation: quantitative results not yet collected are identified explicitly as future work, and every citation in the reference list is confirmed against a resolvable source. - G4. **Signature and date block:** author name, student ID, signature, and date, completed by the author.

Reproducibility statement. Appendices A, B, D, and F are extracted directly from artefacts that ship in the repository (`crates/behaviour-dsl/src/actions.rs` and `.../bounds.rs`, `crates/code-policy/, scripts/verify-all.sh` and `scripts/verify-baseline.txt`, `scripts/sandbox-run-docker.sh / sandbox-run-gvisor.sh`, `crates/sandbox/, services/sandbox-worker/`). Appendix C is extracted from `fixtures/csharp/` and `crates/csharp-policy/tests/`, with the archived pre-hardening bypass rate identified as not reproducible from the delivered (already-hardened) code. Appendices E and G are templates/checklists whose outcomes (Unity play-test results; the final AI-assistance declaration; the DPIA referenced in the ethics chapter) are completed only when the corresponding step is performed, and are presented honestly as pending where they are not yet done. The action-plan JSON Schema of Appendix A is derived from the Rust data model rather than a shipped schema file, and the gVisor containment results of Appendix F are Docker-only (the `runsc` runtime is wired and unit-tested but was not run).